Machine Learning for Theorem Prover Strategies

Chaitanya Mangla



Department of Computer Science and Technology
Fitzwilliam College

December 2024

This dissertation is submitted for
the degree of Doctor of Philosophy

Declaration

This thesis is the result of my own work and includes nothing which is the outcome of work done in collaboration except as declared in the preface and specified in the text. It is not substantially the same as any work that has already been submitted, or, is being concurrently submitted, for any degree, diploma or other qualification at the University of Cambridge or any other University or similar institution except as declared in the preface and specified in the text. It does not exceed the prescribed word limit for the relevant Degree Committee.

This dissertation does not exceed the regulation length of 60 000 words, including tables and footnotes.

Abstract

Theorem provers have wide-ranging applications, including formal verification of large mathematical proofs [123] and reasoning in knowledge-bases [61]. Optimizations that generate more successful proofs and decrease proof-discovery time are desirable. Machine Learning (ML) offers provers the opportunity to learn from previous proof attempts and apply that knowledge for improved successive attempts. In this work, we develop methods to improve theorem provers, by means of ML.

The broad research question we explore in this work is as follows.

> How can theorem provers utilize ML techniques, to develop more efficient methods of proof discovery, and to expand their capabilities?

We advance along this line of enquiry by developing a series of complementary methods. We have implemented these methods for the purposes of detailed analysis and experimentation. Certain core concepts in our implementation are original and useful in related research. Along with the answers we find through a detailed analysis of our results, we present tests of numerous hypotheses derived from our research question.

We focus this work on the concept of strategies in theorem provers. Provers employ strategies to adapt their actions to specific categories of theorems, and by leveraging a diverse arsenal of strategies, they enhance their success rate across a wide range of theorems. The sequencing of such strategies is called *strategy scheduling* — the topic this work narrows into. We develop feature-vector representations of theorems for this purpose, then examine an effective method for strategy optimization. Eventually, we develop an ML method for strategy scheduling, building it from the ground up. Tested on the Thousands of Problems for Theorem Provers problem library, our method yields a time saving of more than 50%. Our methods are transferable across theorem provers.

Acknowledgments

I would like to foremost thank my supervisors, Prof. Lawrence C. Paulson FRS and Dr. Sean B. Holden, for their enduring support, confidence and patience. The completion of this work has been a stringent test of perseverance on all our parts, for which they have far exceeded the expected norms. Core elements of this work stem from their ideas, and my technical progress would have been impossible without their continuous assistance. They have helped me produce work that exceeds what I could have expected of myself, both in ambition and quality. I would also like to thank Prof. Mateja Jamnik for co-supervising the initial years of my study.

I would like to thank my friends and family for their support, encouragement and frequent practical assistance. My parents have consistently motivated me through their unwavering pride and constant concern for my future. Dr. Sneh Lata Mangla, my mother, has been an inspiration to me, as she is thus far the only doctoral graduate in my immediate family. My father, Kamal Mangla, is a driving force behind my technical inclination. I absorbed his enthusiasm for science and technology, and my early exposure to computer science is due to him. My brother, Shashank Mangla, has supported me all throughout, but particularly so in the most difficult of times. I am grateful to my close friend, Dr. Bart Andrews, for proofreading parts of this thesis. I must also thank my partner, Shreya, for her review of the final pieces of this text.

I am grateful to the community of scholars of my subjects, who have been welcoming and supportive. I express my gratitude to the Engineering and Physical Sciences Research Council (EPSRC) for their generous funding.

Summary of Notation

We have written this text with notation that is standard in the research community. However, there are some variations in such literature, and thus we clarify our conventions by means of the examples below. The language is as per Cambridge English, except for the use of "iz" spelling in words such as "normalization".

Example	Description
A	A random variable, constant or else — discerned by context.
a	A scalar variable, always in lowercase.
$\Pr(A = a)$	Probability that random variable A takes value a.
$\mathbb{P}(A)$	Probability of discrete event A.
$\epsilon \sim \mathcal{N}(0, \sigma)$	Distribution of a random variable, i.e., $\Pr(E = \epsilon) = \mathcal{N}(\epsilon; 0, \sigma)$.
$\boldsymbol{X}$	A matrix, always in capitals and boldfaced.
$\boldsymbol{X}^\intercal$	The transpose of matrix $\boldsymbol{X}$.
$\boldsymbol{x}$	A vector, in lowercase and bold.
$1:n$	Vector $[1, \ldots, n]^\intercal$.
$[x_i]_{i \in \{1,\ldots,n\}}$	Vector $[x_1, \ldots, x_n]^\intercal$.
$[x_i]_{i=1:n}$	Vector $[x_1, \ldots, x_n]^\intercal$.
$\overline{x}$	The mean value of all x_i.
$[a; b; c]$	Vector $[a, b, c]^T$. The colon starts a new row.
$\boldsymbol{\lambda} \in \mathbb{R}^M_{>0}$	Vector $\boldsymbol{\lambda}$ has M dimensions, all real-valued and greater than zero.
$[x, y, z]$	A list containing objects x, y and z.
$x \leftarrow 1$	Variable x is assigned the value 1.
$10\,\mathrm{s}$	Ten seconds.

Contents

1 Introduction **11**

1.1 Automated Theorem Proving (ATP) 11

1.1.1 Applications of ATP 13

1.1.2 Methods for ATP 21

1.2 Machine Learning 26

1.2.1 Types of Machine Learning 27

1.2.2 Bayesian Learning 30

1.3 Machine Learning for ATP 31

1.3.1 Heuristics in Theorem Provers 32

1.3.2 Machine Learning for Automatic Theorem Proving 33

1.3.3 Machine Learning for Interactive Theorem Proving 35

1.4 This Thesis 36

2 Feature Engineering **37**

2.1 Background 39

2.1.1 Dimensionality Reduction 39

2.1.2 Maximal Information Coefficient 40

2.1.3 Coefficient of Variation 42

2.2 Features of the Goal Theorems 42

2.2.1 Extracting Features of TPTP Theorems 42

2.2.2 Strategies in the LeanCoP Prover 43

2.2.3 Judging Features 45

2.2.4 Characteristic Subsets 47

2.3 Evaluation of the Features . . . 47
2.3.1 Symbol Counts as Features . . . 48
2.3.2 Quantifier-based Features . . . 49
2.3.3 Function-based Features . . . 51
2.3.4 Significance Tests . . . 52
2.4 Related Work . . . 55
2.4.1 Comparision with the iLeanCoP prover . . . 55
2.4.2 Comparison with alternative features . . . 56
2.4.3 Automatic Feature Extraction . . . 58
2.5 Conclusion . . . 58

3 Optimizing Prover Strategies **61**
3.1 Bayesian Optimization . . . 62
3.2 Optimizing Premise Selection . . . 65
3.3 Optimizing Embedded Solvers . . . 68
3.4 Related Work . . . 75
3.5 Conclusion . . . 75

4 Run-time Strategy Scheduling **79**
4.1 Background . . . 81
4.1.1 Zero-phase Component Analysis . . . 81
4.1.2 Gaussian Approximation . . . 82
4.1.3 Markov chain Monte Carlo Sampling . . . 83
4.2 Methodology . . . 84
4.2.1 Distribution of Permutations . . . 84
4.2.2 A Maximum Likelihood Model . . . 85
4.2.3 Bayesian Inference . . . 87
4.2.4 Sampling . . . 88
4.2.5 Basis Expansion . . . 90
4.2.6 Model Selection and Time Allocations . . . 91
4.2.7 Training Data and Feature Extraction . . . 92
4.2.8 Experiments . . . 93

4.3 Results 94
4.3.1 Experiment 1: Optimizing Proof Attempt Time 94
4.3.2 Experiment 2: Proving More Theorems 95
4.4 Related Work 96
4.5 Conclusions 97

5 Discussion and Conclusions **99**

1

Introduction

> Science is what we understand well enough to explain to a computer. Art is everything else we do.
>
> *Donald E. Knuth* [25]

In this chapter, we discuss aspects of theorem proving and machine learning relevant to our work, and conclude with a brief overview of our contributions.

1.1 Automated Theorem Proving (ATP)

Questions about mechanization of mathematics have intrigued scholars for a long time [20]. Some notable developments in mechanization of modern mathematics came due to Hilbert in the 1900s. His excellent work was accompanied by his brilliant ability to pose and curate interesting unsolved problems that captivated mathematicians (see Hilbert's 23 problems [1]). In 1928, Hilbert posed the *Entscheidungsproblem*, which asked if it was possible to mechanically resolve every question in mathematics. Answers in the negative soon came from several mathematicians.

In 1931, Kurt Gödel published a paper that established two of his famous theorems, know as the *first incompleteness theorem* and the *second incompleteness theorem* [50]. In simplified terms, the first theorem states that in a consistent theory that is capable of formalizing a sufficient amount of elementary mathematics, there is a statement in the language of the theory that can neither be proved nor disproved [116]. The second theorem showed that if such a formalized theory is consistent — meaning that there

is no statement in the language of the theory that is both true and false — then the consistency of the system cannot be established within the system itself [50].

Soon after, in 1936, Alonozo Church and Alan Turing independently published results that provided answers (in the negative) to Hilbert's Entscheidungsproblem by means of lambda calculus and Turing machines respectively. Independently, they proved that there existed problems that could not be mechanically solved; and that complete mechanization of mathematics is not possible. Problems that cannot be solved mechanically are referred to as *undecidable* problems. A theory is decidable if a decision procedure exists that can prove arbitrary statements in the language of the theory. First-order logic (discussed in Section 1.1.2) is an example of an undecidable theory, whereas the first-order theory of Euclidean geometry is decidable [33]. Even in decidable theories, the question of *complexity* arises, which constrains what can be mechanically proved with limited resources.

Incompleteness and undecidability of formalized theories place upper bounds on what is possible to achieve from the mechanized formalization of mathematics. However, the incompleteness theorems are widely misinterpreted, and often taken to imply that mathematics cannot be formalized [116]. It is often possible to mechanically (or computationally) reason within approximations of undecidable theories. Although fundamentally limited, such reasoning can still be useful. It is possible to chip away useful decidable fragments from undecidable theories; fragments that are decidable by reasonably efficient procedures. With enough ingenuity, we may put together a collection of such fragments to cover a large solution space, wide enough to produce a useful theorem prover. In the late 1950s, the development of electronic computers and the interest in Artificial Intelligence (AI) led to early experiments in automated theorem proving (see note by Harrison [67]).

Automatic theorem provers attempt to generate proofs without human intervention. In contrast, *proof-checkers* check the validity of proofs written in a formal language. *Proof assistants* or *interactive theorem provers* lie somewhere in between, and various systems developed over the years cover a spectrum. Many proof assistants attempt to strike a balance by allowing the user to provide a proof sketch (or hints) in order to let the automation complete the proof. Influential systems from the 1970s include AUTOMATH, LCF, Mizar and NQTHM [67]. Modern proof assistants, in turn, make extensive use of automatic theorem proving to support their users during proof development. We use the term *Automated Theorem Provers* (ATPs) to refer to proof assistants and automatic theorem provers all together.

1.1.1 Applications of ATP

Theorem provers have found a range of uses beyond the fields of AI research. We discuss a few prominent applications below.

Formalization of Mathematics

The language of mathematical texts often contains a mixture of natural language and specialized notation, and so has all the potential for ambiguity and imprecision [23]. For example, consider the following excerpts:

> "... almost every time we encounter $\binom{n}{n}$ it equals 1, so we can get lulled into thinking that it's always 1. But a careful look at definition (5.1) tells us that $\binom{n}{n}$ is 1 only when $n \geqslant 0$ (assuming that n is an integer); when $n < 0$ we have $\binom{n}{n} = 0$. Traps like this can (and will) make life adventuresome."
>
> — Graham, Knuth, Patashnik [19, p. 154]

> "In definitions and theorems about rings, it may sometimes happen that the condition $A \neq 0$ is omitted even when it is actually necessary."
>
> — Matsumura, H. (1986) [23]

Though such subtleties and ambiguities may not be an issue for an expert in a specific topic, they may be a cause of considerable pain for practitioners of related topics or subjects. Topics in mathematics are becoming increasingly specialized. It is not realistic to expect, say computer scientists, to understand the deep theoretical results in topology or elliptic curves, should they need to use them [23].

Mathematics is considered the most precise of all subjects [23]. Rigour in mathematics has traditionally been enforced though peer review. However, serious errors can and do slip through. Take for example, Euler's theorem that states that for any regular *polyhedra* with V vertices, E edges and F faces, $V - E + F = 2$. In his book [10], Lakatos presents an impressive history of the proof of this theorem as an imaginary dialogue between a teacher and the classroom. The theorem, its solution, and counter-examples were proposed, accepted and refuted over centuries through a long process of trial and error. The theorem itself went through a series of modifications to clarify definitions and incorporate special cases in accordance with the proposed solutions and counter-examples. An implication of Lakatos's discussion is that even definitions of mathematical constructs such as *polyhedra* may be subtly ambiguous, in which case a theorem or proof may turn out to be incorrect in the presence of a supposedly borderline case as a counter-example. For a more recent example, consider the seemingly important

results published by the prominent mathematicians Voevodsky and Kapranov [18] in 1990, which connected two classes of mathematical objects — homotopy types and ∞-groupoids. In 1998, Carlos Simpson published a paper [32] containing arguments that implied that the main results of Voevodsky's and Kapranov's paper were incorrect. In the intervening eight years, those incorrect results were applied in other works. Even though mathematicians stopped using those results thereafter, the results remained unchallenged. It was not until 2013 that Voevodsky finally became convinced of Simpson's critique. Voevodsky's references to his paper are now preceded with the warning "the main theorem of this paper was shown by Carlos Simpson to be false".

Formalization of mathematics serves to improve the rigour, precision and correctness of the subject. Formalization requires expressing mathematical statements and proofs in a formal language with consistent, unambiguous syntactic and semantic rules [23]. The language is often designed for mechanical verification, opening the doors to computational verification of proofs. A sense of the need for such formalization has prevailed in mathematics for a long time, resulting in waves of effort towards analysing the foundations and reducing results to deductions from axioms [23]. *Principia Mathematica* by Whitehead and Russell (first published in 1910) [2] is among the most famous works ever written on the foundations of mathematics. This monumental effort was an attempt to present a comprehensive collection of the mathematics of the time in a formal manner, by presenting all theories as logical deductions from a collection of axioms. Russell complained that his mind never fully recovered from the strain of producing that work [161].

Formalizations in modern proof assistants, with the support of automatic provers, offers substantial improvements in ergonomics and rigour. *Isabelle* [17], for example, is a mature proof assistant with extensive contributions from many researchers, a comprehensive library of lemmas, plenty of automation and a powerful integrated development environment. It provides a multitude of general purpose proof methods, tactics and decision procedures that may be invoked to discharge goals in a proof (summarized by Blanchette [83]). The most commonly used ones are simp and auto. The simp method is a simplifier that uses rewrite rules to manipulate the goal, while auto interleaves simplification with proof search. Among other methods, there is blast, a tableau prover, and metis, a resolution theorem prover. Isabelle also provides assistance tools, like Quickcheck and Nitpick to generate counterexamples for invalid conjectures. Finally, Isabelle provides a tool called Sledgehammer [96] that is able to invoke multiple external automatic theorem provers in parallel to generate complete proofs. Another example is the HOL Light proof assistant, developed by Harrison [74], for which the tool called HOL(y)Hammer (by Kaliszyk et al. [113]) performs a role similar to Sledgehammer.

Formalization of mathematics can help in cases when peer-review becomes infeasible, as was the case, for example, in the proof of the Kepler Conjecture. The Kepler Conjecture

asserts that a packing of equal radius balls in Euclidean three-space cannot exceed a density of $\pi/\sqrt{18}$. Presented by Kepler in 1611, it is the oldest problem in discrete geometry [123]. In 1998, Ferguson and Hales presented a proof of the conjecture. The proof spanned 300 pages of text and relied on computer calculations based on approximately 40,000 lines of computer code [51]. While being reviewed, the team of referees had difficulties in verifying the complex computer proof. The proof was eventually published in full in 2006 without complete certification from the referees. In 2003, Hales launched the *Flyspeck* project, which aimed to fully formalize a proof of the Kepler Conjecture [51] (both the text and the code parts). The name was derived from the acronym FPK, standing for the Formal Proof of the Kepler Conjecture. To clear lingering doubts about the correctness of the proof was not his only motivation, Hales also considered formal methods as fundamental to the future of mathematics [51]. After a conference in 2009, the project became a large international collaboration effort [123], eventually completing in 2014. The formalized proof had four parts, the main statement and three subclaims. One of the three subclaims was implemented in Isabelle/HOL. All other parts were formalized in HOL Light. With prominent results of recent years requiring substantial proofs [67], the traditional process of peer review is becoming an increasingly fragile method of enforcing rigour. Formalization may therefore be essential to future progress in mathematics [51, 105].

Formalizations can also add clarity to accepted results. An interesting development in this regard is the formalization of Gödel's incompleteness theorems by Paulson [116]. These two theorems are widely misinterpreted in popular culture and academia; and Franzén [50] reviews such interpretations in detail. Although the first theorem has been proved with some degree of mechanization before, Paulson has completely formalized both together for the first time ever. Isabelle/HOL was used for this formalization, which sports proof scripts that can mimic the structure of handwritten proofs to aid readability. The *de Bruijn coefficient*, commonly discussed in the formalization community, is ratio of the sizes of the formalized proofs to the paper proofs. In the aforementioned formalization, the coefficient was 12. A substantial expansion in the proof size due to formalization is typical, along with a greater investment of intellectual effort. Many published proofs of the incompleteness theorems are vague, and the missing details encourage misinterpretations and needless debates. Formalization adds to the clarity and detail of even the most substantial and established works, which can improve our broader understanding of mathematics and philosophy.

Formal Verification

Since the 1970s, microprocessors have sustained a growth in performance of over 50% per annum — a rate that is exceptionally high in the technology industry. This achievement is due to a combination of increasing transistor counts and improvements in architectural

design (see Hennessy et al. [92] for an in-depth analysis). As a consequence, although computers have become relatively cheap commodities and easier to use, modern microprocessor development is extremely complex and costly. For instance, consider the Intel 80286 microprocessor released in 1982, which contained 134,000 transistors, compared to the Intel Core i7 processor release in 2010, which contained 1,170,000,000 transistors. Despite rigorous modelling, testing and analysis, costly mistakes — although very rare — have been missed in the past. An example is the hardware bug discovered in 1994 in the Pentium processor that caused errors in a floating-point division instruction (see note by Price [21]). The proof of correctness of the algorithm was itself incorrect, and that combined with a mistake in the design led to a faulty implementation, which was then subsequently also missed in testing. The recall and replacements costed millions of dollars.

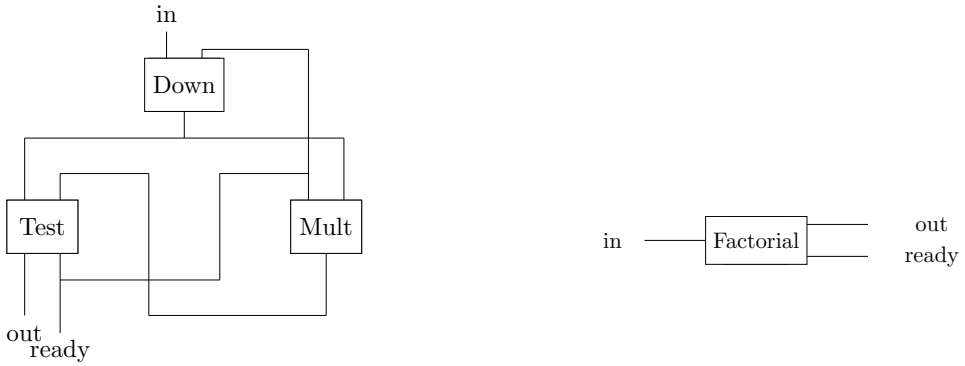
Formal methods are the most rigorous defence against errors devised thus far. They take advantage of theorem provers to check engineered systems with mathematical precision using machine checked proofs. Integrated Circuit (IC) hardware designs are traditionally verified using formal methods. Typically, designs are specified using logical operators such as $\wedge$ (and), $\vee$ (or) and $\neg$ (not) such that the resulting boolean formulae can be checked for a satisfying assignment. A satisfying assignment is one in which the values assigned to the variables make the formula evaluate to true, and if one such assignment exists then the formula is considered satisfiable. The boolean satisfiability problem, that is the task of finding such an assignment, has NP-complete complexity [71]. Yet modern solvers are able to check formulae with hundreds of thousands of variables with ease [65] and thus are often used in hardware verification (see tutorial on the topic by Gupta et al. [57]).

Although the formalization in boolean algebra can be used to verify combinatorial logic circuits, it is insufficient for modelling sequential logic and more complex higher-level behaviour. Camilleri et al. proposed a more powerful alternative [13]: the use of Higher-Order Logic (HOL, based on work by Church [4]) to formalize IC implementation as well as the specifications; and verification by proving equality between the two. HOL is expressive enough to model sequential logic as well as high-level specifications. Skipping much detail, we show an example (borrowed from Camilleri, Gordon and Melham [13]) in Example 1.1.

Example 1.1 (Hardware Verification in HOL)**.** The implementation shown in Figure 1.1a can be formalized as

$$F_{\text{implementaion}}(r, i, o) \equiv \exists l_1 l_2.\ D(i, r, l_1) \wedge M(r, l_1, l_2) \wedge T(l_1, l_2, o, r),$$

where the symbols in the above equation represent elements in the diagram with matching initials. Similarly, the specification shown in Figure 1.1b can be formalized in



(a) Device implementation. **(b)** Specification.

Figure 1.1: A sequential logic device to implement the factorial function.

HOL as

$$\begin{aligned} F_{\text{specification}}(r, i, o) \equiv \forall t.\ & r(t) \wedge n = i(t) \supset \\ & \text{Next}(t, t + n + 2, r) \wedge \\ & \forall t'.\ (t \leqslant t' \wedge t' < t + n + 1) \supset (o(t') = 0) \wedge \\ & o(t + n + 1) = \text{Fact}(n). \end{aligned}$$

To show that the implementation is correct as per the specification, we must prove that

$$F_{\text{implementaion}}(r, i, o) \supset F_{\text{specification}}(r, i, o).$$

This example is abbreviated from the one in Camilleri, Gordon and Melham [13], which should be referred to for details about the formal proof. ◁

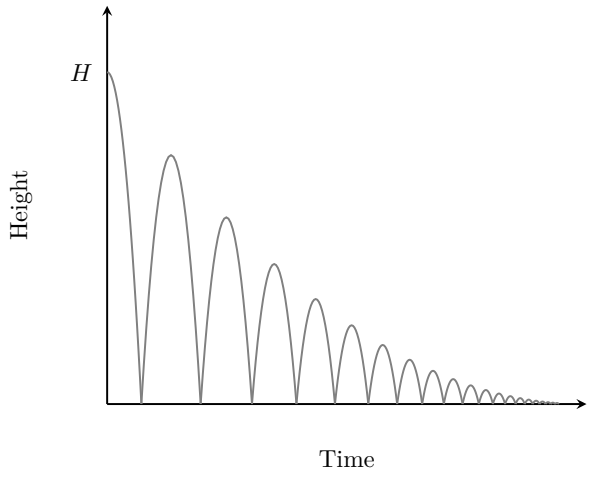
For the purposes of formal verification along the lines of Example 1.1, Gordon led the development of the HOL family of proof assistants [35]. Of that lineage, HOL4 [70] is an actively maintained prover that has been used in a wide range of formalization projects. For example, Fox used it to formally specify and verify the ARM6 micro-architecture [41]. Harrison [52] has contributed a large body of work to formalize floating-point hardware, to avoid errors like the aforementioned bug in Intel processors, using HOL Light. HOL Light, also authored by Harrison [74], belongs to the same family of provers.

Similarly, errors in software are costly. Tassey [39] concluded in 2002 that inadequate infrastructure for software testing created a loss of $59 Billion for the economy of the United States of America. Formal verification of software systems was traditionally employed in safety-critical niches, such as avionics [52]. However, with advances in prover technologies and greater demand for safety in computer systems, its use has widened. More recently, it has seen increasing adoption by cloud computing providers. For example, Amazon Web Services (AWS) is one such provider, which has concertedly invested in automated reasoning tools to facilitate simultaneous growth in both functionality and security (noted by Cook [133]). They have, for example, proved

that the boot code in their data centre machines is memory safe [134], using the C Bounded Model Checker (CBMC [44]) with some extensions. CBMC is a bounded model checker for C, C++ and Java programs, wherein user-specified bounds on all loop iterations enables it to build a logical model of the source code. Subsequently, it checks specified properties of the model by making use of an external solver. Cook et al. [134] were able to statically analyse the boot code using CBMC. That was advantageous because boot code is difficult to test and debug by other means, due to being run without the support of an operating system and before input/output (IO) hardware is fully initialized. For another example, consider the work by Desai et al. [135], who have used the P language to reason about distributed systems; distributed systems are widely used in cloud computing infrastructure. The P language [101] is specifically designed for the domain of asynchronous event driven code: for its specification and for model checking.

Formal methods have been used to improve the security of computer networking. Paulson [28] has developed a method to rigorously verify security protocols, by formalizing backward arguments of the impossibility of events, using inductive definitions which are mechanized in the Isabelle/HOL proof assistant; application to key distribution protocols revealed a new attack. Chudnov et al. [132] have formally verified s2n — an implementation of the Transport Layer Security (TLS) protocol — with the high-level security properties formalized in the Coq proof assistant [162] combined with code-level verification using bounded model-checking. Backes et al. [129] have built a security analysis tool for the AWS cloud, called Zelkova. AWS customers control permissions for subscribed cloud services through detailed access control policies. When many subscriptions are in use, it becomes difficult to ensure that the policies grant the right permissions. Zelkova is able to formally verify policy properties to answer complex queries about them, such as, if a resource is accessible by a particular user. It also has a number of built-in checks for common errors, and enables AWS users to automatically maintain security controls.

The use of formal verification extends into the wider domain of Cyber-Physical Systems (CPSs), wherein physical processes are integrated with computation (see detailed overview by Taha, Taha and Thunberg [156]). For example, the Segway [46] is a self-balancing two-wheeled personal transportation device that relies on a computational component to remain balanced and upright. Mechanically, it is unstable and cannot be kept upright by the rider without the real-time control systems that run on its embedded computer. Similarly, many generations of fighter jets, such as Saab JAS 39 Gripen, are critically dependent on such active controls for aerodynamic stability [156]. Mathematical modelling of CPSs often involves complex interactions between discrete and continuous dynamics. A motivating example, using a simple bouncing ball, is shown in Example 1.2. Hybrid dynamical systems (or hybrid systems) are mathematical models for analysing such interactions (see Platzer [97]). KeYmaera (by Platzer and Quesel [69]) is a verification tool for hybrid systems, which combines interactive and



(a) Height of a bouncing ball over time.

$$
\begin{aligned}
&\text{Ball} \equiv (\\
&\quad (h' = v, v' = -g, t' = 1, h \geqslant 0);\\
&\quad \text{if } (h = 0 \wedge t > 0) \text{ then}\\
&\qquad v := -cv; t := 0\\
&\quad \text{fi}\\
&\quad)^*
\end{aligned}
$$

(b) A specification in KeYmaera [69].

Figure 1.2: Modelling a bouncing ball.

automatic provers for a specially designed natural specification and verification logic. It has been used, for example, to verify several properties of the European Train Control System: properties such as liveness, controllability and reactivity, which together entail non-collision.

Example 1.2 (A Bouncing Ball)**.** Consider a well known example: the bouncing ball. A ball dropped from a height bounces back but looses energy at each bounce, and therefore its rebound height is less than the starting height, and subsequent heights continue to decrease as depicted in Figure 1.2a. The instantaneous change in velocity of the ball at the point of bounce creates difficulties when modelled using differential equations alone. However, when modelled as a hybrid system, the bounce can be conveniently represented as a discrete state transition.

Let v be the current velocity of the ball and h the current height at any time. Let g be the magnitude of acceleration due to gravity and H the initial height from which the ball is dropped. Since the ball looses energy at each bounce by a constant factor c, it can never rebound higher than H. This can be expressed as the safety property $0 \leqslant h \leqslant H$, which, along with the model in Figure 1.2b, can be checked in KeYmaera by verifying

$$(h = H \wedge v = 0 \wedge 0 \leqslant H \leqslant 4 \wedge 0 \leqslant g \leqslant 2) \rightarrow [\text{Ball}](0 \leqslant h \leqslant H).$$

Platzer and Quesel [69] describe this example in more detail, with an explanation of the semantics of the specification in Figure 1.2b. ◁

Automated Reasoning

Methods derived from or otherwise related to ATP can be used to automate tasks that require some degree of logical reasoning, thereby producing seemingly intelligent *automated reasoning* software. Specifically, by expressing a problem domain in terms

of: premises or ground truths; and rules of inference using a logical formalism; the problem can be expressed as a statement in that grammar that may be checked by an ATP. Premises can be derived from facts and assertions of the domain. A balanced logical formalism must be chosen to encode the rules of inference: one that is expressive enough to encode the problem and yet simple enough for formalization and ATP. It is common to translate the problem into a well known mathematical theory, especially one that can be efficiently mechanized, such that an existing prover may be used for checking the problem statement.

Knowledge Representation and Reasoning (KRR) systems leverage formal mathematics to encode knowledge and ATPs to draw conclusions from it. The Web Ontology Language (OWL), for example, is a formal language for representing *ontologies* on the web (see review by Horrocks, Patel-Schneider and Harmelen [42]). *Ontologies*, broadly, represent information about types of objects and how they are interrelated. OWL uses *Description Logics* (DLs) for its underlying formalism. DLs are a family of concept-based logical formalisms for knowledge representation. A knowledge-base for DLs consists of a collection of axioms, some of which describe a particular situation whereas others set constraints on the conceptual schema. An axiom could, for example, assert one concept being a subclass of another or an individual being an instance of a concept. Queries can be answered by a DL prover. Queries can, for example, check if a concept is feasible with respect to the knowledge-base. Or, for another example, one may query the value of an instance of a given concept. In modern DL systems, queries reduce to a check of their satisfiability — a query is satisfied if an instance exists in the knowledge-base that makes it true. The FaCT++ reasoner (by Tsarkov and Horrocks [61]) implements an efficient decision procedure for DLs with support for OWL. Such provers along with an underlying formalism and a knowledge-base encoded in that formalism produce a very capable KRR system.

ATP methods form the foundation of the programming language paradigm called *Logic Programming*, which departs radically from imperative programming. Problems are expressed in that paradigm in terms of the knowledge and assumptions about it. Thus, a program is written in terms of a collection of logical axioms. The program is executed by providing it with a final problem statement, called the *goal*. Computation takes the form of a proof search of the goal given the program. Typically, the goal states the existence of an individual with some specified property. If such a goal is found to be true, the identity of the individual becomes the output of the computation.

A well known logic programming language is Prolog, standing for 'Programmation en Logique' [14]. A program in Prolog consists of a database of clauses, each of which are logically equivalent to

$$(A_1 \wedge \ldots \wedge A_m) \rightarrow B.$$

When $m = 0$, the clause is a fact and is just B. When B is false, the clause is a list of unsolved

goals, and only one such clause is permitted. This constrained logical form enables a deterministic execution strategy for Prolog, and multiple efficient implementations are available [100]. Prolog is suitable for building reasoning systems. An example is the question-answering computer system called IBM Watson, which in 2011 won the televised game show called 'Jeopardy!' by defeating two former champions [87]. Prolog was used to implement the question analysis components (see description by Lally et al. [94]). Lally and Fodor [87] note that Prolog is a good implementation language for problems requiring depth-first search and backtracing.

Using KRR, Logic Programming and a variety of specialized methods, diverse reasoning applications make use of ATPs. ATPs are advanced tools for solving complex reasoning problems with a successful track record.

Summary

Interest in ATP has motivated many fundamental developments in Computer Science. Although it is now in itself a niche field, vital applications rely on it. Therefore, research and development of improved theorem proving methods remain important.

1.1.2 Methods for ATP

The basic foundations of ATP consist of computer models of mathematics. Such models can be used to express theorems and then ATPs can generate, and check, proofs using them.

Models of Mathematics

The history of scholarly work on Mathematical Logic stretches back at least 2000 years [45]. Aristotle, a philosopher in Ancient Greece, tried to describe correct forms of arguments by condensing them into 19 possible variations, which he called *syllogisms*. Around 300 BC, Euclid, a mathematician working in Egypt, invented the modern procedure for establishing truth in mathematics. Beginning with five assumptions about geometry, which seemed fundamental and undeniably true, he established the truth of many additional statements by deriving *proofs*: a sequence of logical deductions from previously established truths that conclude with the statement in question. Fundamental assumptions that are simply accepted as truths are called *axioms*. Euclid's approach is now called the *axiomatic method*.

Logic and axioms together form a mathematical theory. Both, therefore, must be formally modelled for effective computer modelling of mathematics. Logic expresses only the *forms* of mathematical arguments that are considered correct [12]. Axioms are

the fundamental assumptions taken as facts in the theory. Theories can be carefully constructed to have desirable properties, such as *soundness* and *completeness*. A theory is *sound* if every provable statement is in-fact true. A theory is *complete* if every true statement has a proof.

For the purposes of computer implementations, we find it necessary to distinguish between mathematical functions, and procedures than can be programmed in computer code, versus methods that require our ingenuity to execute. In 1936, Turing and Church, working independently, formalized that distinction in work that is altogether called the Church-Turing thesis [142]. By devising the most general models of computation, they formalized the notion of an *effective* method or function: one that can be executed or calculated purely by mechanical means.

Propositional Logic. *Propositional logic* finds many uses in Computer Science; a well known field of application is circuit design [22]. It is based on *propositions* — statements that are either true or false — which are themselves composed of simpler propositions combined using the logical connectives:

- $\neg$, for *not* or negation;
- $\wedge$, for *and* or conjunction;
- $\vee$, for *or* or disjunction;
- $\rightarrow$, for *implies* or implication; and
- $\leftrightarrow$, for *iff* or if and only if.

Remark 1.1 (Metalogic). In order to avoid confusion in texts that use logic to describe mathematical logic, we use the following jargon. We use a *metalanguage* to describe a distinct *object language*, with a corresponding *metalogic* and *object logic*. Here, the object is a specific mathematical theory under consideration. For example, propositional logic as described above is the object logic, while the logic employed in our written text here is the metalogic. Similarly, the ::= symbol used below is a metalogic symbol, as opposed to the propositional symbols defined above. ◁

Propositions are expressed as *formulae*, and propositional formulae are also called *Boolean formulae*. The syntax for Propositional Logic in Backus Naur form (BNF) is

$$\phi ::= p \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \rightarrow \phi) \mid (\phi \leftrightarrow \phi),$$

where ϕ is a propositional formula and p is any atomic propositional formula [45]. Each formula has a semantic of 1, meaning true, or 0, meaning false. The connectives affect the semantics of formulae as shown in Table 1.1. Such tables are called *truth tables*.

A formula is said to be *satisfiable* if there is a semantic assignment to the atoms that makes the whole formula true. A formula that has no such assignment is said to be *unsatisfiable*. A formula that is true for every assignment is called a *tautology*. The task of checking if a propositional formula is satisfiable is known as the *satisfiability problem*, or SAT for short. SAT is a well-known NP-complete problem [71].

Table 1.1: Semantics of Propositional Logic connectives.

A	B	$\neg A$	$A \wedge B$	$A \vee B$	$A \rightarrow B$	$A \leftrightarrow B$
1	1	0	1	1	1	1
1	0	0	0	1	0	0
0	1	1	0	1	1	0
0	0	1	0	0	1	1

First-order Logic. Although propositional logic can be used to formalize declarative sentences, and is sufficient for complex applications like circuit design verification, it seems limited when compared to the logical constructs used in natural languages. Take, for example, the following quote:

> "All models are wrong, but some are useful."

We may encode the first clause (attributed to George Box [15]) as proposition p and the second clause as q, formalizing the whole statement as $p \wedge q$. However, this formalization fails to capture the fact that both propositions are related, since they are both about 'models'. Furthermore, we have no means of formalizing the quantifications 'all' in p and 'some' in q using propositional logic alone.

Predicate logic, also known as *first-order logic*, extends propositional logic with non-propositional variables that can be bound with quantifiers. This enrichment enables more complex and detailed formalization. The logic inherits the concept of formulae from propositional logic, but adds *terms* to denote some additional abstract objects. Terms can be variables bound to objects, constants, or functions applied to variables. In BNF, a term t is

$$t ::= x \mid c \mid f(t, \ldots, t) \tag{1.1}$$

where c is a constant and f is a function. All logical connectives carry over from propositional logic, but atomic propositions are replaced by *predicates*, also called *relations*, applied to terms. For example, if x and y are variables referring to persons, and the predicate $L(x, y)$ means that x loves y, then L is the predicate symbol here.

Predicates and functions accept a finite number of terms as arguments — that number is called their *arity*. Functions accepting zero arguments are effectively constants, and thus are synonymous. Examples are 1 and π. Zero argument predicates are similar in spirit to propositional variables. The adjective *nullary* is used for zero argument functions and predicates. Similarly, *unary* or *monadic* functions and predicates have one argument. With two arguments, they are *binary* or *diadic*. With n arguments, they are n-ary.

In addition to predicates and functions, first-order logic adopts *quantifiers*. The quantifier $\forall$ is called the *universal quantifier*. Informally, the predicate $\forall x.\, p$ means that p is true for

all values of x. Conversely, the symbol $\exists$ denotes the *existential quantifier*. Intuitively, $\exists x.\, p$ means that there exists a value of x such that p is true [73].

The syntax of first-order logic in BNF for a formula ϕ is

$$\phi ::= P(t, \ldots, t) \mid (\neg\phi) \mid (\phi \wedge \phi) \mid (\phi \vee \phi) \mid (\phi \rightarrow \phi) \mid (\phi \leftrightarrow \phi) \mid (\forall x.\, \phi) \mid (\exists x.\, \phi)$$

where P is a predicate symbol and t are terms as in Equation (1.1). Note that in first-order logic, sometimes shortened to FOL, we only quantify over object-denoting variables. In higher-order logic (abbreviated as HOL above), also called *second-order* logic, we permit quantification over functions and predicates [73].

Intuitionistic Logic. Consider the following proof (borrowed from Harrison [73]).

Theorem 1. *There are algebraic irrational numbers a and b such that a^b is rational.*

Proof. The value $\sqrt{2}^{\sqrt{2}}$ is either:

(i) rational, in which case $a = \sqrt{2}$ and $b = \sqrt{2}$ yields a rational a^b; or

(ii) irrational, then $a = \sqrt{2}^{\sqrt{2}}$ and $b = \sqrt{2}$ works.

□

We consider this kind of proof to be *nonconstructive*, because it establishes the existence of something without specifying a method to obtain it. Notably, it makes use of the law of excluded middle — the assumption that either one of the two cases must be true. However, some mathematicians prefer *constuctive* proofs. For a proof to remain constructive, all of its existential and disjunctive statements must take the following specific forms (see Dummett [34]). A proof of:

$\exists x.\, A(x)$ must specify an instance $A(n)$ or provide a means of finding one;

$A \vee B$ must contain a proof of either A or B, or the means of finding either one;

$\forall x \exists y.\, A(x, y)$ must yield an effective function f such that $A(m, f(m))$ holds for each m; and

$\forall x(A(x) \vee B(x))$ must provide an effective method of finding, for each m, a proof of either $A(m)$ or $B(m)$.

L. E. J. Brouwer, a prominent early 20th century mathematician, led a constructivist philosophical movement called *intuitionism* [73]. Intuitionists, the followers of intuitionism, hold the core belief that mathematical objects are mental constructs with no independent ideal existence [153]. They, furthermore, due to Brouwer, reject the belief that every mathematical problem has a solution; Brouwer, in this regard, anticipated Gödel's incompleteness theorems by nearly a quarter-century [153]. As a consequence, intuitionists consider non-constructive proofs to be invalid. Arend Heyting, a student of Brouwer, formalized the ideas in intuitionism to develop *intuitionistic logic* as a logical basis for constructivist mathematics [73]. Intuitionistic logic became a non-standard form of traditional logic, and we now refer to traditional logic as *classical logic*. In essence, intuitionistic logic is classical logic with the exclusion of certain laws [153], namely:

- the excluded middle, $A \vee \neg A$; and
- double negation elimination, $\neg\neg A \rightarrow A$;

while still retaining the laws of:

- contradiction, $(A \rightarrow B) \rightarrow ((A \rightarrow \neg B) \rightarrow \neg A)$; and
- explosion, $\neg A \rightarrow (A \rightarrow B)$.

This implies, notably, that the well known De Morgan law $(\neg(A \wedge B)) \rightarrow (\neg A \vee \neg B)$ is unavailable to intuitionists. Similarly, they cannot negate a universal quantifier to obtain the existential. Note, also, that the semantics described in Table 1.1 do not apply to intuitionistic propositional logic.

Although intuitionists originally developed this theory due to specific philosophical choices, it has become useful to computer scientists because their proofs are free from computationally inadmissible decisions [131]. Intuitionistic logic is of interest to us because relatively little is known about it with regard to automation, despite its use in computer science and philosophy of mathematics.

Proof Methods

ATPs use a symbolic approach for mathematical representation and proof generation. Formal logic allows us to clearly separate the syntax of theories from the *semantics*: semantics ascribe meaning to the syntax. ATPs operate on the syntax, with the procedures carefully designed to ensure that semantically correct operations are performed. Such procedures are often presented in the form of a *proof calculus*.

As noted above regarding propositional logic, SAT is NP-complete. In the case of predicate logic, given a formula ϕ, the task of checking if ϕ holds true is undecidable, i.e., there is no uniform mechanical procedure for that task that will work for all ϕ [45].

To work around the complexity of SAT, ATPs for SAT, also called *SAT solvers*, employ procedures that perform well in practice for some subclass of formulae that they regard as being well-formed. An influential example is the Davis, Putnam, Logemann, Loveland (DPLL) algorithm [7]. DPLL is able to check the goal formula for unsatisfiability, and for satisfiable formulae a satisfying assignment can be extracted. To prove a theorem using DPLL, one can prove its negation to be unsatisfiable. One of the steps in the DPLL algorithm splits the goal formula, which are then recursively checked in sequence using backtracking. If one part of the split reveals the result, the other part does not need to be evaluated, and in that case the algorithm can avoid backtracking. DPLL effectively performs a depth-first search, and modern high performance implementations attempt to optimize this search process to reduce the need for backtracking. Modern SAT solvers are able to solve impressively large problems. Other methods implemented in SAT solvers include local search [151] and binary decision diagrams [11]. The need to search through a solution space is common among algorithms in ATPs for complex theories.

Some ATPs for first-order logic are called resolution provers because they make use of the *resolution* rule: the two clauses $p \vee A$ and $\neg p \vee B$ together imply $A \vee B$. Such provers attempt to repeatedly combine clauses using this rule to derive an empty clause, which is semantically a contradiction, and which terminates the procedure. The original goal is negated first, and so if the resolution prover finds an empty clause, the goal is proved to be a theorem. The proof search in resolution provers can be optimized by guiding it to the best clauses to resolve first to quickly reach the empty clause. Another well-known calculus for first-order logic is the Tableau Calculus [81], in which the proof search tree is more explicitly part of the procedure.

Summary

Formal logic enables precise computer modelling of mathematics. Even for complex and undecidable theories such as propositional logic and first-order logic respectively, proof calculi can be designed to target large and useful fragments of the theories for the development of practical automatic proof procedures. Inevitably, in such procedures, a proof search is required, which traditionally has been guided using heuristics. Continuous research in this field has yielded impressively capable theorem provers.

1.2 Machine Learning

Machine Learning (ML) is a branch of AI specializing in data-driven methods for information processing and prediction tasks [90]. In this text, references to machine learning are short for, more specifically, probabilistic machine learning. Such methods contrast from exact methods such as KRR introduced in Section 1.1.1, which may also be applied to data-driven tasks but are vastly different in applicability and implementation. The use of probability theory in the foundations of ML methods is essential, to enable them to work with limited data and our limited understanding [90]. In broad terms, we engineer ML methods to automatically detect underlying patterns in data, and use them to predict future data or to make decisions [95]. The process of identifying patterns is akin to learning. ML is closely related to the fields of statistics and data science, but differs in emphasis. The emphasis in ML is on predictions.

The process of *learning* in ML is the use of data to infer the distribution that generated the data. *Inference* is the process of computing functions of the distribution to draw conclusions from it. We realize such processes by making use of a *statistical model* $\mathfrak{F}$, which is a set of distributions. A *parametric model* is a set $\mathfrak{F}$ that can be parametrized by a finite number of parameters [49]. It takes the form

$$\mathfrak{F} = \left\{ f(x; \theta) : \theta \in \Theta \right\}, \tag{1.2}$$

where θ is the unknown *model parameter* that can take values from the *parameter space* Θ, and $f(x)$ is a probability distribution over the random variable x. A *non-parametric model*, on the other hand, cannot be parametrized by a finite number of parameters [49]. Often, the number of parameters in such models grows with the amount of training data. All instances of ML make use of models, however implicit they may be.

1.2.1 Types of Machine Learning

Machine learning methods can be broadly categorized into *supervised learning*, and *unsupervised learning*. A third category is *reinforcement learning*, which we do not make use of in this work. However, we make use of ideas related to it, and hence we will discuss it briefly. This categorization is not exhaustive, and some methods fall in overlapping categories, for e.g., semi-supervised learning.

The sections below, while describing the aforementioned categories, also introduce some preliminaries and notation used in the rest of this text.

Supervised Learning

Suppose we have data in the form of N input-output pairs such that $\mathcal{D} = \{\boldsymbol{x}_i, y_i\}_{i=1}^{N}$, where $\boldsymbol{x}$ are the inputs and y the outputs. We refer to $\mathcal{D}$ as the *training set*. Vector-valued variables in this text, such as the input, are in boldface. The inputs $\boldsymbol{x}_i$ are D-dimensional, and each dimension is called a *feature*. The output variables y_i, also called *response* variables, may be *categorical*, in which case they take values from a finite set $\{1, \ldots, C\}$. Otherwise, they are real-valued. The task in this so called *supervised* setting is to train an ML model on $\mathcal{D}$ such that it may then be able to predict, for a previously unobserved input $\boldsymbol{x}_*$, a response y_* based on the underlying patterns in $\mathcal{D}$.

Classification. When the response is categorical, that is $y_i \in \{1, \ldots, C\}$, the ML task is called *classification*. When $C = 2$, there are two classes and then the task is called *binary classification*; otherwise it is called *multi-class classification*.

Example 1.3 (Binary classification of medical risk)**.** Severe Traumatic Brain Injury (TBI) is a leading cause of child mortality. When a child with TBI is admitted to hospital, doctors need to make decisions regarding monitoring requirements or potential surgical interventions. A prediction of the potential outcomes at the point of admission would be very useful. Blood test results at the point of admission are a potential source of predictive data.

Kayhanian et al. [140] have evaluated the use of ML to classify such patients, at the time of admission, into two categories: favourable and unfavourable. They trained their models on records of children with TBI admitted between 2009-2013 to the Cambridge

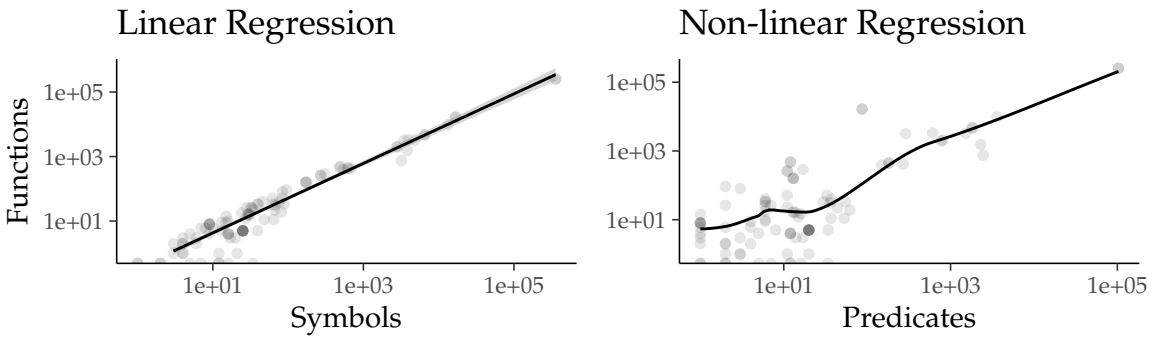


Figure 1.3: Theorem features in TPTP v8.0. Each point is a problem. Counts of functions, symbols and predicates are in the axes, plotted on a log-scale.

University Hospital Paediatric Intensive Care Unit. This data contains blood test results at the point of admission, and outcomes after six months. Their model achieves a good prediction accuracy, and they conclude that such methods are viable in principle.

Kayhanian et al. [140] also deduced that lactate, H+ (venous), and glucose readings were most informative of the potential outcomes. By using the values of only those readings as inputs, that is by having $D = 3$ dimensional inputs $\boldsymbol{x}_i$ with each dimension containing the value of one of the three serum readings, they were able to construct a model with improved performance characteristics. This is an example of *feature engineering*, whereby we make use of domain knowledge to design features for use in ML models. ◁

Regression. For a real valued response variable, that is $y_i \in \mathbb{R}$, the ML task is called regression.

Example 1.4 (Linear and non-linear regression)**.** The Thousands of Problems for Theorem Provers (TPTP) problem library [127] is a large and comprehensive collection of problems, designed for testing automated theorem provers. We use TPTP as our main test-bed in much of our work. In Figure 1.3, we observe some correlations between counts of the functions, symbols and predicates of the problems in version 8.0 of TPTP. We plot only a randomly selected subset of 99 problems. We are able to approximate the correlation between functions and symbols by a straight line. This is an example of *linear regression*, wherein the response is a linear function of the inputs. Its model can be written as

$$y(\boldsymbol{x}) = \boldsymbol{w}^\intercal \boldsymbol{x} + \epsilon = \sum_{j=1}^{D} w_j x_j + \epsilon,$$

where ϵ is the *residual error*. In contrast, as can be observed from Figure 1.3, the relationship between the counts of functions and predicates is more complex. We approximate it by a curve, using a non-linear regression model. ◁

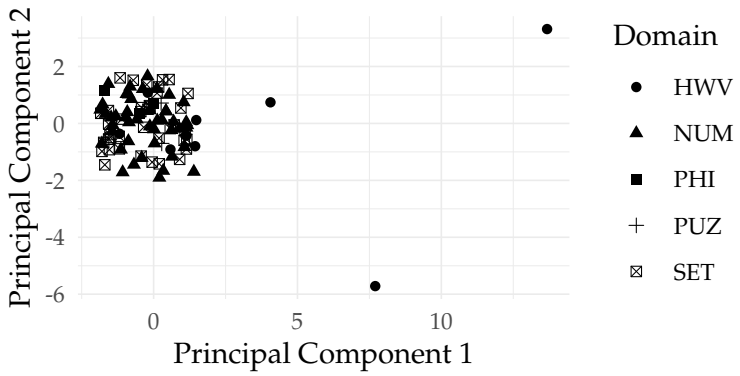


Figure 1.4: PCA of theorem features in a random subset of TPTP v8.0. Each point is a problem.

Unsupervised Learning

When no special response variable is available in the data, the ML task is called *unsupervised learning*. The data takes the form $\mathcal{D} = \{\boldsymbol{x}_i\}_{i=1}^N$. In such instances, we are interested in discovering interesting underlying structures in the data or a plausible compact description of it. Models in this case take the form $p(\boldsymbol{x}_i; \boldsymbol{\theta})$.

Example 1.5 (Principal Component Analysis)**.** Continuing from Example 1.4, we attempt to learn from the counts of functions, symbols and predicates as before, but in an unsupervised manner. We treat all three counts as features, as opposed to Example 1.4, in which we treated function-counts as the response.

Dimensionality reduction is the process of reducing the number of features in the data to produce a related representation of it [143]. Typically, we hope to capture the 'essence' of the data in a lower dimensional space [95]. A common unsupervised ML method for this purpose is Principal Component Analysis (PCA). In Figure 1.4, we plot a subset of the TPTP problems from five domains and randomly select 99 of them. We use PCA to reduce the features from three dimensions to two, and the two new dimensions are in the axes.

It is notable in Figure 1.4 that most problems cluster together but some in the `HWV` domain stand out. Just as we are able to note these anomalies, unsupervised ML methods can also be used for this kind of task, which is called *anomaly detection*. ◁

Reinforcement Learning

For the task of deciding upon the optimal actions to take based on experience, with the goal of maximizing a reward, we make use of *reinforcement learning*, which refers to a subfield within ML. The reinforcement learning problem is often posed in the form of a goal-seeking agent acting within an uncertain environment [138]. Initially the agent faces significant uncertainty regarding the environment. The agent must decide on the actions it takes, the outcome of which is a reward signal as well as some knowledge

about the environment. The agent makes use of ML algorithms to learn from its actions, reason about its uncertainty and decide upon the actions to take. The agent attempts to maximize its reward.

Every action has two consequences for the agent: it receives a reward and it learns about its environment. The agent may choose its action for two potential purposes:

- *exploration*, to reduce its uncertainty about the environment; or
- *exploitation*, to take advantage of its knowledge and extract a higher reward.

There is often a trade-off between these two purposes, and methods to balance between the two are an active field of research [138]. Although we do not make use of reinforcement learning in our work, we encounter a challenge similar to this exploration-exploitation balancing act in Chapter 3.

1.2.2 Bayesian Learning

In Section 1.1.2, we noted how philosophical differences have divided opinion on mathematical logic, leading to the development of intuitionism. With some degree of similarity, philosophical disagreements regarding the nature of uncertainty have led to the development of two main variations of probability theory. The classical methods are called *frequentist*, whilst the alternative is *Bayesian*. Frequentists believe that probabilities are objective and refer to limiting relative frequencies, whereas Bayesians interpret probabilities as subjective *degrees of belief* [49]. For reasons of practical importance to our application area, we primarily make use of the Bayesian flavour of ML, which is also referred to as *Bayesian learning*.

With Bayesian probability theory, we are able to make probability statements regarding more mathematical objects than just those subject to observed random variations. In particular, we are able to treat model parameters θ as random variables. Then, before any data is observed, the probability density of the parameters, $f(\theta)$, reflects our *prior beliefs* — those being our preconceived notions — and is called the *prior distribution*.

As explained by Wasserman [49], inference in Bayesian learning, or *Bayesian inference*, typically requires the following steps.

1. We choose a prior distribution $f(\theta)$.
2. We choose a statistical model $\mathfrak{F} = \{f_m(x \mid \theta) : \theta \in \Theta\}$. Careful attention is needed to contrast this from Equation (1.2). The distinction here is in the use of $f_m(x \mid \theta)$, which is a probability distribution conditioned on θ since that is now a random variable, as opposed to $f(x; \theta)$.
3. After observing the data $\mathcal{D}$ we update our beliefs by calculating $f(\theta \mid \mathcal{D})$, which we call the *posterior* distribution.

The basis of the above operations is *Bayes' theorem* as referenced below [49].

Theorem 2 (Bayes' theorem)**.** *Let $A_1, \ldots, A_k$ partition an event space such that $\mathbb{P}(A_i) > 0$ for each i. If $\mathbb{P}(B) > 0$ then,*

$$\mathbb{P}(A_i \mid B) = \frac{\mathbb{P}(B \mid A_i)\mathbb{P}(A_i)}{\sum_j \mathbb{P}(B \mid A_j)\mathbb{P}(A_j)}. \tag{1.3}$$

Proof. Using the definition of conditional probability and the law of total probability, we have

$$\mathbb{P}(A_i \mid B) = \frac{\mathbb{P}(A_i, B)}{\mathbb{P}(B)} = \frac{\mathbb{P}(B \mid A_i)\mathbb{P}(A_i)}{\mathbb{P}(B)} = \frac{\mathbb{P}(B \mid A_i)\mathbb{P}(A_i)}{\sum_j \mathbb{P}(B \mid A_j)\mathbb{P}(A_j)}.$$

□

With Bayes' theorem, we are able to compute the posterior from the prior and the model using

$$f(\theta \mid \mathcal{D}) \propto \Pr(\mathcal{D} \mid \theta) f(\theta).$$

We calculate $\mathcal{L}(\theta) = \Pr(\mathcal{D} \mid \theta)$, which is called the *likelihood* function, using the statistical model $f_m(\cdot)$.

The use of Bayesian learning is particularly advantageous when training data is limited, because in such cases we can prime the ML model with enough prior knowledge to still generate useful predictions.

Closing Remarks

We have summarized the foundations of ML through the lens of probability theory. To motivate our research question, we have focussed our work on Bayesian learning. It should be noted, however, that some ML methods do not conveniently fit in any probabilistic framework, despite their effectivity.

We return to the quote by Box [15], who said that "All models are wrong". Models are necessary for any form of understanding, but all models make assumptions. In this work, for the purposes of ML, our goal is to make useful predictions, which supersedes our interest in any subjective interpretation of our models. Therefore, we measure the accuracy of our models by the end results of our experiments. This, furthermore, requires us to design our models to carefully balance computational and accuracy trade-offs in favour of our end goals.

1.3 Machine Learning for ATP

In this section, we present an overview of a few components of theorem provers that could benefit from the use of ML. Subsequently, we review some applications of ML in those components.

1.3.1 Heuristics in Theorem Provers

ATPs generate proofs by utilizing inference procedures in combination with heuristic search. Procedures such as rewriting and resolution provide means of mathematically accurate inference in fragments of mathematical theories [12]. Mathematical theories are grounded on axioms and rules of reasoning. The proof of any theorem consists of chains of lemmas linked by the rules of reasoning. The inference procedures in ATPs ensure formal proofs correctly follow these rules to remain valid. ATPs often operate in mathematical theories that are either undecidable or intractably complex. Most use heuristic search techniques within a timeout to discover proofs. Modern theorem provers utilize heuristics in many aspects of their operation. Commonly, heuristics are used for *clause selection*, *search control* and *premise selection*.

Clause Selection. Many steps in an inference procedure like resolution require the selection of the next clause to be used. This is a non-trivial but critical decision. The collection of available clauses can be extremely large and only a small subset of them will lead to a proof. Poorly selected clauses will lead the theorem prover astray in the search space, either generating an unnecessarily long proof or, more likely, a timeout. A good sequence of clauses will generate a shorter proof. Theorem provers use heuristics for clause selection and the parameters of the heuristics often require careful manual tuning. For example, some heuristics are based on the observation that clauses containing symbols similar to those in the goal theorem are likely to be useful in the proof. To calculate the relevance of clauses, Veroff [27] suggests a weight assignment to all the terms and formulae.

Search Control. Theorem proving is fundamentally a search problem. Many ATPs take a backwards search approach. Starting with the theorem as the goal, they search backwards through the space of clauses derived using inference procedures to reach the hypotheses. Multiple inference procedures may be used along with higher-level proof plans [31], all working together in the umbrella of a proof search engine. Again, operating in the infinite solution space of undecidable theories makes an unconstrained search intractable. ATPs use heuristics to prune this search space and to control the search process. Search control is therefore a necessity in modern ATPs. Within a theorem prover, instantiations of various parts along with parameter settings for heuristics may be grouped together into a unit called a *strategy*. ATPs can then use different strategies for different goals to increase their chances of success. Considerable research has been devoted to identifying desirable characteristics of good strategies. For example, Reger and Suda [126] have shown that strategies which lead to a slowly growing search space tend to perform better; a heuristic that is used in the Vampire theorem prover [121].

Premise Selection. Work in modern mathematics relies on a wide base of existing theories and a collection of lemmas and theorems. It is unreasonable to expect any

non-trivial proof to start from fundamental axioms. Such a proof will be tedious and incomprehensible. Accordingly, modern ATPs rely on a formalized library of lemmas to avoid repetition and reuse previous results. Libraries of general purpose theorem provers tend to grow rapidly. For example, consider the Mizar Mathematical Library (MML), which accompanies the Mizar project started by Trybulec in the 1970s [53]. MML now contains more than 50,000 theorems to cover a large proportion of undergraduate mathematics [110]. Such libraries are a double-edged sword. Although a rich collection of lemmas is useful for proofs of new theorems, such a large collection of facts can overwhelm a theorem prover. Only a small set of lemmas are typically relevant to the proof of any given theorem (exemplified by Hoder and Voronkov [85]). Theorem provers need to pre-filter the collection of available facts for any given theorem before they begin the proof search. This pre-filtering phase is referred to as *premise selection*. The correct set of relevant facts for a given theorem is undecidable (see Kühlwein et al. [93]) and therefore premise selection is driven by heuristics. Heuristics in premise selection are based on ideas similar to those of clause selection. For example, the Sumo INference Engine (SInE) is a state-of-the-art premise selection heuristic developed by Hoder and Voronkov [85] and implemented in the Vampire theorem prover. SInE takes advantage of the fact that premises directly relevant to the goal theorem must have symbols in common with it. SInE is described in more detail in Section 3.2.

1.3.2 Machine Learning for Automatic Theorem Proving

Development of pre-programmed or heuristic driven ATP methods is an up-hill battle. New topics in mathematics continue to emerge and existing topics drift towards increasing specialization [23]; automated reasoning for them may require new heuristics or specialized tuning. An alternative to manually developed heuristics is the use of ML models.

Sources of training data for ML are plentiful, thanks to generations of research and development in the fields of formalized mathematics and mechanized proofs. The TPTP library serves as a de-facto standard collection of trial problems for ATPs. TPTP is useful in empirical evaluation and comparison of ATP systems. The Mizar proof assistant [76], developed since the 1970s, has accumulated the aforementioned MML. Mizar Problems for Theorem Proving (MPTP) is a system for translating MML into a format suitable for ATPs [48]. Many popular proof assistants have developed large repositories of formalized mathematics. Isabelle, for example, bundles a large library of definitions and theorems in its software distribution. In addition, the Archive of Formal Proofs (AFP [112]) is a large user-contributed repository of proofs formalized in Isabelle. Complex formalized proofs from proof assistants include many proofs that autonomous ATPs might not be able to prove, and therefore might turn out to be sources of valuable additional insight for ML. In addition, different ATPs vary in the types of theorems they

are able to prove, creating an opportunity for one system to derive applicable knowledge from another.

ML has been successfully applied to the task of premise selection in recent years. For example, Irving et al. [122] have presented work on using neural networks for premise selection, named DeepMath. Their system is able to operate without the need for hand-engineered features and delivers a small increase in the number of automatically provable theorems.

By learning from knowledge extracted from mechanized proofs, ATPs could improve their search controls. This idea has been investigated in recent studies. Bridge, Holden and Paulson [107] investigated machine learnt methods for automatically selecting the appropriate heuristic over a given problem in the E theorem prover. E uses a collection of 82 built-in heuristics and a manually programmed automatic heuristic selection system. The ML automatic selection method developed by the authors uses only 5 of these heuristics to deliver comparable results. Features derived from the description of the problem (the theorem and axioms) are *static features*, as opposed to *dynamic features* which may emerge from the knowledge discovered during the automated proof search. The authors mention that their methods were less successful in the use of dynamic features.

Instead of a single strategy for each problem, some ATPs use *strategy scheduling*, where multiple strategies are time-sliced within the permitted time for a proof search (pioneered in Gandalf [29]). In the MaLeS (*Ma*chine *Le*arning of *S*trategies) framework, Kühlwein and Urban [114] incorporated ML methods in the run-time strategy scheduling function of ATPs. The MaLeS-tuned provers solved 8.67% more problems when tested on a subset of the TPTP library. Employing ML at run-time in the proof search process can allow better use of dynamic features, and possibly more intelligent dynamic decisions by ATPs, with less manual tuning.

Manual development of heuristics and search strategies is a tedious and laborious process, and often requires domain-specific input from the authors. Automatic generation of new heuristics might enable ATPs to broaden their reaches in the increasingly diverse and specialized topics in mathematics.

Färber, Kaliszyk and Urban [120] have evaluated the use of Monte Carlo Tree Search (MCTS) for clause selection. The authors use machine-learnt provability estimates in their proof state evaluation heuristics. Loos et al. [125] have demonstrated the feasibility of using deep network models for clause selection in the E prover. They augmented the neural network inference-driven selection process with manually written heuristics to compensate for the slow response from the CPU intensive neural models; this was critical to achieving improved results.

1.3.3 Machine Learning for Interactive Theorem Proving

In addition to proof automation, as with all work in mathematics, it is beneficial to avoid repetition and reuse the work of others as far as possible when developing formal proofs in a proof assistant. Furthermore, a variety of basic mathematical results need to be readily available to formalize any cutting-edge results. For this purpose, modern proof assistants provide a library of formalized results that can be used for new proofs. Isabelle, for example, provides a large and expanding library containing theorems for elementary number theory, analysis, algebra, set theory and more. Formalizations that are not suitable for the library may be submitted to the AFP.

For inexperienced users of Isabelle, searching for relevant lemmas in the vast library may be challenging. Lemmas may have unexpected names and syntax, which makes a pattern-matching search difficult. This is one reason for the great success of the Sledgehammer tool [96]. Sledgehammer can be invoked at any time in the hope of automatically generating a proof, but it works particularly well with *Isar* structured proofs. Using the Isar structured proof language, the user can sketch a formalized proof and then use Sledgehammer to fill-in the details of low-level tactics and lemmas from the library. This frees the user from needing to memorize lemma libraries, proof methods and tactics. Internally, Sledgehammer uses an ML relevance filter called MeSh which we describe in Section 3.4.

Existing proof repositories can also be mined for useful ideas that might be recommended to the user or reused directly by automation techniques. In work disconnected from theorem proving, Allamanis and Sutton [104] have mined source code repositories for *code idioms*: potentially reusable syntactic fragments that recur across projects. For this purpose, they have borrowed techniques from natural language processing, namely, non-parametric Bayesian probabilistic tree substitution grammars. By extension, it may be useful to mine for *proof idioms*: recurring proof fragments that could be reused. Interactive recommendations in proof assistants could be extended to include these proof idioms.

Summary

Large libraries of formalized proofs have been developed due to decades of research and commercial applications. An array of popular theorem provers such as Isabelle and HOL Light have amassed large libraries of basic identities. Libraries to test and evaluate theorem provers, such as MML and TPTP are also freely available. Large formalization projects such as Flyspeck [108] also produce repositories of proofs. These corpora provide ample training data for ML applications and have encouraged research into ML for theorem proving.

ML has been evaluated for use in ATPs in three main avenues: premise selection, clause selection and search control. In deployment, many ATPs that employ ML use them in conjunction with heuristics for best results. ML has also been used to improve automation in proof assistants, especially through meta-provers such as Sledgehammer and HOL(y)Hammer. ML can also be applied in proof assistants to generate a variety of contextual suggestions to aid in the tedious work of formalization.

1.4 This Thesis

We present our significant contributions to knowledge across three technical chapters. We have presented this work previously in reputable academic conferences, as noted below. Strategy scheduling in theorem provers is the focus of this text — an aspect of our main research question. The background material relevant to each chapter is contained within, and every chapter builds upon the previous ones.

In Chapter 2, we discuss methods to transform the data contained within theorems and provers into forms amenable for strategy scheduling using ML. There, we design some original metrics for use as features; and present methods to extract and analyse them. We also test these features, compare them with those developed in related works, and examine their versatility. Furthermore, additional significant contributions to knowledge emerge in our results, which quantify some common knowledge about prover strategies. Some of the work in this chapter was published, in brief, in Automated Reasoning — 11th International Joint Conference, IJCAR (see Mangla, Holden and Paulson [160]).

In Chapter 3, we explore the use of Bayesian optimization for strategy invention in theorem provers. The experiments presented there were peer-reviewed through two papers, published in the 18th International Workshop on Satisfiability Modulo Theories (see Mangla, Holden and Paulson [146]) and the 5th and 6th Vampire Workshops (see Słowik et al. [147]). Those experiments contain some significant contributions: in how we turn parts of theorem provers into viable optimization subject; and in the results themselves.

In Chapter 4, we combine the work in the previous chapters to then develop an original ML method for strategy scheduling. Specifically, we employ Bayesian statistics to propose alternative sequences for the strategy schedule in each proof attempt. It is a relatively dense chapter in terms of technical material due to the volume of original contributions we present. For our publication containing its core material, see Mangla, Holden and Paulson [160].

We conclude with a summarizing discussion of our work in Chapter 5, where we tie it all back together to review the answers we have discovered.

2

Feature Engineering

> proofs are what Littlewood and I call gas, rhetorical flourishes designed to affect psychology [. . .] This is plainly not the whole truth, but there is a good deal in it.
>
> *G. H. Hardy* [3]

Mathematics has traditionally been expressed using texts and diagrams. Computer representations naturally follow this tradition and typically represent mathematical statements as text. Diagrammatic and graphical representations are the common alternative, typically used for more specialized provers, such as the iCon prover for ontologies [137] or the homotopy.io prover for n-categories [141]. Work presented in this thesis is focussed solely on textual computer representations. Typical provers, such as those mentioned in the previous chapters, accept definitions, axioms, premises and the goal theorem as text in a specified grammar.

Modern theorem provers exhibit complex runtime behaviour, owing to their use of complex mathematical theories and, often, large software implementation. Commonly, provers set parameters to control aspects of their runtime behaviour, and thus good parameter settings become critical to their ability to find proofs. Parameters of integer, float and categorical types are the most common. Additionally, many provers use strategies, as introduced in Chapter 1, to specialize the parameter settings for theorems in some segment of the theory they operate in. Finally, provers use strategy schedules, also introduced in Chapter 1, to hedge their bets across a number of strategies. A *strategy schedule* allocates time to proof strategies that are used in sequence. The complex use of

parameters in theorem provers makes the task of setting them a demanding one.

Example 2.1 (MiniSat parameters)**.** Consider the parameters of the MiniSat SAT solver (by Eén and Sörensson [40]). MiniSat (version 2.2.0) accepts parameters of type integer, double, string or Boolean; of which there are nine, seven, one and six parameters each, respectively. Note that parameters of the integer type are the most numerous. This is typical among theorem provers. ◁

The Machine Learning (ML) algorithms of interest accept inputs in the form of linear vectors. To learn from theorems, strategies and schedules, we must translate them, from their various representations in theorem proving software, into vectors that are members of a carefully designed vector space. Such vector spaces need to be engineered to acquire certain properties that extract good performance from the ML implementations in use. The dimensionality of the vector space, for example, is one such property that deserves attention. Of concern, is the problem called the *curse of dimensionality*, to mean that algorithms tend to become intractable when the input is high-dimensional (see Domingos [91]). Many ML algorithms perform poorly in high dimensional vector spaces. This is because, as the dimensionality of the training samples grows: (a) those samples occupy a rapidly dwindling fraction of that space; and (b) distances between the samples become increasingly similar. Another concern is the range of values in each dimension, which must be adjusted to work within the constraints of finite-precision arithmetic in computer implementations. In ML terminology: each dimension is called a *feature*; each input takes the form of a *feature vector*; and the task as summarized above is called *feature engineering*.

Using feature engineering, we can aggregate another important source of information in ML systems: domain knowledge. In training an ML system for theorem proving, the accumulated knowledge from decades of research in Automatic Theorem Provers (ATP) may be particularly useful. Using our prior understanding, we can manually design new features and carefully tune or select the existing ones, thereby augmenting the training data with our domain knowledge.

Hypothesis and Contributions. In this chapter, we explore the following research question — How do we obtain features of theorems and strategies that are relevant for runtime strategy scheduling? A breadth of prior work explores feature engineering in the wider domain of ML for theorem proving, much of which may be relevant to strategy scheduling, though scheduling is rarely the primary target. Contrastingly, in this work, we produce manually engineered features extracted at low computational cost that are informative of the suitable strategy schedule for the goal theorem. We examine, thoroughly, each individual feature in the small collection of features we develop based on our experience and intuition. Our contribution here is the methodical development of features generally relevant to strategy scheduling using ML. Furthermore, the work in this chapter forms a critical prerequisite to training our ML models described in the

next chapters.

2.1 Background

2.1.1 Dimensionality Reduction

High-dimensional training inputs to ML algorithms lead to even more problems than the two mentioned in the beginning of this chapter. A consequence worth considering is the computational cost of training and prediction. We find even our intuition misleads us in high dimensions. For example, in high dimensional multivariate Gaussian distributions, the mass is not concentrated near the mean: instead it lies in a distant shell around it [91].

To counteract the effects of the curse of dimensionality, we may engage the *blessing of non-uniformity* (phrased by Domingos [91]). Typically, training data does not spread uniformly throughout the feature space; instead, much of the information relevant to ML is concentrated on or near a lower-dimensional manifold. We can reduce training and test data on to this manifold using, as introduced in Example 1.5: dimensionality reduction.

Example 2.2 (Simple Projections for Dimensionality Reduction)**.** Li, Hastie and Church [58] propose a simple method for dimensionality reduction titled *very sparse random projections*, that is able to preserve, in expectation, the pairwise distances between the input feature vectors. Suppose n feature vectors of D dimensions are stacked in an $n \times D$ feature matrix $\boldsymbol{A}$. They construct a $D \times k$ projection matrix $\boldsymbol{R}$ containing entries chosen randomly from the set $\{-1, 0, 1\}$ with probabilities $\{1/2\sqrt{D}, 1 - 1/\sqrt{D}, 1/2\sqrt{D}\}$ respectively. The projected data matrix $\boldsymbol{B}$, where

$$\boldsymbol{B} = \frac{1}{\sqrt{k}}\boldsymbol{A}\boldsymbol{R},\ k \ll \min(n, D),$$

has far lower dimensionality, and yet may be just as good for training an ML algorithm that, for example, attempts to maximize the margin of separation between the classes. ◁

In Example 2.2, dimensionality reduction is achieved by means of projection onto a manifold, using linear algebra alone. Principal Component Analysis (PCA), introduced in Example 1.5, although a more sophisticated method, is similar in the sense that it is also a linear projection. While such projections may offer the convenience of automation, a consequence is the loss of interpretability of the projected matrix $\boldsymbol{B}$, assuming that the matrix $\boldsymbol{A}$ was engineered with interpretability in mind.

In much of this work, we adopt a different approach to reduce dimensionality of the input data: we simply eliminate certain features. The choice of dropped features

is informed by our expertise and aided by statistical insights, which creates another route to incorporate domain knowledge in the training data. This method also retains interpretability of the original features. We, in this work, focus on the interpretability of the features that we engineer because it aids in the further examination and optimization of the ML algorithms that we develop, which make use of these features. This form of feature engineering is called *feature selection*.

2.1.2 Maximal Information Coefficient

In our work, often, we perform feature selection by identifying, and retaining, only those features that are, broadly speaking, most informative of the required response variable. In particular, we will usually examine each feature individually, rather than combinations of features, to avoid the combinatorial computational complexity of the latter. This, however, may result in the loss of relevant information, if there happen to be features that are informative of the response but only in combination with others. The potential for such loss depends very much on the data and ML models in use; and judgements on any actual losses can only be derived from experimental results. ML for strategy scheduling is the broader work presented in this thesis. Therefore, we are willing to sacrifice some learnable knowledge in exchange for other desirable factors, such as interpretability and performance; as long as we are able to eventually deliver improved strategy schedules.

To start with a simple hypothetical method, we can select features by examining the dependence between individual features and the response variable, and drop those that correlate poorly. The Pearson correlation coefficient (PCC), often simply called the *correlation coefficient*, is a standard measure of linear dependence between two random variables.

Definition 2.1 (Correlation Coefficient, r)**.** Let $x_1, \ldots, x_n$ and $y_1, \ldots, y_n$ be samples of two random variables each. Define, for the two variables, the correlation coefficient r such that

$$r = \frac{\sum_{i=1}^{n} (x_i - \overline{x})(y_i - \overline{y})}{\sqrt{\sum_{i=1}^{n} (x_i - \overline{x})^2 \sum_{i=1}^{n} (y_i - \overline{y})^2}},$$

where $\overline{x}$ is the mean value of all x_i, and likewise for $\overline{y}$. ◁

We may check each feature for selection using PCC. If the subsequent ML model to be used is approximately linear, this approach is reasonable. However, since PCC only measures *linear* correlation, we might miss features relevant to non-linear models. For the models we develop in subsequent chapters, we need to employ a non-linear measure of correlation for feature selection.

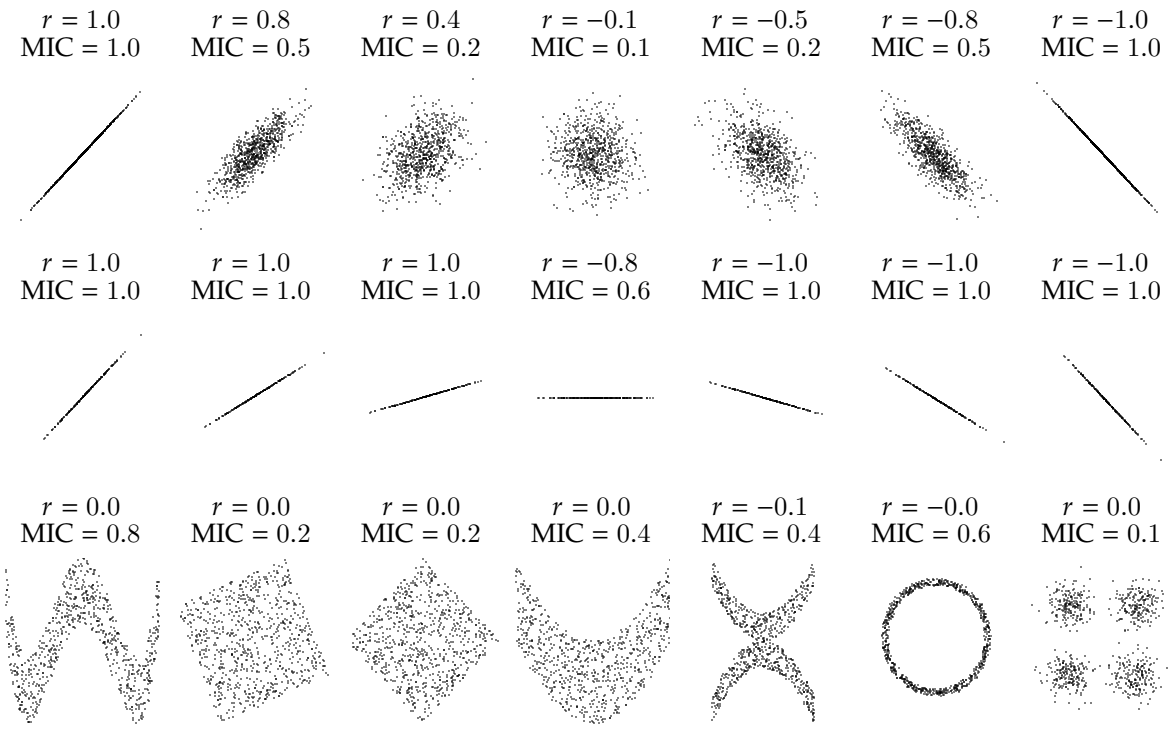


Figure 2.1: Pearson correlation coefficient, r, and the MIC of a few example bivariate distribution samples. Scatterplots display the samples.

For a measure of dependence between two random variables X and Y, that is more general than PCC, we can examine their *mutual information*. Mutual information is a measure of the discrepancy between the joint distribution $p(X, Y)$ and the factored distribution $p(X)p(Y)$.

Definition 2.2 (Mutual Information)**.** For discrete random variables X and Y, their mutual information (MI), $\mathbb{I}$, is

$$\mathbb{I}(X;Y) \triangleq \sum_y \sum_x p(x,y) \log\left(\frac{p(x,y)}{p(x)p(y)}\right).$$

Similarly, for continuous random variables

$$\mathbb{I}(X;Y) \triangleq \int_y \int_x p(x,y) \log\left(\frac{p(x,y)}{p(x)p(y)}\right) dx\, dy.$$

◁

Random variables X and Y are independent iff $p(X, Y) = p(X)p(Y)$ and thus iff the MI is zero. Furthermore, we always have $\mathbb{I}(\cdot) \geqslant 0$. Reshef et al. [88] employ MI to derive a statistic they call the maximum information coefficient (MIC): a more general measure of dependence between two random variables. As with any such summary statistic, MIC has its shortcomings (see the critiques by: Simon and Tibshirani [89]; and Gorfine, Heller and Heller [84]), yet Reshef et al. [88] show it is able to capture complex associations.

Furthermore, MIC, conveniently, falls between 0 and 1 and is symmetric. Figure 2.1 shows the MIC of a few examples, along with the correlation coefficient for comparison. Given its practicalities, we choose MIC to measure non-linear dependencies in our work.

2.1.3 Coefficient of Variation

In later sections, we intend to analyse how individual features are informative of the dispersion in the performance of strategies. Our focus is on the dispersion, and not so much on the indicators of the absolute performance of strategies, because our end goal is strategy scheduling and not strategy selection. Take for example, hypothetically, two theorems t_1 and t_2 and two strategies s_1 and s_2 of a theorem prover. Suppose the s_1 takes 10 s to prove t_1 whilst s_2 takes 11 s. Meanwhile, s_1 takes 1 s to prove t_2 whilst s_2 takes 5 s. We learn more about the relative performance of the two strategies by testing them on t_2, because of the relatively wider spread in runtimes. We need, for such purposes, a statistical measure of the relative dispersion between the performance of different strategies which is agnostic to their absolute performance. The coefficient of variation is one such standard statistical measure of dispersion (see Everitt and Skrondal [79]).

Definition 2.3 (Coefficient of Variation)**.** Define the coefficient of variation (CV or c_V) as the ratio of the standard deviation σ to the mean μ, that is

$$c_V = \frac{\sigma}{\mu}.$$

◁

CV has no units, is scale invariant and allows comparison between sets with different means. Thus, we find it useful when comparing the runtimes of strategies across different theorems.

2.2 Features of the Goal Theorems

We now describe the apparatus for our experiments: the LeanCoP prover, the Thousands of Problems for Theorem Provers (TPTP) problem library and the mechanism for comparing features.

2.2.1 Extracting Features of TPTP Theorems

We source our collection of test theorems from the TPTP library. The TPTP problems, introduced in Example 1.4, are taken from a range of domains such as Logic Calculi, Algebra, Software Verification, Biology and Philosophy, and presented in multiple

logical forms. In version 7.1.0, there are 8157 such problems covering 43 domains. Each problem consists of a set of formulae and a goal theorem. Problems can also source axioms, which are defined in separate files, for inclusion. Multiple TPTP problems may include the same axiom. The problems are of varying sizes. For example, the problem named `HWV134+1` from the hardware verification domain contains 128975 formulae, whilst `SET703+4` from the Set Theory domain contains only 12. We filter only the first-order formula (FOF) theorems for our experiments, because we test with only first-order provers.

TPTP is a common testbed and various methods of extracting features from TPTP problems have been developed in prior work, some of which we will discuss in Sections 2.4.2 and 2.4.3. One obvious source of features is the text of the problems and their included axioms. In this work, we extract features from this text exclusively. In some prior work (for example, by Hůla, Mojžíšek and Janota [152]), authors extract features from the raw text, using ML. Although this approach is popular, we develop a contrasting method. We parse the theorems to generate specially designed metrics; metrics that we believe are relevant to strategy scheduling, based on our experience.

We justify our approach to feature extraction by examining our intended ML application in detail. Our application is ML for runtime strategy scheduling in theorem provers. We intend to augment existing provers with the basic goal of saving proof time or proving more theorems. Since we plan to augment existing theorem provers, any runtime consumed by our additions will be time taken away from the total proof time. This is because, in typical use, there is a fixed time given to the prover per theorem. Thus, clearly, it is wise for us to minimize the prediction runtime of our ML algorithm; of which, feature extraction is the very first stage. Features based on a parse of the goal problem entail little runtime cost, because *it is necessary for the prover to parse the goal theorem*. We can augment the parser in the prover to emit additional statistics to derive our features from, and this can be achieved with negligible overhead. Additionally, we intend to take advantage of this manual feature engineering to incorporate our domain knowledge in the features. Eventually, we find this method delivers both relevant features, as we will see in this chapter, and ML systems that work, as shown in Chapter 4.

2.2.2 Strategies in the LeanCoP Prover

We have chosen to base our work on the *LeanCoP* prover: a compact yet performant classical first-order logic (FOL) theorem prover. The compactness of LeanCoP is an asset in this work: it brings subjective clarity to the interactions between our ML extensions and the base prover. Otten [80] achieves a compact implementation of LeanCoP by use of a proof calculus that has a direct translation to the Prolog language. LeanCoP uses a connection calculus, which is implemented in fifteen lines of Prolog to form

Table 2.1: LeanCoP search-control parameters (see Otten [80]).

Parameter	Effect
`nodef`	Transform goal into standard clausal form.
`def`	Transform goal into definitional clausal form.
`conj`	Use conjecture clauses as start clauses, when set.
`reo(I)`	Clauses are reordered `I` times.
`scut`	A restricted start step is used.
`cut`	Enable restricted backtracking.
`comp(I)`	Disable `scut` and `cut` when iterative deepening exceeds `I`.

the core of the prover whilst the whole totals around two hundred lines of code. For comparison, the performance leading `E` prover [38], version 2.6, is implemented in more than four hundred thousand lines of C code. Yet, LeanCoP performs well on the TPTP FOF problems, in comparison to other state-of-the-art provers (see Otten [80]). The total number of problems LeanCoP is able to prove is enough for scientific analysis of our results. Moreover, and especially relevant to our work, LeanCoP implements strategy scheduling.

LeanCoP controls its proof search by a collection of parameters, listed in Table 2.1. It accepts any combination of these parameters specified as a set, and any single set of parameters is a strategy. Some strategies, affected by their parameters, may be incomplete. Incomplete strategies may abort the proof search without consuming their time allocation when they reach a dead-end. In consequence, any strategy when tested on a goal theorem may lead to one of the following four possible outcomes.

Success: the strategy succeeds in proving the goal as a valid or invalid theorem. In doing so, it would have consumed some proportion of its allocated time.

Failure: an incomplete strategy reaches a dead-end and must abort the proof search with unconsumed time allocation to spare.

Timeout: the strategy runs out of time, consuming its entire allocated time.

Error: the strategy must abort due to an error, possibly with spare time remaining.

Definition 2.4 (Strategy Score)**.** For any given goal theorem, strategies may differ in performance due to: a different outcome, a change in the proof-time consumed, or both. For statistical analysis of such disparities, we need a metric to score the performance of a strategy on each goal theorem. Prior works suggest that the right strategy for the goal theorem will be quick to prove it (see, for example, Reger and Suda [126]). Our metric must therefore give a higher score to strategies that succeed in less time. Similarly, if a strategy fails to prove the goal, it would be better if it does so quickly, because a theorem prover with time left to spare after a failed strategy will then have the choice of trying another one. A timeout is the worst outcome: the goal remains unresolved and all the

Table 2.2: The strategy schedule in LeanCoP 2.0. Only the first and final strategies are complete.

Order	Strategy	Time Allocation
1	`{cut, comp(7)}`	2%
2	`{conj, cut}`	60%
3	`{def, scut, cut}`	16%
4	`{nodef, scut, cut}`	4%
5	`{conj, cut, reo(23)}`	2%
6	`{def, scut, cut, reo(29)}`	2%
7	`{conj, cut, reo(37)}`	2%
8	`{nodef, scut, cut, reo(41)}`	2%
9	`{conj, cut, reo(47)}`	2%
10	`{}`	All remaining

time allocated has been consumed. We condense these judgements into the following formula for the score. Suppose we attempt to prove a certain goal theorem using a specific strategy and examine the outcome as categorized in the paragraph above, along with the time t consumed by LeanCoP and the specified timeout T. We then calculate the score s as

$$s = \begin{cases} 0.5 + r & \text{on success,} \\ r & \text{on any other outcome,} \end{cases} \qquad \text{where } r = \frac{T - t}{2T}.$$

Note that $0 \leqslant s \leqslant 1$, and that a successful outcome leads to a score that is always greater than that for other outcomes. We calculate this numerical score for every strategy on each goal theorem in our repository. ◂

A strategy schedule in LeanCoP is simply a fixed sequence of strategies, along with a pre-defined proportion of time allocated to each strategy. Typically, the final strategy is specifically set to one that is complete, and is allowed to consume all the time left unused by previous strategies; which makes for a schedule that is complete overall. As soon as any one strategy in the sequence is able to prove the goal, LeanCoP can exit with that result, and thus the remaining strategies appearing later in the sequence are left unused. LeanCoP is published with a fixed strategy schedule. The original strategy schedule used in LeanCoP 2.0 is shown in Table 2.2. The schedule in later versions contains more strategies with more granular time allocations.

2.2.3 Judging Features

In following sections, we will examine the relationship between values of every feature we have designed and the performance of different strategies. In other works, for

the task of strategy selection for example, one may be interested in how the values of features predict the performance of each strategy. However, we are interested in strategy scheduling, and thus, we prefer features that are able to differentiate between strategies. Eventually, we will shortlist the most effective of such features to limit the dimensionality of our feature space. Below, we outline our methodology to examine individual features.

Suppose we have a collection of N TPTP problems and M LeanCoP strategies. In this work, we solely utilize the strategies set in the pre-defined fixed strategy schedule of LeanCoP. Thus, for example, if LeanCoP 2.0 was to be used in our experiments, $M = 10$. We use LeanCoP 2.1 in our experiments, which has $M = 30$ strategies. At first, we examine the performance of every individual strategy on each given theorem. We attempt to prove every problem using a modified LeanCoP prover, wherein we remove its strategy schedule and set it up with only one strategy of interest. We give each such attempt a timeout of 610 s. With this setup, we test every combination of problem and strategy, producing $M \times N$ outcomes. For every outcome, we calculate the score as described in Definition 2.4. Let $\{x_i \mid i \in \{1, \ldots, N\}\}$ be the set of problems. By now, for every problem x_i, we have scores $\{s_i^j \mid j \in \{1, \ldots, M\}\}$ that estimate the performance of individual strategies in trying to solve it. To obtain a measure of the variation in strategy performance per x_i, we calculate the coefficient of variation, c_i (see Section 2.1.3), of those M scores $\{s_i^\star\}$. The end result is — for each problem x_i, a dimensionless metric c_i that estimates the variation in the performance of LeanCoP strategies in the proof attempt of x_i.

Meanwhile, we also extract all the K features of interest for each of the same N problems. All of our features are counts and are extracted by parsing the problems. TPTP problems typically include axioms, which are also defined in the problem repository using the same syntax. When parsing a problem, we also parse all the axioms it includes when extracting features. Let us denote each feature by f^k, $k \in \{1, \ldots, K\}$, and the value for each feature extracted for problem x_i by f_i^k.

Now, consider any single feature f^k, in isolation. For every problem x_i, we have a value f_i^k for each feature f^k and, using the method in the above paragraphs, the metric c_i. With these values on hand, for each feature f^k, we construct two vectors: $[f_i^k]_{i=1:N}$ and $[c_i]_{i=1:N}$. These contain, respectively, the measures of the feature f^k and the corresponding dispersion metric, across the N TPTP problems. This brings us, finally, to the goal of this exercise, which is to examine the relationship between these two vectors. A stronger correlation between them indicates that f^k, on its own, will be more useful for strategy scheduling. To measure this, we calculate, primarily, the MIC between the two vectors (see Section 2.1.2); although occasionally, we also examine other metrics, like PCC, for a wider perspective.

In this chapter, we mainly analyse individual features in isolation. Note, however,

that complex combinations of some features may produce higher correlations with the metrics of interest. Linear models take advantage of linear combinations of features, whilst more complex models, such as those presented in subsequent chapters, are able to generate more complex combinations of features. These details need careful consideration when using the results presented in this chapter to build ML models.

2.2.4 Characteristic Subsets

TPTP is a large problem repository: version 8.1.2, for example, contains 9091 FOF problems. To complete the procedure in 2.2.3, we must run every problem for 610 s on each of the 30 strategies of LeanCoP, requiring a computer runtime of 46,213 hours. Each of the above evaluations also need to be comparable; thus, they must be run on machines of the same build and hosted by the same operating system in the same configuration. We may even repeat each evaluation to average-out any system induced noise. Such demand exceeds the computational capacity available to us, and so it becomes necessary for us to work with only a subset of the full TPTP suite.

A typical solution to the problem of a test-suite that is too large, is to derive random subsets of the tests, and hope that such a subset remains representative of the full suite. Fortunately, we can shed our reliance on hope alone and make use of qualified subsets of TPTP, thanks to work by Chvalovský and Jakubův [149]. They have produced specific subsets of TPTP which, when used for prover strategy engineering, ensure that any development using them will yield similar performance when tested with the full TPTP suite. The authors refer to them as *benchmark characteristic subsets*.

Chvalovský and Jakubův [149] construct the subsets by an ML approach that is fairly unconventional. The authors first cluster the TPTP problems, using several clustering algorithms such as k-means, and then select a few from each cluster. They control the desired size of the characteristic subset by setting the cluster count. The features that they use as input to the clustering algorithms are notably different from ours presented in this chapter. They have also evaluated their subsets and determined them to be better than random.

Our use of characteristic subsets is advantageous as well as clearly justified. The advantage gained is the ability to complete experiments within the available computation resources. Since the subsets have been derived in work independent of ours, we can discount the possibility of any implicit or accidental bias in the selection.

2.3 Evaluation of the Features

Aims. We turn our attention to feature evaluation. Our aim in this section is to examine a collection of manually developed features for their relative efficacy in ML for strategy

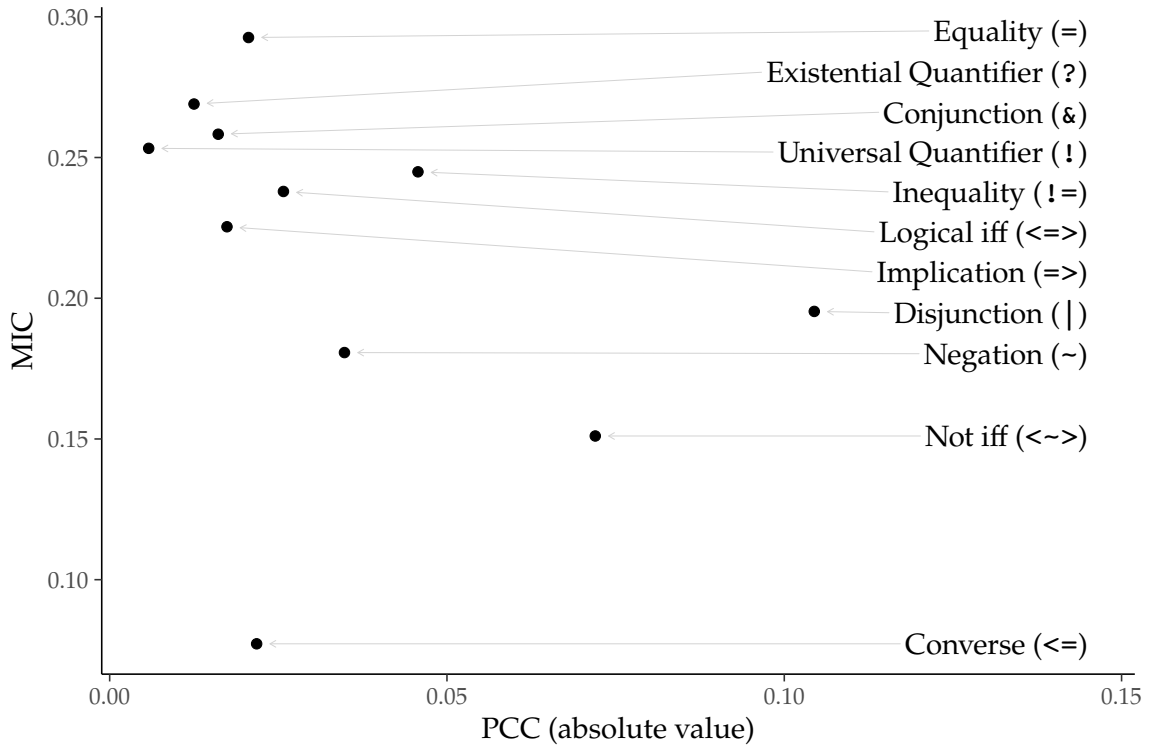


Figure 2.2: Relationship between each of the listed features and disparity in strategy performance in terms of the MIC and PCC. Higher is better, in both coordinates.

scheduling.

In the work presented below we make use of a characteristic subset of TPTP, version 7.1.0, consisting of 2423 theorems. This set was taken from the publicly available data accompanying the work by Chvalovský and Jakubův [149]. We executed LeanCoP using the ECL^{i}PS^{e} Prolog interpreter (see Schimpf and Shen [98]) version 5.10, on Intel Xeon Platinum 8368Q CPU machines clocked at 2.6GHz running the Linux kernel version 4.18. Occasionally in the text below, we will use "MIC" as a shorthand reference to the MIC between a feature's vector of values and the CVs of strategy scores.

2.3.1 Symbol Counts as Features

Based on our prior experience, we understand that logical connectives and quantifiers tend to influence strategy performance. In this section, we extract and evaluate their counts in TPTP theorems as features for strategy engineering. Extraction of these features is especially cost-effective in terms of computation: the symbols representing them can be counted during the lexical analysis stages of the target prover. Figure 2.2 plots these features using the method described in Section 2.2.3.

The symbols are listed in Figure 2.2 from top to bottom in decreasing order of MIC. This forms the order of relevance of these features to any non-linear ML algorithm.

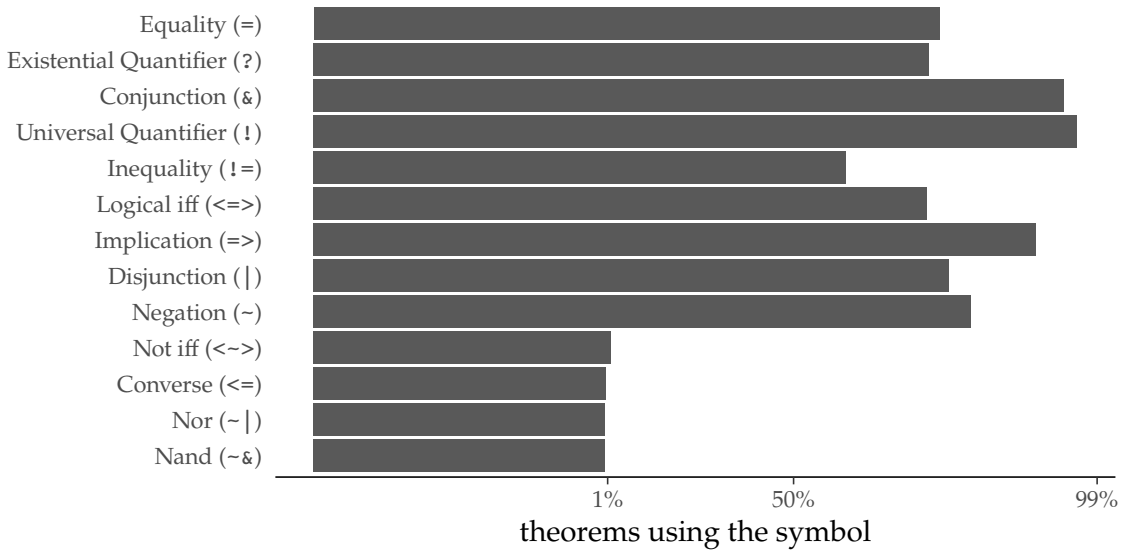


Figure 2.3: Proportion of theorems in which the listed symbols appear. Symbols are ordered by MIC, as in Figure 2.2. The symbols for logical "nor" and "nand" don't appear at all in our dataset.

Furthermore, Figure 2.3 provides an additional insight into these features by plotting their occurrences.

Remark 2.1. In Figure 2.3 we can see that the symbol for the logical converse is rarely used, yet it shows low correlations with our dispersion metrics. These observations contrast our data and model from what is typical in the field of Natural Language Processing (NLP), wherein rare words are stronger differentiators.

There is no clear correlation between MIC and PCC in Figure 2.2. Consequently, for a linear model, it may be desirable to use the PCC to short-list features. The symbol for "disjunction" is a case in point: it has the highest PCC, and thus the highest linear relationship with the variation in strategy performance. Yet, it ranks low in terms of MIC, and thus is less useful than a few others for a non-linear model. ◁

2.3.2 Quantifier-based Features

FOL theorems can make myriad use of existential and universal quantifiers. We conjectured, based on our experience, that the count, nesting, choice and order of the quantifiers influences the performance of strategies. Accordingly, we have designed features related to quantifiers that we will evaluate below.

A count of the quantifiers in the theorem is one elementary feature. It can be obtained, like the symbol counts in Section 2.3.1, by simple lexical analysis. It is also equal to the sum of two of the features in Section 2.3.1, those being the symbol counts of the quantifiers. Another feature we have chosen is the maximum depth of nesting of the

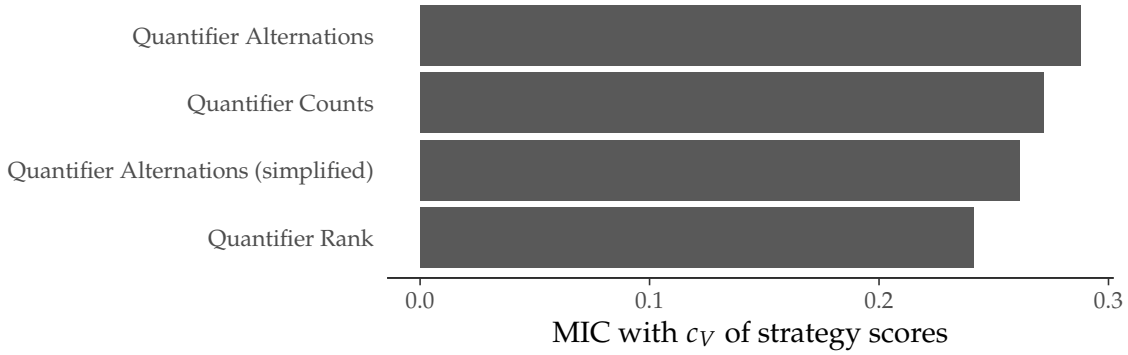


Figure 2.4: MIC of quantifier features.

quantifiers which, however, can only be obtained from a full parse of the goal theorem. A parse is also required for yet another feature — quantifier alternations. Alternating quantifiers in the theorem statement present, what we have observed, is a specific type of complexity that has varying influence on strategies. Our algorithms for extracting such features can be implemented as an augmentation of the parse routines in the target theorem prover, and thus a separate parse of the goal is not needed. This minimizes the computational cost of feature extraction.

To summarize, we present three features in this section:
Quantifier Counts: A count of the quantifiers.
Quantifier Rank: Maximum depth of nesting of quantifiers.
Quantifier Alternations: Number of times the quantifiers alternate.
All three are accumulated over an entire problem, including the text of the dependencies. For logical accuracy, when counting quantifier alternations, it is necessary to track negations and their logical effect on the quantifiers. Negation of the universal quantifier produces an existential, and vice versa. Our algorithm for extracting quantifier alternations is as shown in Algorithm 2.1. It is, however, reasonable to question the necessity of such logical accuracy for this feature. After all, the feature only needs to be stochastically relevant. To address this question, we extract a version of this feature using a simplified algorithm: we simply look for the quantifier alternations in the text of the theorem, ignoring the negations. We additionally present this simplified quantifier alternation feature. We compare the MIC of the features in Figure 2.4. Their PCC is roughly the same.

Remark 2.2. In Figure 2.4, we observe the MIC of the four features defined in this section. The accurate version of the quantifier alternations feature offers notably higher correlation with the dispersion in strategy performance. Quantifier rank, on the other hand, perhaps unexpectedly, seems to offer the poorest correlation; nonetheless it does offer some and so may still be useful. ◁

Algorithm 2.1: *Count quantifier alternations.*

Create a stack Γ capable of holding variables $\gamma \in \{\neg\neg, \neg\}$ which represent an environment that is either positive, $\neg\neg$, or negative, $\neg$.
Initialise the environment stack with a positive environment: $\Gamma \leftarrow [\neg\neg]$.
while we traverse the parse tree of the goal in depth-first order **do**
  Let γ be the environment on top of Γ.
  if we enter a subtree of a symbol containing `~`, the text for negation, **then**
    Flip γ, that is $\gamma \leftarrow \neg\gamma$.
    Push γ onto Γ.
  else if we exit a subtree of a symbol containing `~`, **then**
    Pop from top of Γ into γ to recover the previous environment.
  else if we visit a quantifier $\lambda \in \{\exists, \forall\}$, of text `?` or `!` respectively, **then**
    if the environment is negating, that is $\gamma = \neg$, **then**
      Flip λ to the other quantifier.
    end if
    if the previously encountered quantifier was different from λ **then**
      We found a quantifier alternation, add to the count.
    end if
  end if
end while

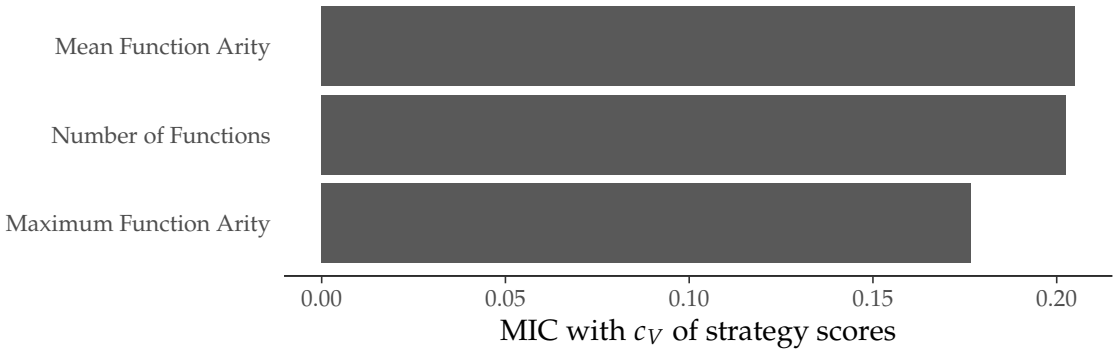


Figure 2.5: MIC of features based on functions in the theorem.

2.3.3 Function-based Features

We have designed three function-based features: mean function arity, maximum function arity and the number of functions. We certainly need to parse the theorem to extract these features. However, it would be easy to augment provers to collect these statistics while parsing the theorem: functions and their arguments just need to be counted. Whilst the PCC of these features is roughly the same, their MIC varies notably and is presented in Figure 2.5 for comparison.

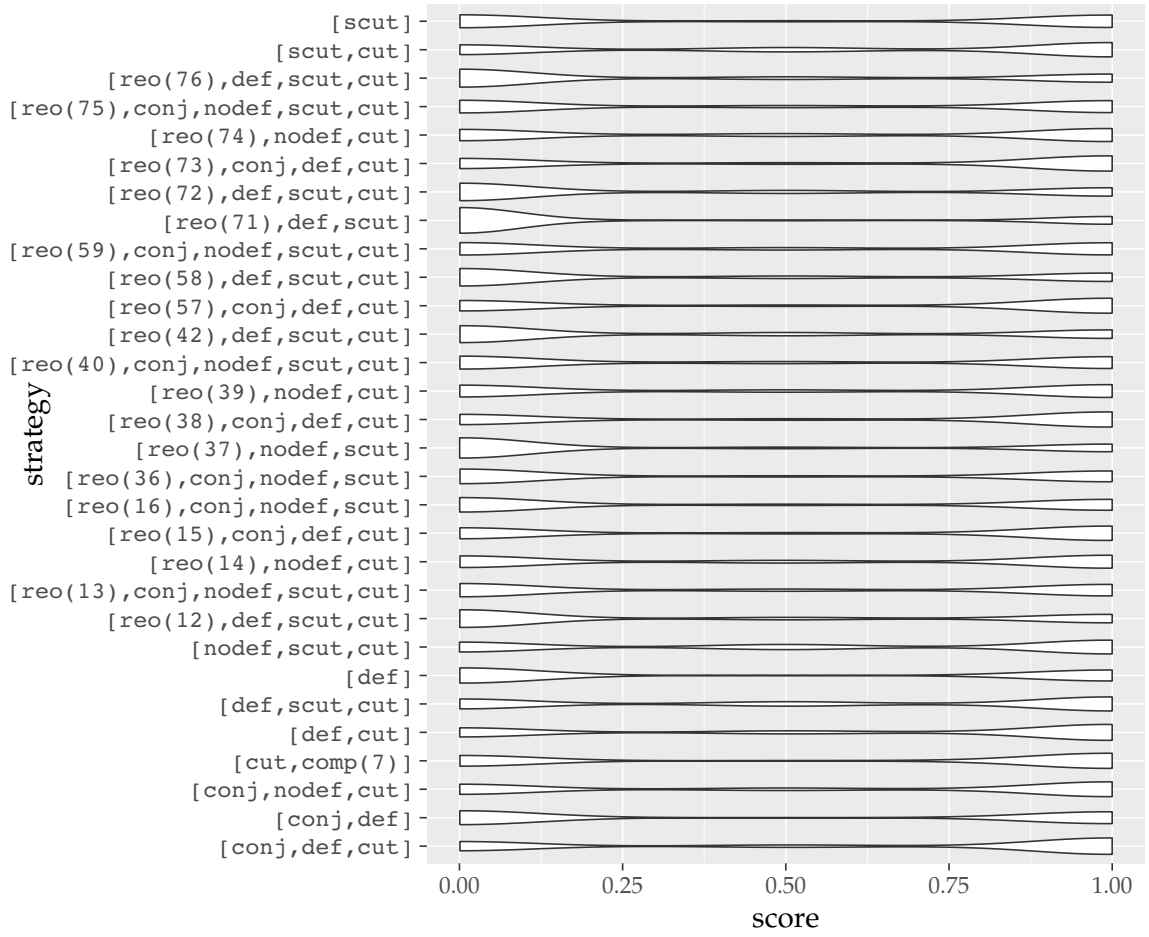


Figure 2.6: Violin plots, i.e., compact density plots, to compare the scores (see Definition 2.4) of LeanCoP strategies.

Remark 2.3. All three features shown in Figure 2.5 offer some information on strategy dispersions and are useful, although they seem less useful than the quantifier-based features. ◁

2.3.4 Significance Tests

In the last few sections, we have examined a collection of features for their correlation with our metric of interest: the breadth of variation in strategy performance, as explained in Section 2.2.3. The correlation metrics are comparable across features, and thus we are able to rank features. While this ranking is useful for developing ML models in strategy scheduling, it does not offer scientific clarity on the efficacy of these features. In this section, we shift our focus to the latter.

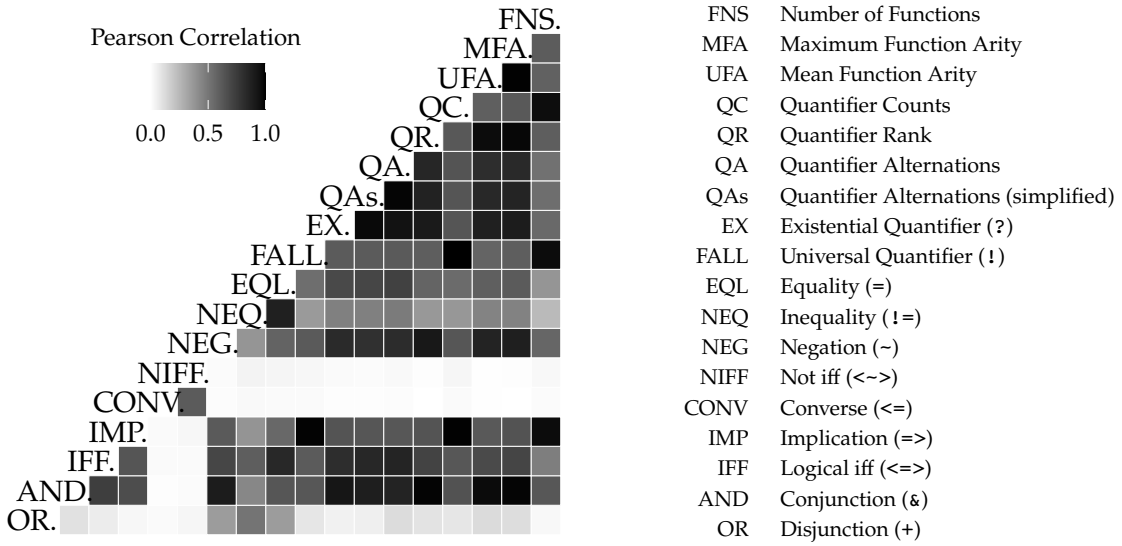


Figure 2.7: Correlations between the features, in terms of absolute PCC.

Strategy Scores

We start by examining the strategy scores themselves, which are plotted all together in Figure 2.6. The scores, clearly, tend to cluster close to either 1 or 0. This implies that LeanCoP strategies follow the common pattern observed across provers — the right strategy for the goal theorem tends to prove it quickly (see Reger and Suda [126] and Definition 2.4). Clearly, we cannot expect the scores to follow a normal distribution, but as is customary, we test for it. With the null hypothesis that the strategy scores are normally distributed, a Shapiro-Wilk test delivers a vanishingly small p–value for each strategy, thus confirming that the scores are not indeed normally distributed.

Next, we check if the choice of strategy has a significant effect on the score. We make use of non-parametric tests as the scores are not normally distributed. With the alternative hypothesis, H_1, that the strategy has a significant effect on the score, a Kruskal-Wallis rank sum test returns p–value $< 2.2\text{e}{-16}$, thus confirming H_1.

We, however, obtain a more complex result when we compare the strategies in pairs. With the alternative hypothesis, H_1, that each pair of strategies obtain significantly deviating scores, we perform a pairwise Wilcoxon rank sum test with continuity correction. With 30 strategies, this produces 435 p–values, of which 181 are below 0.05, thus confirming that 181 pairs of strategies produce significantly deviating scores. Those pairs with significantly deviating scores do contain all 30 strategies.

Features

We turn our attention to the features themselves, and start by checking if they are normally distributed. A Shapiro-Wilk test on all of them reveals that they are not, with

a vanishingly small p–value, just like the scores above. However, as expected, they do correlate with each other to varying degrees, as expressed in Figure 2.7.

Remark 2.4. Irrespective of the significance of such correlation between individual pairs, any ML model using these features could benefit by adjusting for these correlations. Unlike the strategy scores, the features vary in scale and range. And thus, without modifications, any test of significance between them is uninteresting. ◁

Significant linear relationships between the features and strategy scores do emerge, which we have discovered through a manual search. Consider the following linear regression model:

$$y = \beta_0 + \beta_1 f^1 + \ldots + \beta_k f^k + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma). \tag{2.1}$$

For any TPTP theorem x_i, therein, $y = c_i$ is the CV of the strategy scores, $[f^k]_{k=1:K}$ are the features of the theorem, $\beta_{i,i\in\{1,\ldots,n\}}$ are the model parameters, β_0 is the intercept and ϵ accounts for noise. We estimate the parameters using the method of least-squares. We can check the efficacy of any feature by creating a second model that omits that feature, and then using a t test to check for the null hypothesis that the two models have the same mean. To check the whole model, we use an F test. The above procedure is standard regression analysis.

Using all the features defined in Section 2.3, we examine the model above. The model proves to be significant since the F test produces p–value < 0.05. However, only four features emerge as individually significant with a t test resulting in p–value < 0.05, namely — Equality (=), Disjunction (|), Not iff (<~>) and the Existential Quantifier (?). Alternative model specifications produce different sets of significant features. For example, in a model containing only the four significant features listed above, only one feature, Disjunction (|), emerges as significant, even though the whole model is still significant, as per the F test. When we construct another model containing only Disjunction (|), Existential (?) and the mean function arity: all three features are significant, and so is the model.

Remark 2.5. The variation in the sets of significant features above is a consequence of the methodology we have used. The model is a simple linear regression, and we check each feature for significance individually. The results above cannot be interpreted to imply that the other features are unimportant, as they may be non-linear predictors of the score. Furthermore, there are linear correlations between them (see Figure 2.7). What we do have are indications that these features are useful for predictive strategy scheduling, even with linear ML models. ◁

2.4 Related Work

2.4.1 Comparision with the iLeanCoP prover

The LeanCoP prover has an intuitionistic sibling — iLeanCoP — which is written by the same author and is a similarly compact connection prover (Otten [55]). It generates proofs of the goal theorem using intuitionistic first-order logic (introduced in Section 1.1.2), as opposed to classical logic. In latter chapters, we test with iLeanCoP because we are interested in exploring automation for intuitionistic logic (also noted in Section 1.1.2). In this chapter, on the other hand, we used LeanCoP to generate our data to take advantage of the fact that, in comparision with iLeanCoP, at least in newer versions, it is shipped with a bigger set of pre-defined strategies and it is able to prove more of the TPTP theorems. This produces a richer dataset that is more amenable for the detailed scientific analysis we have presented. In this section, we will examine how the same features relate to the performance of iLeanCoP strategies.

We run TPTP theorems on all iLeanCoP strategies, with a timeout of 610 s, to generate the dataset examined below. We use iLeanCoP version 2.0, which contains only five strategies, of which two are logically complete. Instead of limiting the set of theorems to a characteristic subset, we test on the whole TPTP library. However, for analysis, we ignore any theorems that cannot be successfully solved by any of the strategies; because such theorems do not offer any meaningful insight for the strategy scheduling systems we evaluate in later chapters. That leaves 2241 theorems that we do consider. Between these and the dataset we used earlier for LeanCoP, 352 theorems are common. The common intersection of theorems only needs to be used for pair-wise analysis of the two provers; otherwise, we consider all theorems of both datasets.

We start by checking if different strategies do significantly affect the performance of iLeanCoP across the TPTP theorems, just as we did for LeanCoP. For every theorem-strategy combination, we calculate the score as per Definition 2.4. As the scores are not normally distributed, the following tests are non-parametric. With the alternative hypothesis, H_1, that the strategy has a significant effect on the score, a Kruskal-Wallis rank sum test returns p–value < 0.05, thus confirming H_1.

Next, we check all pairs of strategies. With the alternative hypothesis, H_1, that each pair of strategies obtain significantly deviating scores, we perform a pairwise Wilcoxon rank sum test with continuity correction. With 5 strategies, this produces 10 p–values, of which 6 are below 0.05, thus confirming that 6 pairs of strategies produce significantly deviating scores. Those pairs with significantly deviating scores do contain all 5 strategies.

Let us now examine the relationships between our features and the iLeanCoP strategy scores. In Figure 2.8, we see how our features correlate with the c_V of scores in the

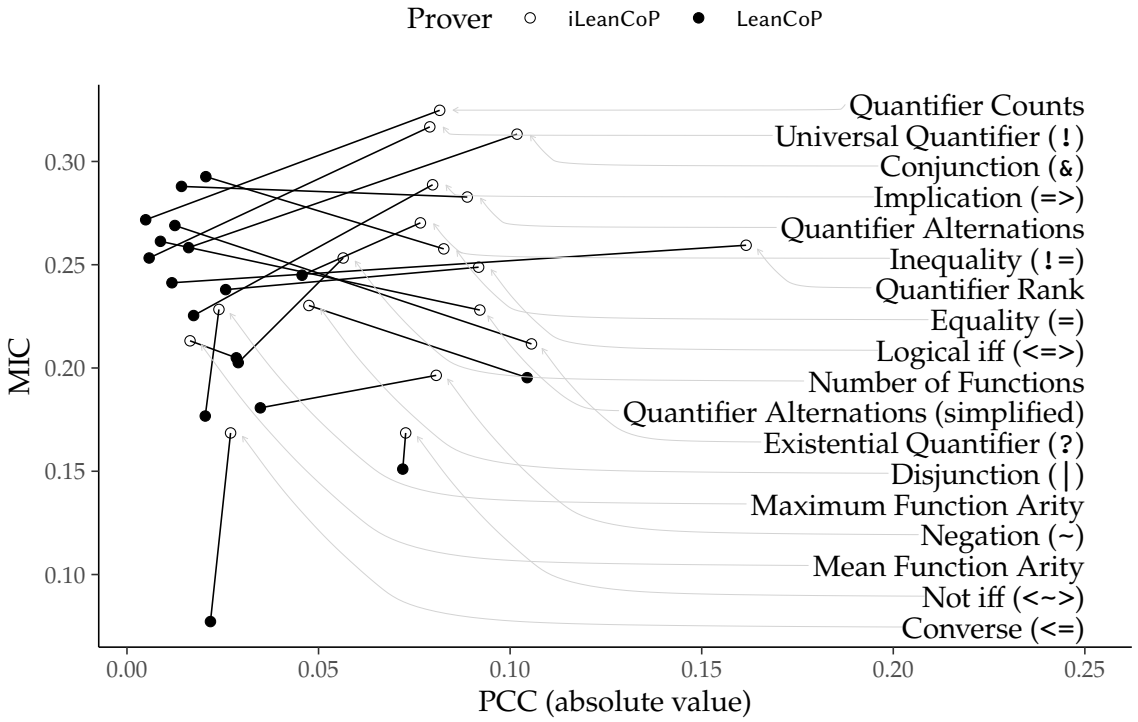


Figure 2.8: Relationship between our features and disparity in strategy performance of both LeanCoP and iLeanCoP, in terms of the MIC and PCC. Higher is better, in both coordinates. Line segments connect the same feature of both provers.

iLeanCoP dataset. The figure also compares that to the same for LeanCoP. We see, in comparison with LeanCoP, a notable increase in the MIC and PCC values of nearly every feature: PCC only decreases for disjunction and mean function arity, whilst MIC decreases for equality, existential, and quantifier alternations.

Our final examination of iLeanCoP is through the linear regression model of Equation (2.1). With the response being the c_V of iLeanCoP strategy scores for each TPTP theorem, and the input being the corresponding theorem features, we fit a linear model using our data. An F test of the model produces p–value < 0.05, proving the linear model to be significant. Furthermore, the following features, in increasing order of p–value, emerge as individually significant with p–value < 0.01: Quantifier Rank, Conjunction (&), Mean Function Arity, Inequality (!=), Logical iff (<=>), Negation (~), Not iff (<~>) and Quantifier Alternations.

Remark 2.6. In conclusion, our features appear to be just as relevant to strategy scheduling in iLeanCoP as they were in LeanCoP. ◁

2.4.2 Comparison with alternative features

Thus far, we have analysed features that we designed using our own method. Above, we have shown the portability of our system across to a very similar prover — from LeanCoP

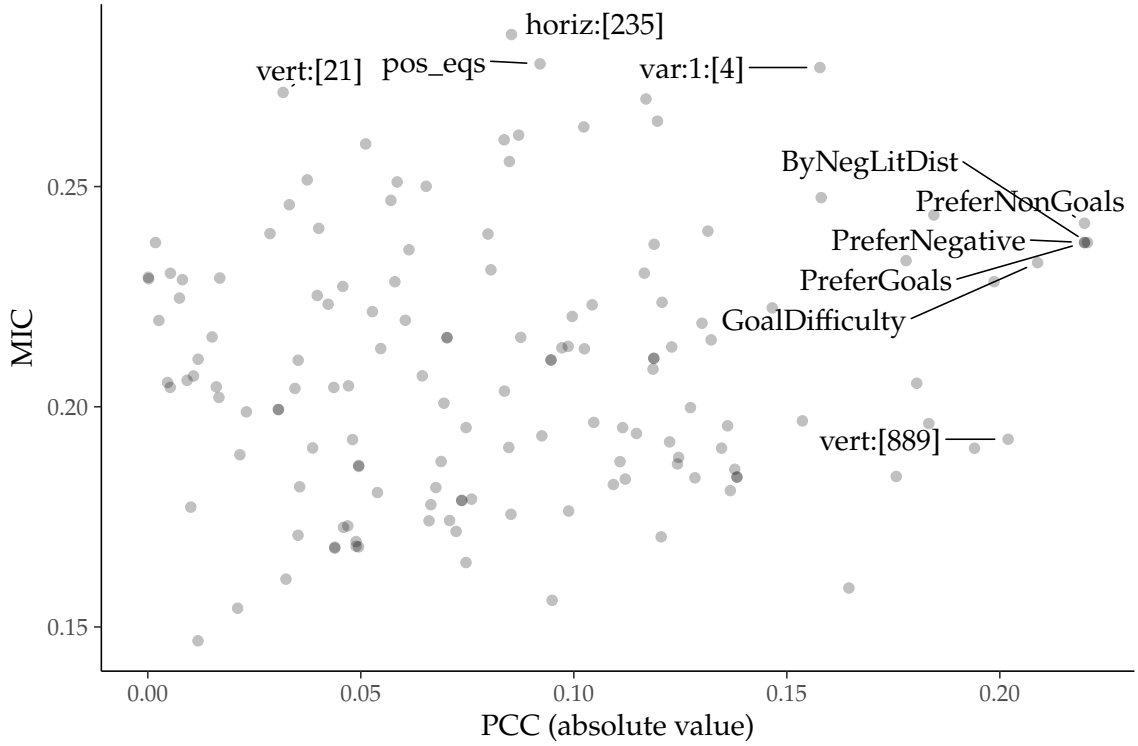


Figure 2.9: How the ENIGMA features correlate with the dispersion in LeanCoP strategy performance. Relatively sparse features are omitted, and the ones with the highest correlations are labelled.

to iLeanCoP. Here, we will review how our methods apply to an entirely different set of theorem features; ones that were produced by other authors.

Jakubův and Urban [124] have presented work with the title *Efficient learNing-based Internal Guidance MAchine*, or ENIGMA in short, in which, for the purpose of ML, they produce certain features derived from a set of clauses. We will refer to those features as the ENIGMA features. For the purposes of generating the characteristic subsets, which we explained in Section 2.2.4, Chvalovský and Jakubův [149] use ENIGMA features extracted from the set of clauses in the TPTP goal. In the ENIGMA feature set, feature vectors of a clause are the sum of the feature vectors of literals in the clause. Features of literals, in turn, are derived from the tree of symbols in its syntax. Trigrams of symbols connected by edges of the tree form individual features. To constrain the total number of features, some symbols, for example variable names, are substituted by the same special symbol repeated over. An example feature is the trigram $(g, \odot, \otimes)$, which consists of the function g, a Skolem symbol replaced by $\odot$ and a variable replaced by $\otimes$. In addition to such trigrams, the ENIGMA features also include a small collection of nominal metrics. ENIGMA features are similar, in at least one aspect, to the features we have presented earlier in this chapter: they are derived from the parse tree of the goal; the diverging methodology, however, certainly produces a very different feature set.

Chvalovský and Jakubův [149] freely publish ENIGMA features along with characteristic

subsets of TPTP, which we conveniently make use of. We stick to the subset we have used thus far in this chapter, for which the published features are 4221 in count. In the context of our research question, we examine the efficacy of these features for the purpose of strategy scheduling, using the methodology we have presented thus far. With our data generated using LeanCoP, we can examine how these features correlate with the divergence in strategy performance, which we have presented in Figure 2.9. These features tend to be very sparse, so we plot only those that appear in at least half of the theorems in the characteristic subset.

For some analysis, we review the two features labelled in Figure 2.9 with the prefix 'vert'. These are "clause vertical features", i.e., the trigrams mentioned above, categorized into 1024 buckets. Bucket no. 21 contains fourteen trigrams, one of which, for example, is `-:p1:?f800`, which refers to the sequence: negative literal, predicate of arity one and a function of 800 variables. A less esoteric example from the same bucket is `+:p4:?f3`. An example from bucket no. 889, which contains eleven trigrams, is `+:p2:*`, which is a positive literal, followed by a binary predicate and then a variable.

Although the ENIGMA features have been designed with the benefit of substantial insight in the domain of ATP, subjective analysis of individual ENIGMA features becomes difficult due to their sheer quantity. Our method enables us to prioritize the features for such analysis, and may offer the authors of ENIGMA further insight into their results, which could be used for further optimization. Our method also benefits the task of dimensionality reduction. One may shrink the dimensions of the features by using a method such as PCA, as mentioned in Section 2.1.1, but the shortened feature vectors do not retain the domain-specific interpretability of the original features. With our method, the lower priority features can simply be truncated to limit the feature space.

2.4.3 Automatic Feature Extraction

In an approach that contrasts with ours, Hůla et al. [152] use graph neural networks (GNNs) to automatically discover features for ML. They combine this feature extraction with training of a regression model. They achieve an increase in the number of problems solved as well as a reduction in the total proof time. Meanwhile, our use of manual feature engineering combined with statistical methods for selection has certain advantages. For one, we can analyse our features and derive a subjective interpretation of their efficacy.

2.5 Conclusion

In this chapter, we have presented a collection of manually engineered features that are effective at predicting the divergence in the performance of different prover strategies.

Those features are designed to be extracted, with minimal overhead, while a prover parses the goal theorem. We have also presented methods for evaluating those features, which could be used to short-list them for the specific learning algorithms in use. We presented an evaluation of those features for the LeanCoP and iLeanCoP provers tested on the TPTP library.

Our features effectively impart our domain knowledge onto any ML model that uses them. Such domain knowledge may not be available in the raw data itself. The methods of evaluation presented here are also applicable to other feature sets, should they be for the purpose of strategy scheduling. For large feature sets or automatically generated ones, our methods offer additional insights into individual features by statistical analysis. And, the features we have presented should be relevant for ML-based strategy scheduling in other FOL provers. Manual feature engineering such as ours can be combined with automatic feature extraction to reap the benefits of both. Altogether, the work presented in this chapter is a strong foundation for feature engineering for the purpose of automated strategy scheduling.

3

Optimizing Prover Strategies

> We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil.
>
> *Tony Hoare* [75]

Strategies in Automatic Theorem Provers (ATPs) are essentially specific parameter settings. Prover strategies are useful because the optimal parameter choice is sensitive to the class of target problems. Sinz and Iser [77] show, for example, how different classes of SAT problems influence the restart parameters of the DPLL algorithm. Modern solvers offer, for high performance, a portfolio of strategies to target a wide variety of proof goals. The E-prover, for example, bundles with it more than eighty strategies (see Schulz [38]). Here, we investigate some methods for inventing strategies. Strategy invention bears close resemblance to the general task of parameter optimization in theorem provers; and that is how we approach it in this work.

Effective strategy invention is an expensive task. The performance of individual strategies can only be deduced by testing: a costly endeavour when targetting a theorem repository such as the Thousands of Problems for Theorem Provers (TPTP, see Example 1.4). Take MiniSat for example, which, as mentioned in Example 2.1, offers dozens of parameters, in spite of its relatively small codebase. Even a handful of parameters, such as those in LeanCoP, which is designed specifically for a compact implementation, can produce a large parameter space. Strategies in LeanCoP version 2.0 consist of six parameters, two of which are integers while the remaining four are boolean. Limiting the integer parameters to a maximum value of 10 leads to a total of 1600 possible strategies. Such

width becomes expensive when each strategy needs extensive performance testing. For a contrastingly large alternative, consider the Lingeling SAT solver, which has more than three hundred parameters (see Biere [106]).

Our work joins a large chorus of prior work on automatic parameter optimization of provers. We can relate to work by Hutter et al. [64], who used ParamILS to optimize parameters for their SAT solver Spear, which yielded speed-ups, in comparison to manually tuned parameters, of 4.5 times on bounded model checking instances and 500 times on software verification problems. ParamILS is an automatic parameter optimization tool for general algorithms, developed by Hutter, Hoos and Stützle [63]. ParamILS begins with a random collection of parameter settings, searches around the neighbourhood of these parameters, and uses a system of randomization and restarts to avoid getting lost in local minima. In addition to being a black-box method, it is also *model-free*, since it does not explicitly attempt to model the algorithm it is optimizing. In subsequent work, Hutter et al. [86] also developed an influential *model-based* method, called the Sequential Model-based Algorithm Configuration (SMAC). SMAC attempts to relate the parameters of the target prover to its performance using a model, which then guides the parameter search. It builds the model using Machine Learning (ML) methods.

Our method of choice for this work is Bayesian optimization: a model-based method, wherein a probabilistic model of the optimization target is constructed and improved iteratively, and that model is used to guide the parameter search. We are motivated by our research questions to investigate this method, to which its relationship is direct since it is rigorously built on Bayesian statistics. Although published research on the use of Bayesian optimization for provers is limited, it has been successfully utilized in varied disciplines and is a topic of active research in the machine learning community. Shahriari et al. [118] have noted some interesting applications, such as A/B testing, policy parametrization for robots, highway traffic congestion management and combinatorial optimization. Oh, Yang and Yi [115] have published work related to our work presented in this chapter, on the use of Bayesian optimization for adapting a static analyser to given programs.

3.1 Bayesian Optimization

Bayesian optimization is a model-based black-box optimization method, suitable for optimizing targets that are expensive to evaluate and lack an analytical form. The optimization target is modelled as an *objective function*, which maps from the parameters of the target system to a performance metric. The typical performance metric, in our case, for theorem provers, is the wall-clock time consumed by it to find a proof. In this work, we will often refer to the target system being optimized as the objective.

Since the objective is a black-box, the shape of the objective function is modelled using *Gaussian Process* (GP) regression (see Rasmussen and Williams [60]), a machine learning method that produces a non-linear regression model using Bayesian statistics. Instead of modelling the objective as a single function, the GP models it as a probability distribution over functions. In effect, every point in the parameter space is mapped by the GP to a probability distribution over the value of the objective function.

Since Bayesian optimization uses a machine learning model of the objective function, training data must be produced by testing the objective, initially at random. Subsequently, the optimization process cycles between prediction, testing and training. Using the predictions from the latest model, a point of interest in the parameter space is chosen for testing, based on a criterion explained later in this section. The point is then evaluated on the objective. The result is a new item of data for the GP model to learn from, refining its estimate of the objective. This process is repeated until a pre-defined resource budget is consumed. The resource budget may be, for example, a fixed number of iterations or wall-clock time. The persistent goal throughout the process is to intelligently search for better performing parameters.

Suppose the noisy black-box objective we wish to optimize using Bayesian optimization is f, such that

$$y = f(x) + \epsilon.$$

Our goal, therefore, is to find the point x that minimizes $f(x)$, where x is vector valued. In our setting, x is a vector of parameter values, f measures the time spent in the prover, and the noise ϵ is due to the inherent non-determinism in the underlying system which influences that measure. After some initial evaluations of f, Bayesian optimization progresses in the following steps.

1. We treat all previous evaluations of f as training data and use a machine learning algorithm to build a predictive distribution of the values of y for any given x: $\Pr(y|x)$. With the use of a GP model, this distribution is Gaussian:

   $$y|x \sim \mathcal{N}(\mu(x), \sigma^2(x)).$$

   We build this distribution instead of a point estimate because many estimates of f will interpolate the points we have evaluated thus far. Two additional observations reinforce the need for this design: our measurements of the objective may be noisy; and the objective may be expensive to evaluate. When the objective is expensive to evaluate, as is the case with the work presented here, models of the objective will be derived from only a limited number of previous evaluations, and so we must account for the wider distribution of functions that could interpolate them. This is achieved by estimating the variance in addition to the mean.
2. We then use this distribution to build an *acquisition function*, α, such that

   $$\alpha(x) = \mathbb{E}_{p(y|x)}[U(y|x)], \tag{3.1}$$

where U is the *utility function*, discussed in further detail below. The acquisition function is an inexpensive function that measures the expected utility of collecting data about any point x. The point x' that maximizes $\alpha(x)$ is therefore the point to evaluate next.

3. We evaluate the objective at point x', which adds to our training dataset, and we return to step 1. However, if the pre-allocated resources for the optimization process are exhausted, we terminate the optimization process. The minimum is extracted from the collection of points evaluated so far.

Exploration Versus Exploitation

Given a model of the objective function, consider the *utility* of evaluating points in the parameter space. The expectation as well as the uncertainty of the performance of any point in the parameter space will vary. Some regions of the parameter space may be predicted with high expectation and low variance, wherein it may be useful to *exploit* the knowledge in the model in search of an optimal point. Alternatively, it may be beneficial to *explore* the parameter space in regions of high uncertainty in the model, to reduce the uncertainty in the model and to reveal potentially hidden optimal points. Exploration and exploitation are trade-offs within the pre-determined resource budget allocated for the optimization process and must be carefully balanced.

Utility Function

The choice of the point of interest during Bayesian optimization at any stage is the point that maximizes the acquisition function. This acquisition function maps the model of the objective, which is a distribution over functions, to a single function which measures the expected utility of any point in the parameter space. The acquisition function is simply the mean of a user-defined *utility function*, as shown in Equation (3.1). One uses the utility function to control the balance between exploration and exploitation. As explained before, we model our objective using GP regression and thereby obtain a distribution over functions f:

$$\Pr(f) = \mathcal{GP}(f; \mu, K)$$

where $\mu(x)$ is the mean function and $K(x, x')$ is the kernel covariance function. We discuss kernel covariance functions in Appendix 3.A.

A commonly used utility function is *expected improvement*. Suppose the minimum value of the objective that we have observed so far is v. The expected improvement utility function is defined as:

$$U(x) = \max(0, v - f(x)).$$

This function rewards improvements proportionally, but does not reward degradations.

Discrete parameters

The Bayesian optimization framework presented thus far is defined for continuous arguments to f. Theorem provers, however, often have discrete parameters. One solution is to map the discrete parameters to integers, and yet model them in the GP regression as any other real-valued parameter. When the objective needs to be evaluated, values supplied for the discrete parameters can be rounded to integers. Rounding is a common workaround for integer parameters, but it can lead to problems, since the points that are actually evaluated are different from the points that are modelled in the GP, as shown in work by Garrido-Merchán and Hernández-Lobato [144]. Their suggested solution is to modify the kernel covariance function $K(.,.)$ in the GP to

$$K'(x_i, x_j) = K(T(x_i), T(x_j)), \tag{3.2}$$

where $T(x)$ rounds all integer-valued variables to their closest integer.

In this work, we have used commercially available Bayesian optimization frameworks that provide built-in facilities for integer parameters. In MATLAB, specifically, such integer parameters receive a treatment that contrasts from the modification suggested above. MATLAB models those integer parameters as if they were real-valued in all stages of the Bayesian optimization process, except for the stage in which it chooses the next point to evaluate. Ideally, that should be the global maximum of the acquisition function $\alpha(x)$ in Equation (3.1). At that stage, MATLAB:

1. samples $\alpha(x)$ at one-thousand random points;
2. then rounds the integer parameters of those points to the nearest ones;
3. chooses ten points with the highest value of $\alpha(\cdot)$;
4. conducts a local optimization around those ten points on the function $\alpha'(\cdot)$, where
$$\alpha'(x) = \alpha(T(x));$$
5. and finally; chooses the best of the ten local maxima for the next candidate to evaluate the original objective function at.

For the above local optimization in step 4, MATLAB uses the Nelder-Mead method (see Nelder and Mead [8]). Despite the discontinuous shape of $\alpha'(\cdot)$, the Nelder-Mead method, being a gradient-free method, is able to search for the local minima of $\alpha'(\cdot)$.

3.2 Optimizing Premise Selection

When proofs may call upon a large collection of axioms, lemmas or theorems, good premise selection, introduced in Section 1.3.1, improves the chances of success for a prover. Premise selection is, therefore, an important component of many prover strategies. In this section, we show the effectiveness of Bayesian optimization as a method for finding good parameter settings for an example premise selection algorithm.

The Heuristic

We have chosen the Sumo INference Engine (SInE) premise selection algorithm, mentioned briefly in Section 1.3.1, for the experiments presented here. SInE is a popular and effective premise selection algorithm. In SInE, symbols appearing in the goal theorem *trigger* lemmas that become the premises of that goal. This *trigger* relationship is defined as follows. Let $occ(s)$ denote the number of lemmas in the library in which symbol s appears. Let $trigger(s, A)$ denote the aforementioned trigger relationship, such that it holds when symbol s triggers lemma A. Consider the least general symbol s' in lemma A such that

$$occ(s') \leqslant occ(\chi), \quad \forall \chi : \ \chi \in \text{symbols of } A.$$

The central heuristic of SInE is that a lemma is only triggered by the least common symbol occurring in it, that is, $trigger(s', A)$ holds. However, this heuristic alone is too fragile and small differences in $occ(\cdot)$ of symbols may cause important lemmas to be missed. Therefore, two heuristic parameters are added to allow more of the rare symbols in lemmas to act as triggers. A *tolerance threshold* $t \geqslant 1$ and a *generality threshold* g are added to the formulation; with which the trigger relationship is updated such that

$$trigger(s, A) \Leftrightarrow \{occ(s) \leqslant g\} \vee \{occ(s) \leqslant t \cdot occ(\chi), \forall \chi : \ \chi \in \text{symbols of } A\}.$$

Lemmas immediately triggered by symbols of the goal may contain more symbols that are further indicators for other relevant lemmas. SInE captures this idea by design. A lemma or a symbol is triggered if it is k-step triggered for some $k \geqslant 0$ such that:

1. Symbol s is 0-step triggered if s is a symbol in the goal.
2. Lemma A is k+1-step triggered if s triggers A and s is k-step triggered.
3. If A is k-step triggered and s occurs in A, then s is also k-step triggered.

The SInE algorithm therefore has three parameters:

- the tolerance threshold t,
- the generality threshold g, and
- the maximum trigger steps k, also called the *depth*.

It is reasonable to evaluate the trigger relation to infinite depth, which in some theorem provers is a good default (see evaluation by Hoder and Voronkov [85]).

The Data

We have prepared a dataset of premises, theorems and proofs that we use to test our implementation of SInE. We extract this data from the Archive of Formal Proofs (AFP), mentioned briefly in Section 1.3.2. The AFP is a large and expanding collection of proofs formalized in the Isabelle proof assistant (see Blanchette et al. [112]). We use the facilities already available in Isabelle to extract all theorems, lemmas, definitions and axioms — which we refer to altogether as *facts* — and the essential relationships between them, as follows.

- Every fact ϕ and symbol s is represented by a unique string.
- A fact f can only be used in the proof of fact ϕ if f is *visible* from ϕ. The visibility relation $\prec$ is a partial order. The set ρ_ϕ defined as

$$\rho_\phi = \{f \mid f \prec \phi \wedge \nexists \psi : \ f \prec \psi \prec \phi\}$$

  is the parents of fact ϕ and contains only those facts that are *immediately visible* from it.
- The set of facts visible from ϕ that have been used in its proof is Π_ϕ, also called the *dependencies* of ϕ or simply its proof.

Our data contains ten articles of varying sizes, ranging approximately one hundred to fifteen hundred conjectures. As it is noteworthy, we acknowledge that the dependencies of any theorem in this repository will only contain what represents one possible proof. However, since the AFP is manually written by numerous authors of varying backgrounds, it is reasonable to expect variations in proof styles across the submissions; corresponding to variations in dependency choices.

Objective Function

Premise selection must be biased towards overspecification. That is because, although too many premises lead to longer proof times, even one too few will entirely eliminate the possibility of finding a proof. Therefore, we design an objective function to measure the quality of premise selection with this condition in mind.

Let $\widetilde{\Pi_\phi}$ be the premises suggested by the heuristic for fact ϕ. Define the *score* S_ϕ for the heuristic for this fact as

$$S_\phi = \frac{|\widetilde{\Pi_\phi} \cap \Pi_\phi|}{|\Pi_\phi|} + \frac{|\widetilde{\Pi_\phi} \cap \Pi_\phi|}{2^{|\Pi_\phi|}}$$

Note that when the no premises are suggested, the score is zero because then a proof is impossible. The final total score S across all facts is then

$$S = \sum_\phi S_\phi.$$

Results

We optimized SInE for the ten AFP articles independently to yield parameters that are specialized for each of them, as shown in Table 3.1. For each article we performed three iterations of the Bayesian optimization process with random starting points.

We find notable variations in all three parameters across the articles, as well as in the final effectiveness of SInE. Suppose strategies were partitioned with these AFP articles as examples. This method could then be used to set parameters of their premise-selection algorithm.

Table 3.1: Premise selection results on the AFP articles. We check the predicate $\widetilde{\Pi_\phi} \subseteq \Pi_\phi$ for every fact in the article and report the proportion satisfied.

AFP article	Goals	$\widetilde{\Pi_\phi} \subseteq \Pi_\phi$	Parameter		
			t	g	k
Polynomials	135	87%	16.3	58	131
AbstractHoareLogics	793	63%	17.6	57	130
Completeness	475	89%	18.9	63	134
FinFun	263	95%	19.6	57	132
HeardOf	716	93%	19.5	57	131
InductiveConfidentiality	1425	82%	19.6	58	130
RefineMonadic	1509	95%	14.7	64	123
MiniML	345	84%	19.1	58	131
RecursionTheory	656	85%	19.0	57	130
SortEncodings	776	80%	14.0	64	123

3.3 Optimizing Embedded Solvers

F* is a programming language aimed at formal program verification (Nikhil et al. [103]). Dafny is another such language, with built-in specification constructs and a corresponding static analyser for formal verification (Leino et al. [111]). CBMC is a static analysis tool for the C and C++ programming languages that uses bounded model checking to formally verify programs (Clarke, Kroening and Lerda [44]). All three systems share an architectural similarity: formal verification is achieved by transforming the properties in question into logical formulae, which are then solved using external tools. This architecture is advantageous, because it allows the authors of a variety of tools to exploit the advancing capabilities of high performance solvers to perform complex logical reasoning, while allowing them to focus on tasks specific to their applications.

Any solver embedded in an application will in effect be used to check the specific class of formulae generated by that application. Could the performance of such a solver be improved by tuning its parameters according to the target application? We show this to be true in the case of a test suite that uses CBMC to formally verify properties of a library written in C. In this work, we use Bayesian optimization to find an improved parameter configurations for the solver in CBMC, that is tuned especially for that test suite. In effect, we generate a strategy for the solver specific to each test.

In the following sections, we describe the relevant aspects of CBMC in detail, and our experiments.

Listing 3.1: An example CBMC test harness from the aws-c-common library.

```
void aws_string_eq_ignore_case_harness() {
    struct aws_string *str_a = nondet_bool() ? ensure_string_is_allocated_bounded_length(
        ↪  MAX_STRING_LEN) : NULL;
    struct aws_string *str_b =
        nondet_bool() ? (nondet_bool() ? str_a : NULL) :
            ↪  ensure_string_is_allocated_bounded_length(MAX_STRING_LEN);
    if (aws_string_eq_ignore_case(str_a, str_b) && str_a && str_b) {
        assert(aws_string_is_valid(str_a));
        assert(aws_string_is_valid(str_b));
        assert(str_a->len == str_b->len);
    }
}
```

CBMC

CBMC is a bounded model checker for C and C++. CBMC performs code analysis using *symbolic simulation*, that is, it compiles code into logical formulae which are then sent to a solver for verification. With this method, assertions in the code can be verified statically. CBMC can also automatically generate assertions to check for common errors, such as buffer overflows, unsafe pointer access, memory leaks and arithmetic faults.

Test harness. Any project utilizing CBMC will typically define a suite of *test harnesses*: carefully written C functions that act as entry-points for CBMC, along with specifications for loop unrolling, function pointer destinations, variables that should be treated as non-deterministic and other optimizations. Each test harness should aim to formally verify some aspect of the code. Test harnesses may be viewed as static analysis unit tests.

aws-c-common. *aws-c-common* is an open source library of commonly used datastructures and algorithms written in the C programming language, developed for use in the Amazon Web Services Software Development Kit and utilized by multiple projects. Given the central role of this library, the developers have committed considerable resources towards its formal verification using CBMC. At the time of writing, the library contains 166 CBMC test harnesses.

An example test harness from aws-c-common is shown in Listing 3.1. This harness was designed to verify certain properties of `aws_string_eq_ignore_case()` — a function exported by the library. Of note, is the use of `nondet_bool()` to specify a non-deterministic value to CBMC. Non-deterministic values are also available for other data types, such as `nondet_ushortint()` for unsigned short integers. Such values represent input read from the environment and thus CBMC analyses the test harness for any possible choice of those values.

Goto files. CBMC uses a modified C compiler to compile source code down to a simplified format that is more amenable to further conversion into logical formulae. One

Listing 3.2: A snippet from the goto file generated for the test harness shown in Listing 3.1

```
aws_string_eq_ignore_case_harness
struct aws_string *str_a;
_Bool return_value_nondet_bool;
return_value_nondet_bool = NONDET(__CPROVER_bool);
struct aws_string *tmp_if_expr;
IF !(return_value_nondet_bool != FALSE) THEN GOTO 1
struct aws_string *return_value_ensure_string_is_allocated_bounded_length;
ensure_string_is_allocated_bounded_length((size_t)16);
return_value_ensure_string_is_allocated_bounded_length =
    ↪  ensure_string_is_allocated_bounded_length#return_value;
dead ensure_string_is_allocated_bounded_length#return_value;
tmp_if_expr = return_value_ensure_string_is_allocated_bounded_length;
GOTO 2
1:  tmp_if_expr = (struct aws_string *)(void *)0;
2:  str_a = tmp_if_expr;
...
```

such simplification, for example, is the unrolling of every loop to a specified number of iterations. The output files are called *goto* files in CBMC parlance. Formal verification of each test harness can be viewed as a two-stage process. First the test harness is compiled down to a goto file, and in the next stage the goto file is formally verified by conversion into logical formulae and checked by a solver. This project focuses on optimization of the solver, and therefore we only need to generate the goto files for every test harness once. Only the second stage needs to be re-executed on any changes to the solver.

The goto files are binaries but a human readable text representation can be generated using CBMC. The text version of the goto file generated from the test harness shown in Listing 3.1 spans 623[1] lines. The large size is in part due to inclusion of other code from the library relevant to the harness in the goto file. A snippet of that text is shown in Listing 3.2, which shows the first few lines of the code generated for the test harness function.

aws-batch-cbmc. Formal verification using CBMC is a computationally intensive task, but each test harness can be verified independently and in parallel. The developers of aws-c-common run their suite of CBMC tests for their continuous integration process and therefore require a fast turnaround from them. Their solution is to run each test harness as an independent batch process on a public cloud service, using the system from another open source project called *aws-batch-cbmc*.

MiniSat. CBMC can use multiple external SMT and SAT solvers to check internally generated formulae, but its default solver is MiniSat — an open source SAT solver by Een et al. [40]. When using a SAT solver, CBMC performs bit-vector flattening of its formulae before verification by the solver. CBMC is compiled with MiniSat using its default parameters. For the purposes of this project, we have modified CBMC to accept

[1] 337 lines if the generated comments are excluded

Table 3.2: MiniSat SimpSolver parameters

Parameter	Type	Default	Description
`asymm`	Bool	False	Shrink clauses by asymmetric branching
`rcheck`	Bool	False	Check if a clause is already implied
`elim`	Bool	True	Perform variable elimination
`grow`	Int $(-\infty, \infty)$	0	Allow a variable elimination step to grow by a number of clauses
`cl-lim`	Int $[0, \infty)$	20	Variables are not eliminated if it produces a resolvent with a length above this limit
`sub-lim`	Int $[0, \infty)$	1000	Do not check if subsumption against a clause is larger than this
`simp-gc-frac`	Double $(0, \infty)$	0.5	The fraction of wasted memory allowed before a garbage collection is triggered during simplification

solver parameters as runtime options. Although the experiments presented here are for a SAT solver, our methods are similarly applicable for parameter optimization of SMT solvers.

Methodology

In this work, we derive a parameter configuration for MiniSat that is optimized for a subset of the test harnesses in aws-c-common.

Goto files. Using aws-batch-cbmc, we completed one batch of the CBMC tests for aws-c-common. This generated goto files for each of the test harnesses, which we used subsequently for all our tests. The system also reported time spent by the tests in the solver. The 166 tests spend a total of 4252 s in the solver, with a median of 1.2 s; of these, 159 tests spend less than 100 s, and another 4 spend less than 200 s. The remaining three tests are notably slower, taking up 488 s, 687 s and 1005 s respectively in the solver.

Tests and parameters. Given the resources available to us, we limited our sample of test harnesses to the 125 tests that spend the least amount of time in MiniSat. They represent254 s of total time in the solver before optimization. CBMC uses the SimpSolver in MiniSat, the parameters of which are shown in Table 3.2. We decided to limit our optimization task to the SimpSolver specific parameters, leaving the remaining MiniSat parameters at their defaults. Early experiments revealed that any search for more optimal parameters would maintain the three Boolean parameters `asymm`, `rcheck` and `elim` at their default settings of `false`, `false` and `true` respectively. Therefore,

we decided to fix these Boolean parameters to their default values and attempted to optimize only the remaining four parameters.

Objective function. Our goal was to find a single parameter configuration for the whole set of test harnesses that could reduce the total time spent in evaluating the entire set. In more detail, suppose there are N test harnesses, and functions t_i map from the parameters to the evaluation time of the test harness indexed by i, such that

$$t_i : (g, c, s, r) \mapsto \tau$$

where g, c, s and r are the MiniSat parameters `grow`, `cl-lim`, `sub-lim` and `simp-gc-frac` respectively, and τ is the time spent in MiniSat when that test harness is evaluated. For any fixed parameter configuration tuple (g', c', s', r'), the objective function can simply be $\sum t_i(g', c', s', r')$. However, it is necessary to adjust the objective function with a timeout, since some parameter configurations will lead to a prohibitively long evaluation time. Therefore, when the objective begins to exceed the pre-defined timeout, we take note and terminate it.

Implementation. Our implementation of the Bayesian optimization process for this task is in MATLAB [157]. MATLAB 2019 includes a comprehensive Bayesian optimization framework that was suitable for this task. Since our objective is a measure of elapsed real time on a local machine, small variations can be expected in the objective for any fixed set of parameters. The framework in MATLAB can be set to accept such a non-deterministic objective. The framework also accepts integer parameters, which was another key requirement of our project, since nearly all parameters are discrete.

In the MATLAB implementation of Bayesian optimization, the objective function may return with `NaN` (not a number) to signify an error. MATLAB builds an error model to accommodate errors in the objective, and this model can accommodate non-deterministic errors. We chose to model timeouts in our objective function as errors. Modelling timeouts as errors allows the Bayesian optimization framework to avoid parameter spaces that frequently cause timeouts. In this scenario, the choice of the timeout value has trade-offs that must be considered. A short timeout will be resource efficient, reducing the time spent in parameter spaces that perform worse than the original ones. However, poor performing parameters are also informative of the shape of the objective function, but when they are modelled as errors due to the timeout, the GP model has less data to learn from. Based on the practical resources available to us, we specified a large timeout of 1200 s, more than four times the time taken by MiniSat in the original configuration.

Acquisition function & GP kernel. We used the *expected improvement plus* acquisition function from the framework in MATLAB. This is similar to the expected improvement function described above, with a modification to avoid excessively exploiting an area.

Table 3.3: Parameter bounds for optimization.

Parameter	Bounds
`grow`	$[-10000, 10000]$
`cl-lim`	$[0, 1000] \cup \{\infty\}$
`sub-lim`	$[0, 100000] \cup \{\infty\}$
`simp-gc-frac`	$(0, 50.0]$

Table 3.4: Improved parameters discovered using Bayesian optimization

Parameter	Value
`grow`	1615
`cl-lim`	2
`sub-lim`	61603
`simp-gc-frac`	28.32

We used the default exploration ratio of 0.5. To obtain the maximum control over the Bayesian optimization process in MATLAB, we used the `bayesopt` function. However, the kernel covariance function used by this is fixed to the Matérn $5/2$ kernel, which is detailed in Appendix 3.A. The kernel hyperparameters are optimized internally by MATLAB, and options to influence them are unavailable.

Results

It is necessary to specify bounds on the parameter space for the optimization process. Our specified bounds are shown in Table 3.3. These bounds were chosen to be sufficiently large, again based on preliminary experiments. In the case of `cl-lim` and `sub-lim`, it is possible to set both parameters to -1 in MiniSat, which then removes the clause limit and subsumption limit entirely, and we included that option in our parameter search space.

After running the optimization for a total of 210 iterations, which completed in thirty-four hours, the best discovered parameters are shown in Table 3.4. In comparison to the original parameters, these parameters reduce the time spent in the solver from 254 s to 184 s. Figure 3.1 plots the minimum objective that is observed until each iteration during Bayesian optimization. The times presented here are a measure of the times spent in MiniSat. Each invocation of CBMC to evaluate a goto file will require relatively expensive additional steps such as the preparation of formulae, which are unaffected by changes in MiniSat parameters.

We have optimized a collection of test harnesses all together to find a single parameter

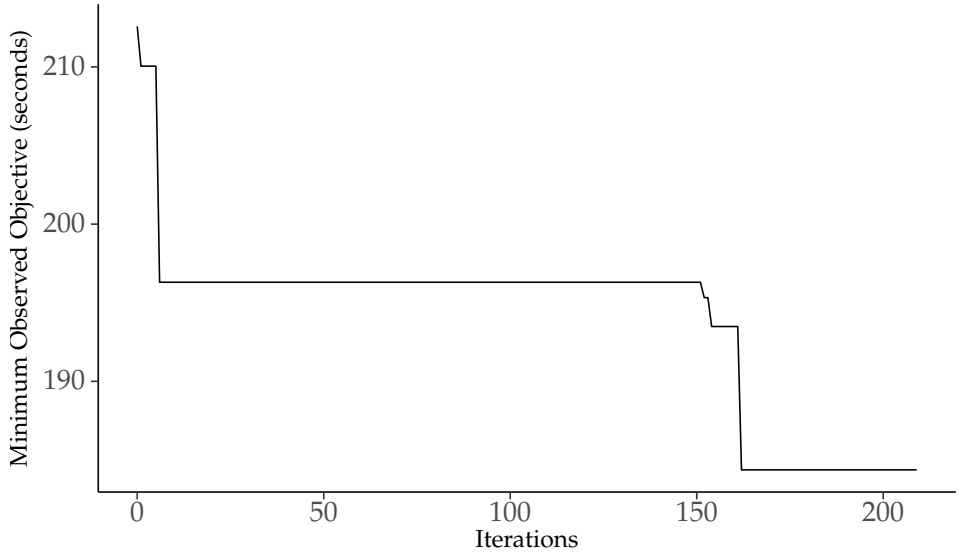


Figure 3.1: The minimum observed objective as Bayesian optimization progresses through each iteration.

Table 3.5: Parameters optimized for two test harnesses individually.

Test harness	grow	cl-lim	sub-lim	simp-gc-frac	Solver time		
					Original	Optimized	Gain
`aws_byte_buf_cat`	0	−1	1000	0.5	56 s	51 s	9%
`aws_hash_table_put`	−8208	230	36	29.68	78 s	59 s	24%

configuration that, applied to all the tests, reduces the total time spent in the solver. An alternative is to optimize each test harness individually, and save the discovered parameters along with the test harness. When the suite of test harnesses is evaluated, each test harness can then be evaluated with individualized solver parameters. The trade-off between this approach versus our earlier one depends on the resources available and the nature of the test harnesses. For a basic comparison, we optimized two randomly selected tests harnesses individually for the same number of iterations. The results, shown in Table 3.5, show that the optimized parameters discovered for the two are very different, and further improvement is achieved.

3.4 Related Work

Optuna is a modern, well engineered software solution for hyperparameter optimization (see Akiba et al. [139]). Although it is typically used for optimizing parameters for machine learning models, it is also well suited for prover optimization. It has good built-in support for integer and categorical parameters. In addition, it is capable of searching the parameter space in parallel, distributed across multiple machines. It features a technique they call *pruning*, wherein it identifies poor parameter candidates early — ahead of the complete execution of the objective — and terminates it to free resources for better candidates. It is a worthy option to consider for similar future work.

In Section 3.2 we used the AFP as our test-bed, which contains proofs formalized in Isabelle. Isabelle contains the Sledgehammer tool, introduced in Chapter 1, which is able to automatically complete proofs on behalf of the user. The latest incarnation of Sledgehammer in Isabelle uses a premise selection algorithm called MeSh, which is a combination of:
MePo, the original heuristic-based relevance filter in Isabelle; and
MaSh — *Ma*chine Learning for *S*ledge*h*ammer — a new ML relevance filter.
MaSh (see Blanchette et al. [119] and Kühlwein et al. [102]) uses naive Bayes and k nearest neighbours models combined with inverse document frequency (IDF) to increase precision. It outperforms the old fact selector on large formalizations and, in combination with MePo, enables effective premise selection in Sledgehammer [119]. HOL(y)Hammer is a similar tool for the HOL Light proof assistant (Kaliszyk and Urban [113]), implemented as an online service.

In work that contrasts from the methods presented here, Schäfer and Schulz [117] have presented a way to automate the development of new heuristics, by using genetic algorithms to breed new heuristics from existing good strategies. Their results show that automatically generated heuristics can match, and sometimes outperform, manually developed heuristics.

3.5 Conclusion

We have shown, in this chapter, some examples of parameter optimization of theorem provers using Bayesian optimization. Parameter optimization is an essential part of strategy invention. The continued development of parameter optimization technologies in the machine learning community are of great benefit to theorem provers.

3.A Kernel covariance functions in GP regression

A *kernel function* is a real-valued function of two vector-valued arguments,

$$k(x, x') \in \mathbb{R},$$

for $x, x' \in \chi$ where χ is some abstract space. Kernels may be viewed as a measure of similarity between the two arguments when the function is symmetric and non-negative. For example, the following kernel measures cosine similarity:

$$k(x, x') = \frac{x^T x'}{||x||_2 ||x'||_2}.$$

Kernel functions used as covariance functions in Gaussian processes must be positive definite. Such a kernel is called a *Mercer kernel*. Mercer's theorem (see Minh, Niyogi and Yao [59]) shows that for such kernels there exists a function

$$\phi : \chi \mapsto \mathbb{R}^D$$

such that

$$k(x, x') = \phi(x)^\intercal \phi(x').$$

This kernel is effectively computing an inner product of the two arguments in a D dimensional space, where D is typically larger than the dimensionality of χ and potentially infinite. For example, consider the following kernel when $x, x' \in \mathbb{R}^2$:

$$\begin{aligned} k(x, x') &= (1 + x^\intercal x')^2 \\ &= (1 + x_1 x_1' + x_2 x_2')^2 \\ &= 1 + 2x_1 x_1' + 2x_2 x_2' + (x_1 x_1')^2 + (x_2 x_2')^2 + 2x_1 x_1' x_2 x_2' \\ &= \phi(x)^\intercal \phi(x'), \end{aligned}$$

where

$$\phi(x) = \left[1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1 x_2\right]^\intercal.$$

Therefore, this kernel represents a measure of similarity in a six-dimensional space for two-dimensional vectors. The cosine similarity kernel shown above is also a Mercer kernel. As Snoek, Larochelle and Adams [99] explain, the squared exponential kernel is often a default choice for the covariance function in Gaussian processes regression:

$$k_{SE}(x, x') = \theta_0 e^{\{-\frac{1}{2} r^2(x, x')\}}, \quad r^2(x, x') = \sum_{d=1}^{D} \frac{(x_d - x_d')^2}{\theta_d^2}.$$

However, this may lead to sample functions in the GP that are unrealistically smooth for practical optimization problems. Instead, Snoek, Larochelle and Adams [99] recommend the use of the Matérn $5/2$ kernel:

$$k_{M52}(x, x') = \theta_0 \left(1 + \sqrt{5r^2(x, x')} + \frac{5}{3} r^2(x, x')\right) e^{-\sqrt{5r^2(x, x')}}.$$

This is the default kernel used by MATLAB for Bayesian optimization.

3.B The Risks of Overfitting

A reviewer of this work was concerned about the risks of overfitting discovered parameters to the test set. For the task of optimizing MiniSat for CBMC, in the work presented above, overfitting is essentially a goal. This is because we are optimizing the setup for regression tests. Those regression tests remain largely unchanged across multiple runs. Since the proof goals are not changing, overfitting is not a concern in this instance. If one was to optimize MiniSat for general proof goals, overfitting may be avoided by means of cross-validation and a full separation between the test-set and the evaluation-set.

3.C Acknowledgments

The work in Section 3.2 was co-supervised by Prof. Mateja Jamnik, Computer Laboratory, University of Cambridge, UK.

The work in Section 3.3 is derived from initial investigations during an internship at the Automated Reasoning Group of Amazon Web Services, supervised by John Harrison.

4

Run-time Strategy Scheduling

> Essentially, all models are wrong, but some are useful.
>
> *George Box* [15]

Automated Theorem Provers (ATPs) generate proofs by utilizing inference procedures in combination with heuristic search. A specific configuration of a prover, which may be specialized for a certain class of problems, is termed a *strategy*. Provers such as E [38] can select from a portfolio of strategies to solve the goal theorem. Furthermore, certain provers hedge their allocated proof time across a number of proof strategies by use of a *strategy schedule*, which specifies a time allocation for each strategy and the sequence in which they are used until one proves the goal theorem. This method was pioneered in the Gandalf prover [29].

Prior to a proof attempt, the success of a strategy for the specified goal theorem is indeterminate. In modern provers, that prediction is often intractable or undecidable [109]; and yet, it is needed for strategy selection and scheduling. The practical solution is the use of approximations for this task, traditionally with heuristics, more recently with Machine Learning (ML) and even with a bit of both. ML methods for strategy invention, selection and scheduling are actively researched. ML methods for strategy selection conditioned on the proof goal have shown promising results [107]. Good results have also been reported for strategy synthesis using ML [130]. Work on ML for algorithm portfolios — which allocate resources to multiple solvers simultaneously — has goals similar to strategy scheduling. An example of such work is by Silverthorn and Miikkulainen [82], who propose latent class models for this purpose.

In the work presented here, we develop a method for *run-time strategy scheduling*

specifically for automated theorem proving. That means, specifically: when presented with a goal theorem, our method is able to generate a strategy schedule specialized for it, with which a base automatic theorem prover then attempts a proof. In a strategy schedule, we consider the time allocation of an individual strategy to be a component of that strategy. Hence, any schedule is simply a sequence of strategies. The schedule generated by our method is a permutation of the original fixed strategy schedule of the base prover. We do, however, present a minor extension of our core method in which we attempt to optimize, additionally, the time allocations to the strategies. The goal of our experiments is to examine the merits of these methods; we aim to measure the performance of these methods on their own, independent of the base prover.

Remark 4.1 (The choice of iLeanCoP). Our method will necessarily interact with the theorem prover it is tested with to generate results that vary accordingly, but we must select one prover for the purposes of our experiments. In drawing conclusions, the use of a simple prover enhances the clarity of any interpretation. This is why we chose iLeanCoP, whose core is implemented in around thirty lines of Prolog. The use of similar simple provers is common in related work such as [150]. For contrast, consider the E-prover (version 2.6), which is written in 405K lines of C code and already employs a complex system of strategies, whose interaction with our method would be difficult to assess. Our choice of an intuitionistic prover reflects our interest in that subfield. Despite widespread interest in intuitionistic logic in computer science, it has received little attention concerning automation. Among intuitionistic provers, iLeanCoP is seen as impressive and is able to prove a sufficient number of theorems in the classical TPTP library for significance testing. We test iLeanCoP on the TPTP library and make use of the TPTP features elaborated in Chapter 2. We have already discussed the strategies in iLeanCoP in some detail in Chapter 2; it is a good prover for this work and enables the rigorous evaluation we present later in this chapter. ◁

Hypothesis and Contributions. We will explore the following *research question* in this chapter:

> How can ML methods enhance strategy scheduling in ATPs?

We present, in this chapter, the following *contributions* that provide some answers to the above research question.

(a) We present a method for run-time strategy scheduling that predicts schedules specialized for the goal theorem, which it is able to achieve by learning from previous proof attempts during an earlier training phase. The predictions are computationally efficient, to the extent that the prediction time is practically negligible.

(b) We develop this method from the ground up with rigorous theoretical detail. To that end, we employ Bayesian statistics, which: enables the use of priors to incorporate domain knowledge into the model; facilitates the inclusion of

regularization and stability, and provides for a computationally tractable model with the trade-offs we need.

(c) We prototype that method using: iLeanCoP with its pre-defined strategy schedule as the base prover; the TPTP problem library for testing and training; and the features designed in Chapter 2. To demonstrate its performance, we train and rigorously evaluate this prototype. Our results show that the schedules generated by our method are able to significantly reduce the time iLeanCoP takes to produce proofs, by more than 50% in aggregate, when compared to use of the pre-existing schedule that iLeanCoP ships with. Furthermore, a small extension to our method, wherein we also change the time allocations to each strategy, is able to increase the number of theorems proved.

With that in place, we test for the *alternative hypothesis* that our prototype implementation has a significant main effect on the time taken to prove the theorems in the TPTP repository. We show that there is indeed a significant reduction in the median time saved across the TPTP theorems, and a mean time saving of more than 50%.

Remark 4.2 (Prediction costs must be minimal). First-Order Logic (FOL) provers such as iLeanCoP are necessarily time bounded when attempting a proof. In public competitions, that bound is typically 300 s. Any time taken to predict a strategy schedule must be accounted for within this time allocation. Time taken for this prediction is time taken away from the prover itself which could have been invested in the proof search. Therefore, it is essential to minimize schedule prediction time. It is particularly wise to favour a saving in prediction time at the cost of model optimization and training time. Prediction time efficiency, therefore, takes precedence over training time, which is a trade-off we will make throughout this work. ◁

4.1 Background

4.1.1 Zero-phase Component Analysis

In this chapter, we will be making use of the features developed in Chapter 2. Unfortunately, our features range over vastly different scales. Consider the examples below for perspective.

(a) Take each problem in the TPTP and consider the arity of every function in it. Consider the maximum arity in each problem as a feature. The average of this feature across the TPTP problems is 2.

(b) The number of predicate symbols, similarly calculated as a feature, averages 2097.

Furthermore, as is evident from Figure 2.7, various correlations exist between the features. It is desirable to remove feature correlations to alleviate some burden on the subsequent modelling phase, and to standardize the features to zero mean and unit

variance to create a feature space with similar length-scales in all dimensions. The former is achieved by *decorrelation*, the latter by *standardization*, and both together by a *sphering transformation*. Decorrelation helps models identify the unique contributions of individual features. Additionally, standardization scales numbers down to smaller values, thereby reducing the chances of computational overflow errors.

Definition 4.1 (Zero-phase Component Analysis (ZCA))**.** ZCA is a sphering transform. Let $\boldsymbol{F}$ be the feature matrix, created by, for example, taking rows of features of each TPTP theorem and stacking them up to form a matrix. Say $\boldsymbol{F}$ has also been centred, i.e., the mean of each column is zero. The Principal Component Analysis (PCA, see Example 1.5) decomposition of $\boldsymbol{F}$ gives the correlation matrix $\boldsymbol{C}$ such that

$$\boldsymbol{C} = \boldsymbol{E}\boldsymbol{D}\boldsymbol{E}^{\intercal},$$

where $\boldsymbol{D}$ is the diagonal matrix of the eigenvalues of $\boldsymbol{F}$, and $\boldsymbol{E}$ its eigenvectors. If we multiply $\boldsymbol{F}$ with the transformation

$$\boldsymbol{W}_{\text{PCA}} = \boldsymbol{D}^{-1/2}\boldsymbol{E}^{\intercal},$$

we obtain a matrix that will have a correlation matrix that is diagonal, i.e., there will be no cross-correlations (see Duboue [143]). Such a decorrelation property is maintained across rotations of $\boldsymbol{W}_{\text{PCA}}$. The ZCA transformation matrix is

$$\boldsymbol{W}_{\text{ZCA}} = \boldsymbol{E}\boldsymbol{D}^{-1/2}\boldsymbol{E}^{\intercal} = \boldsymbol{C}^{-1/2}.$$

We obtain the ZCA transformed version of $\boldsymbol{F}$ by multiplying it with $\boldsymbol{W}_{\text{ZCA}}$. ◁

ZCA, among the various other sphering transformations, especially ensures the transformed data is as close as possible to the original, in terms of the Euclidean distance.

4.1.2 Gaussian Approximation

The mode of a posterior probability distribution is a point estimate. For full Bayesian inference, we need an estimate of the uncertainty in the posterior — but the full posterior distribution is sometimes intractable. A compromised estimate is a view of the uncertainty at the mode, obtained by estimating the shape of the posterior around the mode as a Gaussian distribution.

For some posterior distribution $\Pr(\boldsymbol{\theta} \mid D)$, let

$$\Pr(\boldsymbol{\theta} \mid D) = \frac{1}{Z}e^{-E(\boldsymbol{\theta})},$$

where

$$E(\boldsymbol{\theta}) = -\log \Pr(\boldsymbol{\theta}, D)$$

and $Z = \Pr(D)$, that being the normalization constant. A Taylor series expansion of E around the mode $\boldsymbol{\theta}^*$ gives

$$E(\boldsymbol{\theta}) \approx E(\boldsymbol{\theta}^*) + (\boldsymbol{\theta} - \boldsymbol{\theta}^*)^\intercal \boldsymbol{g} + \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}^*)^\intercal \boldsymbol{H}(\boldsymbol{\theta} - \boldsymbol{\theta}^*),$$

where $\boldsymbol{g}$ is the gradient and $\boldsymbol{H}$ is the Hessian, both evaluated at the mode:

$$\boldsymbol{H} = \frac{\partial^2}{\partial \boldsymbol{\theta} \partial \boldsymbol{\theta}^\intercal} E(\boldsymbol{\theta})\bigg|_{\boldsymbol{\theta}^*} \quad \text{and} \quad \boldsymbol{g} = \frac{\partial}{\partial \boldsymbol{\theta}} E(\boldsymbol{\theta})\bigg|_{\boldsymbol{\theta}^*} = \mathbf{0}\ .$$

The gradient $\boldsymbol{g}$ at the mode is zero. Thus, we have

$$\Pr(\boldsymbol{\theta} \mid D) \approx \underbrace{\frac{1}{Z} e^{-E(\boldsymbol{\theta}^*)}}_{\text{constant}} \underbrace{\exp\left[-\frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}^*)^\intercal \boldsymbol{H}(\boldsymbol{\theta} - \boldsymbol{\theta}^*)\right]}_{\text{matches Gaussian PDF exponent}} \propto \mathcal{N}(\boldsymbol{\theta} \mid \boldsymbol{\theta}^*, \boldsymbol{H}^{-1}).$$

Simply put, we can approximate the local shape of the posterior at the mode by the Gaussian $\mathcal{N}(\boldsymbol{\theta}^*, \boldsymbol{H}^{-1})$. The normalization constant Z, which is often intractable, is conveniently ignored since it is not involved in the formulation of $\boldsymbol{H}$. This method is known as the *Laplace approximation* (see Murphy [95]).

4.1.3 Markov chain Monte Carlo Sampling

Consider an integral of the form

$$I = \int h(x) f(x)\, \mathrm{d}x,$$

where $f(x)$ is a Probability Density Function (PDF). It may be difficult to evaluate I in closed form, especially if $h(x)$ is a complicated function. We can approximate I as follows. Draw samples $X_1, \ldots, X_N \sim f$ and take

$$\hat{I} \equiv \frac{1}{N} \sum_{i=1}^{N} h(X_i).$$

This simple, general and scalable method is called *Monte Carlo integration* [49]. It is useful in Bayesian statistics, where one often finds integrals such as I with h being the likelihood function and f the prior probability distribution. A complication, however, is that even f may not be a known distribution that can be easily sampled from. A solution explained below is known as Markov Chain Monte Carlo (MCMC) sampling.

In MCMC, one constructs a Markov chain whose stationary distribution matches the *target distribution* f (see Wasserman [49]). The Metropolis-Hastings algorithm for MCMC is given in Algorithm 4.1. It makes use of a *proposal distribution* g, which is a distribution that one can easily sample from. Note that while calculating r in Algorithm 4.1, the normalization constant of the target density f cancels out. This is another convenience offered by the algorithm: we need not compute the often-intractable normalization constant.

Algorithm 4.1: *Metropolis-Hastings Algorithm*

Suppose we have generated samples $\{\boldsymbol{\omega}^{(1)}, \ldots, \boldsymbol{\omega}^{(i)}\}$ from the target distribution f. Generate $\boldsymbol{\omega}^{(i+1)}$ as follows.

1: Generate candidate value $\dot{\boldsymbol{\omega}} \sim g(\boldsymbol{\omega}^{(i)})$, where g is the proposal distribution.

2: Evaluate $r \equiv r(\boldsymbol{\omega}^{(i)}, \dot{\boldsymbol{\omega}})$ where

$$r(x, y) = \min\left\{\frac{f(y)}{f(x)}\frac{g(x|y)}{g(y|x)}, 1\right\}.$$

3: Set

$$\boldsymbol{\omega}^{(i+1)} = \begin{cases} \dot{\boldsymbol{\omega}} & \text{with probability } r \\ \boldsymbol{\omega}^{(i)} & \text{with probability } 1 - r. \end{cases}$$

4.2 Methodology

4.2.1 Distribution of Permutations

We model a strategy schedule using a vector of strategies, and thus all schedules are *permutations* of the same. The foundation of our ML algorithm, therefore, is a probability distribution over permutations.

Definition 4.2 (Permutation)**.** Let $M \in \mathbb{N}$. A permutation $\boldsymbol{\pi} \in \mathbb{N}^M$ is a vector of indices, with $\pi_i \in \{1, \ldots, M\}$ and $\forall i \neq j : \pi_i \neq \pi_j$, representing a reordering of the components of an M-dimensional vector $\boldsymbol{s}$ to $[s_{\pi_1}, s_{\pi_2}, \ldots, s_{\pi_M}]^\intercal$. ◁

In this text, vector-valued variables, such as $\boldsymbol{\pi}$ above, are in boldface, which must change when they are indexed, like π_1 for example. For probabilistic modelling of schedules represented using permutations, we use the Plakett-Luce model [6, 9] to define a parametric probability distribution over permutations.

Definition 4.3 (Plakett-Luce distribution)**.** The Plakett-Luce distribution $\mathrm{Perm}(\boldsymbol{\lambda})$ with parameter $\boldsymbol{\lambda} \in \mathbb{R}^M_{>0}$, has support over permutations of indices $\{1, \ldots, M\}$. For permutation $\boldsymbol{\Pi}$ distributed as $\mathrm{Perm}(\boldsymbol{\lambda})$,

$$\Pr(\boldsymbol{\Pi} = \boldsymbol{\pi}; \boldsymbol{\lambda}) = \prod_{j=1}^{M} \frac{\lambda_{\pi_j}}{\sum_{u=j}^{M} \lambda_{\pi_u}}.$$

◁

In latter sections, we use the parameter $\boldsymbol{\lambda}$ to assign an abstract 'score' to strategies when modelling distributions over schedules. In effect, we use $\boldsymbol{\lambda}$ as a latent variable in the model we will eventually build later in this chapter. This latent score is particularly useful due to the following theorem.

Theorem 3. *Let $\boldsymbol{\pi}^*$ be a mode of the distribution* $\mathrm{Perm}(\boldsymbol{\lambda})$*, that is*

$$\boldsymbol{\pi}^* = \operatorname*{argmax}_{\boldsymbol{\pi}} \Pr(\boldsymbol{\pi}; \boldsymbol{\lambda}).$$

Then, $\lambda_{\pi_1^*} \geqslant \lambda_{\pi_2^*} \geqslant \lambda_{\pi_3^*} \geqslant \ldots \geqslant \lambda_{\pi_M^*}$.

Thus, assuming $\boldsymbol{\lambda}$ is a vector of the score of each strategy, the highest probability permutation indexes the strategies in decreasing order of scores. Conversely, the highest probability permutation can be obtained efficiently by sorting the indices of $\boldsymbol{\lambda}$ with respect to their corresponding values in decreasing order. Cao et al. [62] have presented a proof of Theorem 3, and Cheng, Dembczynski and Hüllermeier [78] have discussed some further interesting details.

Example 4.1. Let $\boldsymbol{\lambda} = [1; 9]$, $\boldsymbol{\pi}^\dagger = [1; 2]$ and $\boldsymbol{\pi}^\ddagger = [2; 1]$. Then,

$$\Pr(\boldsymbol{\Pi} = \boldsymbol{\pi}^\dagger; \boldsymbol{\lambda}) = \frac{\lambda_{\boldsymbol{\pi}_1^\dagger}}{\lambda_{\boldsymbol{\pi}_1^\dagger} + \lambda_{\boldsymbol{\pi}_2^\dagger}} \cdot \frac{\lambda_{\boldsymbol{\pi}_2^\dagger}}{\lambda_{\boldsymbol{\pi}_2^\dagger}} = \frac{1}{1+9} \cdot \frac{9}{9} = \frac{1}{10}.$$

Similarly, $\Pr(\boldsymbol{\Pi} = \boldsymbol{\pi}^\ddagger; \boldsymbol{\lambda}) = 9/10$. ◁

Theorem 4. $\mathrm{Perm}(c\boldsymbol{\lambda}) = \mathrm{Perm}(\boldsymbol{\lambda})$*, for any scalar constant* $c > 0$.

In other words, the Plakett-Luce distribution is invariant to the scale of the parameter vector.

Lemma 5. $\mathrm{Perm}(\exp(\boldsymbol{\lambda} + c)) = \mathrm{Perm}(\exp(\boldsymbol{\lambda}))$*, for any scalar constant* $c \in \mathbb{R}$.

Lemma 5 follows from Theorem 4, and shows the same distribution is translation invariant if the parameter is exponentiated. Cao et al. [62] give proofs of both.

4.2.2 A Maximum Likelihood Model

We model a strategy schedule as a ranking of known strategies, where each strategy is constructed by a parameter setting and time allocation. A ranking therein is a permutation of strategies, with each strategy retaining its time allocation irrespective of the ordering. We construct, in this section, a model for inference of such permutations that is linear in the parameters.

Suppose we have a repository of N theorems which we test against each of our M known strategies to build a data-set $\mathcal{D} = \{(\boldsymbol{\pi}^{(i)}, \boldsymbol{x}^{(i)})\}_{i=1}^{N}$, where $\boldsymbol{\pi}^{(i)}$ is a desirable ordering of strategies for theorem i and $\boldsymbol{x}^{(i)}$ is a feature vector representation of the theorem. In Section 4.2.8, we detail how we instantiate $\mathcal{D}$ for our experiments, which serves as an example for any other implementation. We assume that $\boldsymbol{\pi}^{(i)}$ has Plakett-Luce distribution conditioned on $\boldsymbol{x}^{(i)}$ such that

$$\Pr(\boldsymbol{\pi}; \boldsymbol{x}, \boldsymbol{\omega}) = \mathrm{Perm}(\boldsymbol{\Lambda}(\boldsymbol{x}, \boldsymbol{\omega})), \tag{4.1}$$

where $\boldsymbol{\omega}$ is a parameter the model must learn and $\boldsymbol{\Lambda}(\cdot)$ is a vector-valued function of range $\mathbb{R}^M_{>0}$. We use the notation $\Lambda(\cdot)_i$ to index into the value of $\boldsymbol{\Lambda}(\cdot)$. We represent our prover strategies with feature vectors $\{\boldsymbol{d}^{(j)}\}_{j=1}^M$. To calculate the score of strategy j using $\Lambda(\cdot)_j$, we specify

$$\Lambda(\boldsymbol{x}^{(i)}, \boldsymbol{\omega})_j = \exp\left(\boldsymbol{\phi}(\boldsymbol{x}^{(i)}, \boldsymbol{d}^{(j)})^\intercal \boldsymbol{\omega}\right) \tag{4.2}$$

to ensure that the scores are positive valued, where $\boldsymbol{\phi}$ is a suitable basis expansion function. Assuming the data is i.i.d, the likelihood of the parameter vector is given by

$$\mathcal{L}(\boldsymbol{\omega}) = p(\mathcal{D}; \boldsymbol{\omega}) = \prod_{i=1}^{N} \Pr(\boldsymbol{\pi}^{(i)}; \boldsymbol{\Lambda}(\boldsymbol{x}^{(i)}, \boldsymbol{\omega})). \tag{4.3}$$

An $\hat{\boldsymbol{\omega}}$ that maximizes this likelihood can then be used to forecast the distribution over permutations for a new theorem $\boldsymbol{x}^*$ by evaluating $\mathrm{Perm}(\boldsymbol{\Lambda}(\boldsymbol{x}^*, \hat{\boldsymbol{\omega}}))$ for all permutations. This would incur factorial complexity; however, we are often only interested in the most likely permutation, which can be retrieved in polynomial time. Specifically for strategy scheduling the permutation with the highest predicted probability should reflect the orderings in the data. For this purpose, we use Theorem 3 to find the highest probability permutation $\boldsymbol{\pi}^*$ by sorting the values of $\{\boldsymbol{\Lambda}(\boldsymbol{x}^*, \hat{\boldsymbol{\omega}})_j\}_{j=1}^M$ in descending order.

Remark 4.3. A method named ListNet designed to rank documents for search queries using the Plakett-Luce distribution is evaluated by Cao et al [62]. Their evaluation uses a linear basis expansion. We can derive a similar construction in our model by setting

$$\boldsymbol{\phi}(\boldsymbol{x}^{(i)}, \boldsymbol{d}^{(j)}) = \left[\boldsymbol{x}^{(i)\intercal}, \boldsymbol{d}^{(j)\intercal}\right]^\intercal. \tag{4.4}$$

◁

Remark 4.4. The likelihood in Equation (4.3) can be maximized by minimizing the negative log likelihood $\ell(\boldsymbol{\omega}) = -\log \mathcal{L}(\boldsymbol{\omega})$, which (as shown by Schäfer and Hüllermeier [136]) is convex and therefore can be minimized using gradient-based methods. The minima may, however, be unidentifiable due to translation invariance, as demonstrated by Lemma 5. This problem is eliminated in our Bayesian model by the use of a Gaussian prior, as explained in Section 4.2.3. ◁

Example 4.2. Let there be $N = 2$ theorems and $M = 2$ strategies. Let the theorems and strategies be characterized by univariate values such that $x^{(1)} = 1$, $x^{(2)} = 2$, $d^{(1)} = 1$ and $d^{(2)} = 2$.

Suppose strategy $d^{(1)}$ is ideal for theorem $x^{(1)}$ and strategy $d^{(2)}$ for $x^{(2)}$, as shown on the right, where a + indicates the preferred strategy.

	$d^{(1)}$	$d^{(2)}$
$x^{(1)}$	+	−
$x^{(2)}$	−	+

This is evidently an example of a parity problem [26], and hence cannot be modelled by a simple linear expansion using the basis function mentioned in Remark 4.3. A solution in this instance is to use

$$\phi(x^{(i)}, d^{(j)}) = x^{(i)} \cdot d^{(j)}.$$

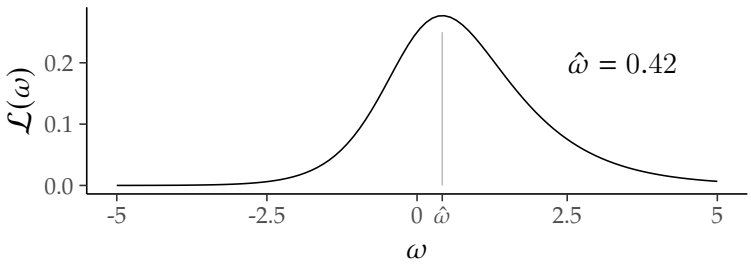


Figure 4.1: The likelihood function in Example 4.2.

The parameter ω is then one-dimensional, and the required training data takes the form $\mathcal{D} = \{([1,2]^\intercal, 1), ([2,1]^\intercal, 2)\}$. We find that $\mathcal{L}(w)$ is convex, with maxima at $\hat{\omega} = 0.42$ as shown in figure 4.1. ◁

4.2.3 Bayesian Inference

We place a Gaussian prior distribution on the parameter $\boldsymbol{\omega}$ of the model described in Section 4.2.2. This has two advantages: first, the posterior mode is identifiable, as noted by Johnson et al. [145] and demonstrated in Example 4.3; second, the parameter is regularized. With this prior specified as the normal distribution

$$\boldsymbol{\omega} \sim \mathcal{N}(\boldsymbol{m_0}, \boldsymbol{S_0}), \tag{4.5}$$

and assuming $\boldsymbol{\pi}$ is independent of $\mathcal{D}$ given $(\boldsymbol{x}, \boldsymbol{\omega})$, the posterior predictive distribution is

$$p(\boldsymbol{\pi}|\boldsymbol{x}^*, \mathcal{D}) = \int p(\boldsymbol{\pi}|\boldsymbol{x}^*, \boldsymbol{\omega}) p(\boldsymbol{\omega}|\mathcal{D}) \mathrm{d}\boldsymbol{\omega},$$

which may be approximated by sampling from the posterior,

$$\boldsymbol{\omega}^s \sim p(\boldsymbol{\omega}|\mathcal{D}), \tag{4.6}$$

to obtain

$$p(\boldsymbol{\pi}|\boldsymbol{x}^*, \mathcal{D}) \approx \frac{1}{S} \sum_{s=1}^{S} p(\boldsymbol{\pi}|\boldsymbol{x}^*, \boldsymbol{\omega}^s). \tag{4.7}$$

Given a new theorem $\boldsymbol{x}^*$, to find the permutation of strategies with the highest probability of success, using the approximation above would require its evaluation for every permutation of $\boldsymbol{\pi}$. This process incurs factorial complexity. We instead make a Bayes point approximation [95] using the mean values of the samples such that,

$$\begin{aligned} p(\boldsymbol{\pi}|\boldsymbol{x}^*, \mathcal{D}) &\approx p(\boldsymbol{\pi}|\boldsymbol{x}^*, \langle \boldsymbol{\omega}^s \rangle) && \text{using Equation (4.7)} \\ &= \Pr(\boldsymbol{\pi}|\boldsymbol{\Lambda}(\boldsymbol{x}^*, \langle \boldsymbol{\omega}^s \rangle)) && \text{using Equation (4.1),} \end{aligned}$$

where $\langle \cdot \rangle$ denotes mean value. The mean of the Plakett-Luce parameter for Bayesian inference has been used in prior work [72] to obtain good results. Furthermore, using

that, the highest probability permutation can be obtained by using Theorem 3, thereby incurring only the cost of sorting the items. This saving is substantial when generating a strategy schedule, because it saves on prediction time, which is important for the reasons stated in Remark 4.2.

Remark 4.5. In our implementation we set $\boldsymbol{m}_0 = 0$. This has the effect of prioritizing smaller weights $\boldsymbol{\omega}$ in the posterior. Furthermore, we set $\boldsymbol{S}_0 = \eta \boldsymbol{I}, \eta \in \mathbb{R}$, where $\boldsymbol{I}$ is the identity matrix. Consequently, the hyperparameter η controls the strength of the prior, since the entropy of the Gaussian prior scales linearly by $\log |\boldsymbol{S}_0|$. ◁

Remark 4.6. A specialization of the Plakett-Luce distribution using the Thurstonian interpretation admits a Gamma distribution conjugate prior [72]. That, however, is unavailable to our model when parametrized as shown in Equation (4.1). ◁

4.2.4 Sampling

We use the MCMC Metropolis-Hastings algorithm (Section 4.1.3) to generate samples from the posterior distribution. It is to our advantage that we do not need to calculate the normalization constant of the target density for MCMC sampling; to generate samples $\boldsymbol{\omega}^s$ from the posterior, which is,

$$\begin{aligned} p(\boldsymbol{\omega}|\mathcal{D}) &\propto p(\mathcal{D}|\boldsymbol{\omega})p(\boldsymbol{\omega}) \\ &= \mathcal{L}(\boldsymbol{\omega})\mathcal{N}(\boldsymbol{m_0}, \boldsymbol{S_0}) \text{ by Equations (4.3) and (4.5),} \end{aligned} \tag{4.8}$$

the posterior only needs to be computed in this unnormalized form.

In this work, we choose a random walk proposal of the form

$$q(\boldsymbol{\omega}'|\boldsymbol{\omega}) = \mathcal{N}(\boldsymbol{\omega}'|\boldsymbol{\omega}, \Sigma_q), \tag{4.9}$$

and tune Σ_q for efficient sampling simulation. We start the simulation at a local mode $\hat{\boldsymbol{\omega}}$, and set $\mathcal{N}(\hat{\boldsymbol{\omega}}, \Sigma_q)$ to approximate the local curvature of the posterior at that point using methods by Rossi [56]. Specifically, our procedure for computing Σ_q is as follows.

1. First, writing the posterior from Equation (4.8) as

   $$p(\boldsymbol{\omega}|\mathcal{D}) = \frac{1}{Z} e^{-E(\boldsymbol{\omega})},$$

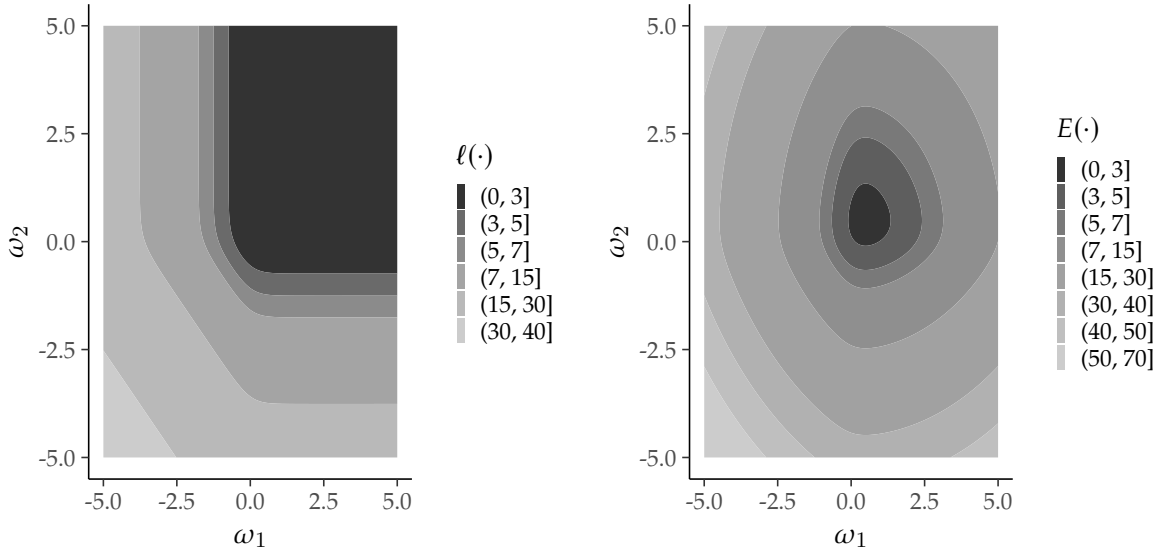
   where Z is the normalization constant, we have

   $$E(\boldsymbol{\omega}) = -\log \mathcal{L}(\boldsymbol{\omega}) - \log \mathcal{N}(\boldsymbol{m_0}, \boldsymbol{S_0}). \tag{4.10}$$

   We find a local mode $\hat{\boldsymbol{\omega}}$ by optimizing $E(\boldsymbol{\omega})$ using a gradient-based method.
2. Then, using a Laplace approximation (Section 4.1.2), we approximate the posterior in the locality of this mode to

   $$\mathcal{N}(\hat{\boldsymbol{\omega}}, H^{-1}), \text{ where } H = \nabla\nabla E(\boldsymbol{\omega})|_{\hat{\boldsymbol{\omega}}}$$

   is the Hessian matrix of $E(\boldsymbol{\omega})$ evaluated at that local mode.



(a) Likelihood function $\ell(\omega_1, \omega_2)$ **(b)** Posterior function $E(\omega_1, \omega_2)$

Figure 4.2: Comparison of the shape of the likelihood and the posterior functions.

3. Finally, we set

$$\Sigma_q = s^2 H^{-1}$$

in Equation (4.9), where s is used to tune all the length scales. We set this value to $s^2 = 2.38$ based on the results by Roberts and Rosenthal [36].

Remark 4.7. When calculating r in Algorithm 4.1 during sampling, to evaluate the unnormalized posterior at any point $\boldsymbol{\omega}^s$ we compute it from Equation (4.10) as $\exp(-E(\boldsymbol{\omega}^s))$. It is therefore the only form in which the posterior needs to be written in code. ◁

Example 4.3 (Gaussian Prior)**.** To demonstrate the effect of using a Gaussian prior, we build upon Example 4.2, with the data taking the form

$$\mathcal{D} = \{([1, 2]^\intercal, 1), ([2, 1]^\intercal, 2)\}.$$

We perform basis expansion as explained in Section 6 with prior parameter $\eta = 1.0$, kernel $\sigma = 0.1$ and $\varsigma = 2$ centres. Thus, the model parameter is

$$\boldsymbol{\omega} = [\omega_1, \omega_2]^\intercal, \quad \boldsymbol{\omega} \in \mathbb{R}^2.$$

The unnormalized negative log posterior $E(\omega_1, \omega_2)$, as defined in Equation (4.10), is shown in Figure 4.2b; and the negative log likelihood $\ell(\omega_1, \omega_2) = -\log \mathcal{L}(\omega_1, \omega_2)$ as mentioned in Remark 4.4, is shown in Figure 4.2a. Note the contrast in the shape of the two surfaces. The minimum is along the top-right portion in Figure 4.2a, which is flat and leads to an unidentifiable point estimate, whereas in Figure 4.2b, the minimum is in a narrow region near the centre. The Gaussian prior, in informal terms, has lifted the surface up, with an effect that increases in proportion to the distance from the origin. ◁

4.2.5 Basis Expansion

Example 4.2 shows how the linear expansion in Remark 4.3 is ineffective even in very simple problem instances. The maximum likelihood bilinear model presented by Schäfer and Hüllermeier [136] is related to our model defined in Section 4.2.1 with the basis performing the Kronecker (tensor) product $\phi(x, d) = x \otimes d$. Their results show such an expansion produces a competitive model, but falls behind in comparison to their non-linear model. On that basis, we proceed with a non-linear expansion.

To model non-linear interactions between theorems and strategies, we use a *Gaussian kernel* for the basis expansion.

Definition 4.4 (Gaussian kernel)**.** The Gaussian kernel κ, also known as the squared exponential kernel (see Appendix 3.A) or radial basis function, is

$$\kappa(\boldsymbol{y}, \boldsymbol{z}) = \exp\left(-\frac{1}{2}(\boldsymbol{y} - \boldsymbol{z})^T \Sigma^{-1} (\boldsymbol{y} - \boldsymbol{z})\right),$$

where Σ is a covariance matrix. As is typical, we set Σ to be diagonal and spherical, with $\Sigma_{j,j} = \sigma$, to obtain

$$\kappa(\boldsymbol{y}, \boldsymbol{z}) = \exp\left(-\frac{\|\boldsymbol{y} - \boldsymbol{z}\|^2}{2\sigma^2}\right), \quad \text{for any } \sigma > 0.$$

Parameter σ sets the length-scale along every dimension. ◁

The Gaussian kernel $\kappa(\boldsymbol{y}, \boldsymbol{z})$ effectively represents the inner product of $\boldsymbol{y}$ and $\boldsymbol{z}$ in a space of theoretically infinite dimensionality. Smaller values of σ correspond to a more flexible inner product space. Larger values of σ will reduce the kernel to a constant function, as detailed in [47]. For our ranking model, we must tune σ to balance between over-fitting and under-performance.

We use the Gaussian kernel for basis expansion by setting

$$\phi(\boldsymbol{x}, \boldsymbol{d}) = \left[\kappa\left([\boldsymbol{x}^\intercal, \boldsymbol{d}^\intercal]^\intercal, \boldsymbol{c}^{(1)}\right), \ldots, \kappa\left([\boldsymbol{x}^\intercal, \boldsymbol{d}^\intercal]^\intercal, \boldsymbol{c}^{(C)}\right)\right]^\intercal,$$

where $\{\boldsymbol{c}^{(i)}\}_{i=1}^{C}$ is a collection of *centres*. By choosing centres to be themselves composed of theorems $\boldsymbol{x}^{(.)}$ and strategies $\boldsymbol{d}^{(.)}$, such that $\boldsymbol{c}^{(.)} = [\boldsymbol{x}^{(.)\intercal}, \boldsymbol{d}^{(.)\intercal}]^\intercal$, the basis expansion above represents each data item with a non-linear inner product against other known items.

To find the relevant subset of $\mathcal{D}$ from which centres should be formed, we follow the method described in the steps below.

1. Initially, we set the collection of centres to every possible centre. That is, for N theorems and M strategies, we produce a centre for every combination of the two, thereby producing $C = N \cdot M$ centres.

2. Next, we use ϕ to expand every centre to produce the $C \times C$ matrix $\mathbf{\Gamma}$ such that

$$\mathbf{\Gamma}_{i,j} = \phi(\boldsymbol{c}^{(i)})_j = \kappa(\boldsymbol{c}^{(i)}, \boldsymbol{c}^{(j)}).$$

3. Then, we generate a vector $\boldsymbol{\gamma}$ such that γ_i represents a *score* for centre $\boldsymbol{c}^{(i)}$. Since each centre is a combination of a theorem and a strategy, we set the score to signify how well the strategy performs for that theorem, as detailed in Remark 4.8 below.
4. Finally, we use Automatic Relevance Determination (ARD) [24] with $\mathbf{\Gamma}$ as input and $\boldsymbol{\gamma}$ as the response variable. The result is a weight assignment to each centre to signify its relevance. The highest absolute-weighted ς centres are chosen, where ς is a parameter which decides the total number of centres.

This method is inspired by the procedure used in Relevance Vector Machines [37] for a similar purpose.

Remark 4.8 (score). For a strategy that succeeds in proving a theorem, the score for the pair is the fraction of the time allocation left unconsumed by the prover. For an unsuccessful strategy-theorem combination, we set the score to a value close to zero. Note that we designed a more nuanced score in Definition 2.4 for a similar purpose, which may be a viable alternative in this context. ◁

Remark 4.9 (ς). The parameter ς is another tunable parameter which, in similar fashion to the parameter σ earlier in this section, controls the model complexity introduced by the basis expansion. Both variables must be tuned together. ◁

4.2.6 Model Selection and Time Allocations

From Remark 4.9, ς and σ are hyperparameters that control the complexity introduced into our model through the Gaussian basis expansion; and Remark 4.5 introduces η, the hyperparameter that controls the strength of the prior. The final model is selected by tuning them. Tuning must aim to avoid overfitting to the training data; and to maximize, during testing, either the savings in proof-search time or the number of theorems proved. However, we do not have a closed-form expression relating these parameters to this aim, thus any combination of the parameters can be judged only by testing them.

In this work we have used *Bayesian optimization* [118] to optimize these hyperparameters. Bayesian optimization is a black-box parameter optimization method that attempts to search for a global optimum within the scope of a set resource budget. It models the optimization target as a user-specified *objective function*, which maps from the parameter space to a loss metric. This model of the objective function is constructed using *Gaussian Process* (GP) regression [60], using data generated by repeatedly testing the objective function.

Our specified objective function maps from the hyperparameters $(\varsigma, \sigma, \eta)$ to a loss metric ξ. We use cross-validation within the training data while calculating ξ to penalize

hyperparameters that over-fit. Hyperparameters are tuned at training time only, after which they are fixed for subsequent testing. The final test set is never used for any hyperparameter optimization.

In the method presented thus far we are only permuting strategies with fixed time allocations to build a sequence for a strategy schedule. In this setting, the number of theorems proved cannot change, but the time taken to prove theorems can be reduced. Therefore, with this aim, a useful metric for ξ is the total time taken by the theorem prover to prove the theorems in the cross-validation test set.

However, we can take further advantage of the hyperparameter tuning phase to additionally tune the times allocated to each strategy, by treating these times as hyperparameters. Therefore, for each strategy $\boldsymbol{d}^{(i)}$ we create a hyperparameter $\nu^{(i)} \in (0, 1)$ which sets the proportion of the proof time allocated to that strategy. We can then optimize our model to maximize the number of theorems proved; a count of the remaining theorems is then a viable metric for ξ. Note that once the $\nu^{(\cdot)}$ are set, time allocation for $\boldsymbol{d}^{(i)}$ is fixed to $\nu^{(i)}$, irrespective of its order in the strategy schedule.

Remark 4.10. Our results include two types of experiment:

- one where the time allocations for each strategy are set to the defaults shipped with our reference theorem prover, and so we optimize for saving proof time; and
- another wherein we allocate time to each strategy during the hyperparameter tuning phase, and so we optimize for proving the maximum number of theorems.

◁

4.2.7 Training Data and Feature Extraction

Our chosen theorem prover, iLeanCoP, is shipped with a fixed strategy schedule consisting of five strategies. It splits the allocated proof time across the first four strategies by 2%, 60%, 20% and 10%. The first strategy is complete, with respect to FOL, and therefore usually expected to take up its entire time allocation. The second, third and fourth strategies are incomplete, and may exit early on failure. Any leftover time due to early termination of these strategies should be used up by the final strategy to maximize the chances of finding a proof. Therefore, the fifth and final strategy in iLeanCoP, which we refer to as the *fallback strategy*, is designed to be complete and is allocated all the remaining time.

Emulating iLeanCoP**.** We have constructed a dataset by attempting to prove every theorem in our problem library using each of these strategies individually. With this information, the result of any proof attempt can be calculated by emulating the behaviour of iLeanCoP. This is how we evaluate the predicted schedules — we emulate a proof attempt by iLeanCoP using that schedule for each theorem in the test set. For a faithful

Table 4.1: Features of the four main strategies.

Strategy	Parameter					Completeness
	`def`	`scut`	`cut`	`comp(7)`	`conj`	
`def,scut,cut,comp(7)`	1	1	1	1	0	1
`def,scut,cut`	1	1	1	0	0	0
`conj,scut,cut`	0	1	1	0	1	0
`def,conj,cut`	1	0	1	0	1	0

emulation of the fallback strategy, it is always attempted last, and therefore any new schedule is only a permutation of the first four strategies. Our experiments allocate a time of 600 s per theorem. The dataset is built to ensure that, within this proof time, any such strategy permutation can be emulated. We kept a timeout of 1200 s per strategy per theorem when building the dataset, which is more than sufficient for current experiments and gives us headroom for future experiments with longer proof times.

Strategy Features. Each strategy in iLeanCoP consists of a time allocation and parameter settings; the parameters are described by Otten [68]. We use a one-hot encoding feature representation for strategies based on the parameter setting as shown in Table 4.1. Another feature noting the completeness of each strategy is also shown. Yet another feature (not shown in the table) contains the time allocated to each strategy. Note the fallback strategy is used in prover emulation but not in the schedule prediction.

Theorem Features. We have constructed a dataset containing features extracted from the first-order logic problems in TPTP (see Appendix 4.A). To minimize the computational overhead of feature extraction, in keeping with our goal noted in Remark 4.2, we use features that can be collected when the theorem is parsed by the prover. We have described those features in Chapter 2. We transform our features using ZCA (see Definition 4.1).

Coverage. As mentioned above, we run iLeanCoP on every first-order theorem in TPTP with each strategy allocated 1200 s. Although every theorem in intuitionistic logic also holds for classical logic, the converse does not hold. For that reason and because of the limitations of iLeanCoP, many theorems remain unproved by any strategy. We exclude these theorems from our experiments, leaving us with a data-set of 2240 theorems.

4.2.8 Experiments

We present two experiments in this work, as noted in Remark 4.10. In this section, we describe our experimental apparatus in detail.

As noted in Section 4.2.7, our data contains:

- $N = 2240$ theorems that are usable in our experiments;
- five strategies, of which $M = 4$ are used to build strategy schedules since one is a fallback strategy; and
- features $\boldsymbol{x}^{(i)}$ of theorems where $i \in [1, N]$ and features $\boldsymbol{d}^{(j)}$ of strategies where $j \in [1, M]$.

This data needs to be presented to our model for training in the form of $\mathcal{D} = \{(\boldsymbol{\pi}^{(i)}, \boldsymbol{x}^{(i)})\}_{i=1}^{N}$, as described in Section 4.2.2. Since the two experiments have slightly different goals, we specialize $\mathcal{D}$ according to each.

When aiming to predict schedules that minimize the time taken to prove theorems, a natural value for $\boldsymbol{\pi}^{(i)}$ is the index order that sorts strategies in increasing amounts of time taken to prove theorem i. However, some strategies may fail to prove theorem i within their time allocation. In that case, we consider the failed strategies equally bad and place them last in the ordering in $\boldsymbol{\pi}^{(i)}$.

When the goal is only to prove more theorems, the strategies that succeed are all considered equally ranked above the failed strategies. In this mode, the successful strategies are similarly permuted in the data, in addition to those that failed.

In each experiment, a random one-third of the N theorems are separated into a *holdout* test set $\dot{\boldsymbol{N}}$, leaving behind a training set $\ddot{\boldsymbol{N}}$. This training set is first used for hyperparameter tuning using BO. As explained in Section 4.2.6, each hyperparameter combination is tested with five-fold cross-validation within $\ddot{\boldsymbol{N}}$, to penalize instances that overfit to $\ddot{\boldsymbol{N}}$. This results in estimated optimum values for the hyperparameters. These are used to set the model, which is then trained on $\ddot{\boldsymbol{N}}$ and then finally evaluated on $\dot{\boldsymbol{N}}$. The whole process is repeated ten times with new random splits $\dot{\boldsymbol{N}}$ and $\ddot{\boldsymbol{N}}$ to create one set of ten results for that experiment.

4.3 Results

Each experiment, repeated ten times, is conducted in two phases: first, hyperparameter optimization; and second, model training and evaluation. The bounds on the search space in the first phase were always the same (see Appendix 4.A). The holdout test set contained 747 theorems. A proof time of 600 s was emulated.

4.3.1 Experiment 1: Optimizing Proof Attempt Time

The results are shown in Figure 4.3. The total prediction time for all 747 theorems, averaged across the trials, is 0.14 s.

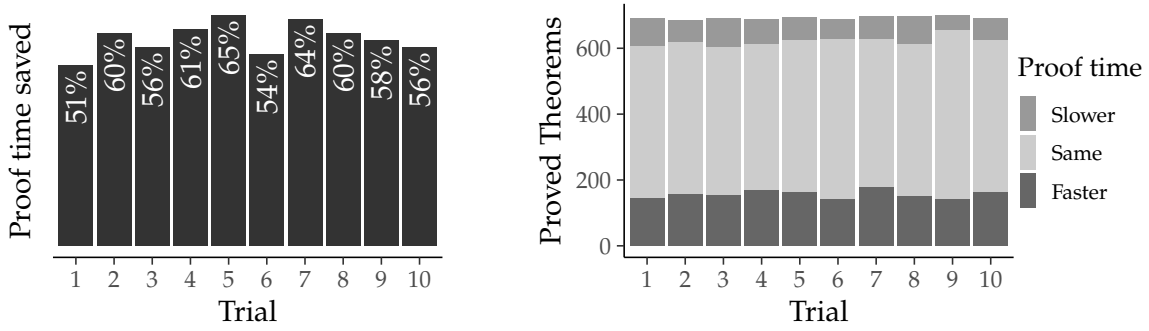


Figure 4.3: Results of Experiment 1. Proof times are compared with precision 10^{-6}s.

The times across proof attempts are not normally distributed, for both the unmodified iLeanCoP schedule and the predicted ones, as confirmed by a Jarque-Bera test [16]. Therefore, we used the right-tailed Wilcoxon signed-rank test for a pair-wise comparison of the times taken for each theorem by the original schedule in iLeanCoP versus the predicted schedules, resulting in a p-value of less than 10^{-6} in each trial, confirming the alternate hypothesis that the reduction in time taken to prove each theorem comes from a distribution with median greater than zero. This confirms that the time savings are statistically significant. Furthermore, we note from Figure 4.3 a saving of more than 50% in the total proof-time in each trial.

4.3.2 Experiment 2: Proving More Theorems

We set our hyperparameter search to find time allocations for strategies. The resulting predicted schedules have gains and losses when compared to the original schedule, as shown in the four facets of Figure 4.4. However, there is a consistent gain in the number of theorems proved and a gain of five theorems on average

In facet (†) of Figure 4.4, we see counts of the number of theorems proved by iLeanCoP using the schedules our system predicted, out of the set of theorems that iLeanCoP could not resolve using its original schedule. We can see a positive outcome in every trial in this regard, and an average of around eleven across the trials.

Facet (‡) of Figure 4.4 counts instances where iLeanCoP would have succeeded had our method not intervened to alter its strategy schedule. This is typically lower than the counts in (†). However, in trial seven, the losses equal the gains, and so we do not obtain a net improvement in that round of this experiment. There are net gains in every other round.

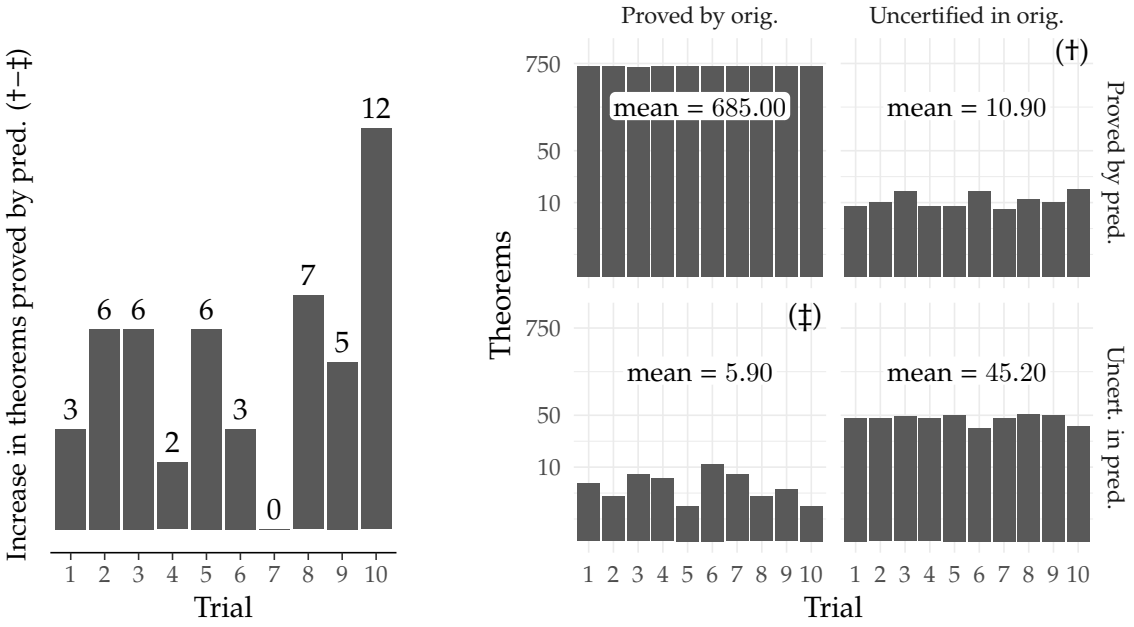


Figure 4.4: Comparison of the proof attempts by the original (orig.) and predicted (pred.) schedules in Experiment 2. Theorems which are proved by pred. but could not be proved by orig. are counted in †, and the vice versa in ‡. When iLeanCoP fails to prove a theorem, we place it here under the title of "Uncertified" or, in short, "Uncert".

4.4 Related Work

Prior work on machine learning for *algorithm selection*, such as that introduced by Leyton-Brown et al. [43], is a precursor to our work. In that topic, the machine learning methods must perform the task of selecting a good algorithm from within a portfolio to solve the given problem instance. Typically, as was the case in the work by Leyton-Brown et al. [43], the learning methods predict the runtime of all algorithms, and then pick the fastest predicted one. This line of enquiry has been extended to select algorithms for SMT solvers — a recent example is MachSMT by Scott et al. [155]. The machine learning models in MachSMT are trained by considering all the portfolio members in pairs for each problem in the training set. This method is called *pairwise ranking*, which contrasts from our method, called *list-wise ranking*, in which we consider the full list of portfolio members all together.

In terms of the machine learning task, the work on scheduling solvers bears greater similarity to our presented work. In MedleySolver, for example, Pimpalkhare et al. [154] frame this task as a multi-armed bandit problem. They predict a sequence of solvers as well as the time allocation for each to generate schedules for the goal problems. MedleySolver is able to solve more problems than any individual solver would on its own.

Hůla et al. [152] have made use of Graph Neural Networks (GNNs) for solver scheduling. They produce a regression model to predict, for the given problem, the runtime of all the solvers; which is used as the key to sort the solvers in increasing order of predicted runtime to build a schedule. This is an example of *point-wise ranking*.

4.5 Conclusions

We have presented a method to specialize, for the given goal theorem, the sequence of strategies in the schedule used in each proof attempt. A Bayesian machine learning model is trained in this method using data generated by testing the prover of interest. When evaluated with the iLeanCoP prover using the TPTP library as a benchmark, our results show a significant reduction in the time taken to prove theorems. For theorems that are successfully proved, the average time saving is above 50%. The prediction time is on average low enough to have a negligible impact on the resources subtracted from the proof search itself.

We also extend this method to optimize time allocations to each strategy. In this setting, our results show a notable increase in the number of theorems proved.

This work shows, by example, that Bayesian machine learning models designed specifically to augment heuristics in theorem provers, with detailed consideration of the computational compromises required in this setting, can deliver substantial improvements.

4.A Implementation, Code and Data

This work is implemented primarily in MATLAB [157]. All experiments can be reproduced using the code, data and instructions available at [159]. The hyperparameter search space in all experiments was restricted to

$\varsigma \in [10, 300]$,
$\sigma \in [0.01, 100.0]$ and
$\eta \in [1, 100]$.

For reproduction, the search can be skipped by using the hyperparameter values listed in the table to the right.

Experiment 1			Experiment 2		
ς	σ	η	ς	σ	η
152	3.15	17.98	56	0.04	39.07
91	3.25	6.29	49	94.40	3.23
119	2.94	69.44	10	51.86	1.70
130	2.26	9.29	29	22.22	8.35
124	2.88	9.43	25	0.02	1.03
145	5.38	85.68	29	20.98	4.45
139	3.90	61.59	14	42.79	4.90
57	2.85	31.04	21	2.81	1.57
93	2.58	12.13	11	98.49	5.12
34	2.70	42.62	42	2.61	34.92

4.B Why not just use classification?

A reviewer of this work proposed an alternative: treat each possible schedules, i.e., each permutation, as a distinct individual class and train a classifier to predict them. This method is inferior, at least theoretically, because it precludes crucial domain knowledge from the model. The different schedules are very strongly related to each other, specifically by being permutations of the same. Without this information available as prior knowledge to a model, it would be handicapped by having to treat each schedule as an atomic category.

4.C Acknowledgments

Initial investigations for this work were co-supervised by Prof. Mateja Jamnik, Computer Laboratory, University of Cambridge, UK.

This work was supported by: the UK Engineering and Physical Sciences Research Council (EPSRC) through a Doctoral Training studentship, award reference 1788755; and the ERC Advanced Grant ALEXANDRIA (Project GA 742178).

Computations for this work was performed using resources provided by the Cambridge Service for Data Driven Discovery (CSD3) operated by the University of Cambridge Research Computing Service, provided by Dell EMC and Intel using Tier-2 funding from the Engineering and Physical Sciences Research Council (capital grant EP/P020259/1), and DiRAC funding from the Science and Technology Facilities Council.

5

Discussion and Conclusions

> We can only see a short distance ahead, but we can see plenty there that needs to be done.
>
> *Alan Turing* [5]

Inspired by theories of human cognition in prior work, researchers such as Booch et al. [148] have suggested that the capabilities of Artificial Intelligence (AI) systems could be enhanced by the mechanisms roughly described as — thinking fast and slow. Such a mechanism consists of two cooperating processes: one that "thinks fast", i.e., an imprecise but fast reasoning engine that is analogous to human intuition; and another that, analogous to careful and rational thought processes, "thinks slow", to provide for an exact, perhaps formal, reasoning engine. One may view modern Automated Theorem Provers (ATPs) in such light, with the heuristics "thinking fast" to navigate through a bewilderingly large search space, whilst the decision procedures "think slow" to ensure a formally correct solution. Much of the work presented in this thesis focuses on "fast thinking" enhancements to ATPs, by use of Machine Learning (ML).

In the work presented in this thesis, we chose to either custom build or notably tailor the ML methods we employed, with focus on strategies in ATPs. Wolpert and Macready [30] have discredited the expectation that one could invent an optimization algorithm that is generally better than all others. They produce a collection of theorems under the title of *no free lunch*, to show that if one algorithm outperforms another over a set of optimization problems, then the converse is true over the complement set. Since optimization is fundamental in ML, we cannot — a priori — expect algorithms that work for other domains, such as document ranking, to lend themselves equally well to ATP tasks, such

as strategy scheduling. The experience among ML practitioners, the authors included, concurs with this theory. This is the prime motivation behind our choice of custom ML models.

Modern ATPs draw upon a vast and long-standing research base, which we addressed in brief in Chapter 1. Our aim, when developing the ML methods presented in this thesis, was to prime them with this knowledge, in addition to what could be learnt from just the raw training data. Original developments, small and large, achieve this across this work, some of which we highlight below.

- In Chapter 2, we engineered features based on an expert understanding of prover strategies. That transformation of the raw data into specialized features is an essential component of the ML model we eventually develop in Chapter 4; and so is the method for feature selection. This effectively transfers our knowledge of strategies on to any ML system that uses those features.
- In addition to the ML model, Bayesian ML lets us inform the prior probability distribution with additional domain-specific knowledge and stochastic preferences. The model we developed in Section 4.2.2 simply cannot function for our application without the prior probability distribution added in Section 4.2.3.
- The ML model we developed in Chapter 4 is specifically for strategy scheduling in ATPs. Engineering trade-offs and design choices that tailor the ML model are highlighted in the text. Through the engineered features as well as the scores defined in Remark 4.8, the model is a-priori informed of numerous aspects of strategy scheduling in ATPs.
- We designed objective functions for Bayesian optimization of specific ATP components in Chapter 3. Additionally, we discussed how integer parameters, which are prevalent in ATPs, demand special handling when they are optimization targets.

We have reaped multiple benefits from the use of well-informed ML models. Models with good prior information need less data to converge on learnings. Although plenty of training data is now available in the field of ATP, as noted in Section 1.3.2, features for ML need to be generated, sometimes, by actually proving a large selection of theorems. The computational cost of this may limit the volume of usable training data. Eventually, we achieved a few impressive results, despite a constrained use of computational resources.

We have good reasons to avoid the use of opaque ML technologies in ATPs. The behaviour of fully specified ML models is easier to control and analyse. This is evident from Example 4.2 and Example 4.3, wherein we get deep insights into the behaviour of our model. Additionally, transparency is important in ATPs for some of its applications. In Chapter 1, we laid modest foundations for the rest of this text. We introduced, briefly, the theoretical foundations of modern ATPs; reviewed a cross-section of the body of work leading up to ours; and opened a discussion of Bayesian learning and Intuitionism. Applications of ATPs demand rigour, correctness and exact insights: a view that is

reinforced by the examples we offer in that chapter. We prefer enhancements to ATPs by ML methods to maintain the transparency and reproducibility that is expected by such applications.

In the experiments detailed in this thesis, we test with ATPs that perform competitively yet have relatively simple implementations. We attribute, partly, the clarity in our presentation to this choice. In Chapter 4, for example, we develop a method for strategy scheduling specifically for ATPs, and explain in Remark 4.1 why we chose iLeanCoP for the experiments. In Section 3.2, we optimize just the SInE algorithm, which is fully described in Section 3.2. And in Section 3.3, we optimize a few parameters of MiniSat: a compact SAT solver. In Chapter 2, we gain insights of the very large Thousands of Problems for Theorem Provers (TPTP) library, but through the lens of LeanCoP and iLeanCoP for a very detailed view. Another noteworthy contribution in Chapter 2, through Figure 2.6, is the credence added to the expectation that the right strategy for the goal theorem proves it quickly (see Reger and Suda [126]); otherwise, it is unlikely to succeed at all. All throughout, we are able to rigorously evaluate the empirical outcomes and present clear results.

The research question we explore in Chapter 2 concerns the features of goal theorems that could be used by ML methods to prepare customized strategy schedules. We delivered some answers by plotting a collection of empirical examinations, in addition to a series of hypothesis tests. Spread across the chapter, we developed multiple novel contributions. For one, we presented a collection of original features that we have designed for the purposes of strategy scheduling. These features are generic and thus should be reusable in provers other than LeanCoP and goal theorems other than those in the TPTP. Another contribution there is the methodology we employed, with which we were able to extract and prioritize such features. Our methodology is even independent of the strategy-scheduling system. Therefore, the features may be engineered even before the scheduling system is developed, as long as good strategies already exist. In Section 2.4.2, we showed that our method is even portable to entirely different feature sets. An interesting application of our methods is in the examination of features developed elsewhere, which we may not have an intuitive understanding of, as shown in Section 2.4.2. One aspect of Chapter 2 that contrasts it from the other technical chapters is the absence of Bayesian statistics. The authors are pragmatists with regard to probability theory axioms. Fellow researchers may find much readily reusable material in Chapter 2 for future related works.

In Section 2.4.3, we note an example of a popular alternative approach to feature extraction: automatic extraction from the raw text of a proof repository using ML (see, for e.g., Wang et al. [128]). That approach contrasts with ours. There are distinct advantages in either direction.

Automatic extraction: An advantage automatic extraction has is in the automation and

in the discovery of relevant features we may miss with our approach. However, its exposure to training information is limited to what is available in the raw data alone. There is relevant training information contained in the context around the raw data; for e.g., in the TPTP grammar, in the semantics and in the underlying mathematical theories. The grammar, in particular, is available and is complete, unlike in the domain of natural languages.

Manual feature engineering: With manual feature engineering, on the other hand, we can incorporate such contextual information in the features. For example, take the Quantifier Alternations feature extracted using Algorithm 2.1. That feature embodies information that is external to the text of the theorems: the TPTP grammar; the semantics of negation and quantifiers; and an understanding of theorem proving techniques.

It may be beneficial to combine these two approaches in future works. Some of the more complex features developed in Chapter 2 are likely to remain relevant despite advances in automatic extraction.

We offer yet another point in favour of features that are derived from the parse tree of the goal theorem; that includes our features in Chapter 2 as well as the ENIGMA features introduced in Section 2.4.2. Consider the use of such features in deployment, for e.g., for strategy selection or scheduling. There is a computational cost to parsing the theorem to extract these features. However, in any deployment scenario, the actual theorem prover in use will have to necessarily parse the theorem. So, as long as the ML system is able to use the parse tree prepared by the prover itself to extract these features, there is very little additional computational cost of deriving the sort of features we have designed. Such efficiency is important because it is standard to give a prover a fixed wall-clock deadline; typically 300 s. Any time taken up by an auxiliary ML enhancement should be accounted for within this deadline, and that reduces the time available to the core prover to conduct the proof-search itself. By reusing the parse-tree of the goal theorem prepared by the core prover, our features can be extracted with minimal overhead.

We show, in Chapter 3, how Bayesian optimization could be used to invent strategies in ATPs. There is a clear correspondence between strategies and optimization, in that a strategy is an instantiation of an ATP, with that instantiation optimized for some class of theorems. We introduced the theoretical foundations of Bayesian optimization in that chapter. Subsequently, we delved into the nuances of its use for practical ATP optimization, through two disparate experiments. First, we optimized the parameters of the SInE premise selection algorithm, with the Archive of Formal Proofs (AFP) as the proof repository, for individual AFP articles. One novel contribution there is the objective function we designed for this purpose. The second experiment we presented demonstrated the optimization of the MiniSat solver embedded in CBMC. The optimization goal there was slightly different from typical strategy invention, because we optimized parameters for specific proof goals but without an interest in the

generalizability of the discovered parameters. This is because, in the case of regression tests, the proof goals do not change until the tests themselves change. Nonetheless, our contributions there do translate to strategy invention. Of note is our discussion on how we handled errors in the objective function given to the Bayesian optimization framework in MATLAB. We found it advantageous to model timeouts in the prover as errors, as well as to set a larger timeout during the parameter search; we elaborated both decisions. The two experiments hint at the success of our approach.

A thorn in the side of Bayesian optimization when applied to ATPs is the presence of categorical or integer-valued parameters. We discuss the underlying problems and some solutions in Section 3.1. However, Gaussian Processes (GPs) Regression, the ML algorithm underlying Bayesian optimization, is geared towards real-valued variables from the ground up. In a recent strategy invention tool called Grackle, presented by Hula et al. [158], this specific problem is absent owing to their use of the SMAC parameter optimizer (see opening of Chapter 3 or Hutter, Hoos and Leyton-Brown [86]). The trade-offs between such work and ours are as follows.

- The ML algorithm used in SMAC by default is random forests, which are composed of an ensemble of decision trees. The decision trees learn by recursively partitioning the input space; and such partitioning for integer spaces is procedurally identical to that for real-valued spaces. Thus, SMAC can model integer or categorical parameters in their true form, as opposed to the conversion to reals that may be needed by GPs. On the other hand, GPs have the advantage of a theoretically rigorous construction, with closed form equations built on the foundations of Bayesian probability theory. This gives us a grip on its difficulties with discrete-valued inputs and means for analysis of improvements.
- Furthermore, the uncertainty estimates produced by GPs have a clear mathematical interpretation — they represent Bayesian uncertainty. Good uncertainty estimates are essential in model-based optimization (see Section 3.1 for details), to find a balance between exploration versus exploitation of the model of the parameter space. Contrastingly, random forests derive uncertainty estimates from the variation in predictions across its collection of decision trees. It is difficult to explain the meaning of this measure of uncertainty in terms of probability theory.

Such trade-offs across ML algorithms fluctuate over time with progress in research and implementations. Improvements in Bayesian optimization are a field of active research.

Along with specialized ML models, our use of Bayesian optimization was fundamental to the successes in this work. Of particular advantage were two of its capabilities: its ability to work effectively within a computational budget; and in certain implementations, such as in MATLAB, its ability to run evaluations in parallel. We had access to a High Performance Computing (HPC) service for this work, but only its lowest tier which was provided free of cost. Yet, with Bayesian optimization, we used it effectively. The HPC service tier, in addition to restricted compute bandwidth, had time restrictions. Parallel

evaluations helped in this regard. Modern implementations, such as that in Optuna (see Akiba et al. [139]), parallelize even more effectively by pruning runs that show early signs of poor performance. The machinery in Chapter 4 simply does not work without hyperparameter tuning, and manually tuning it is nearly impossible. Thanks to our use of Bayesian optimization, we achieved statistically significant results.

The developments presented in this thesis all came together in Chapter 4. We used the features we developed, our exploration of Bayesian optimization and prior works in list-wise ranking to develop an ML engine for dynamic strategy scheduling. That engine generates a full strategy schedule that is customized to the goal theorem. This entire work is reproducible, with the code and data available online (see Mangla [159]). In deployment, our method would work as follows.

1. The user invokes the main prover on a specified goal theorem.
2. The prover parses the goal, whilst hooks in the parser collect features for ML.
3. Our ML method generates a strategy schedule customized for the goal.
4. The prover continues with the proof attempt with that schedule.

Our approach is novel in this regard, in that, when deployed in a prover, we only intervene in step 3, and therein we generate a full strategy schedule. We optimized our ML algorithm to minimize the time consumed in this intervention, to the point that it is immeasurably small. We built the theory from the ground up and presented our developments in much detail. Additionally, by means of examples, we examined individual parts for how the theoretical constructions translate to empirical outcomes. Eventually, we showed excellent savings in proof times in our experiments.

We gained a few more insights into the nature of our developments in Chapter 4, that we shall report. When we calculated features of the TPTP theorems, i.e., the features we designed in Chapter 2, they obtained a very wide range of values. This was because of the huge variation in the sizes of the theorems included in the TPTP. For example, the logic problems are small, but the hardware-verification problems are comparatively huge. We used the standard trick of calculating our formulas in the $\log$ domain in our model in Chapter 4 to deal with underflows. However, to avoid overflows, normalization of our features was necessary. We gained this automatically through our use of Zero-phase Component Analysis (ZCA, see Definition 4.1). Use of ZCA early on in our development process would have saved us from much confusion regarding the observed empirical outcomes. Another lesson of relevance to future work arrived in the development of the non-linear basis expansion in Section 4.2.5, for which we used Automatic Relevance Determination (ARD). We used ARD to select relevant centres around which we used the Gaussian kernel for basis expansion. Although ARD is a Bayesian method, as per our preference, it has some well-known shortcomings (see Wipf and Nagarajan [66]), including occasional instability. However, these shortcomings did not emerge in our specific use of ARD, which was only to select centres. The selection of centres only needed to be stable enough to limit the uncertainty in the hyperparameter optimization

(of ς specifically, see Remark 4.9). And such uncertainty is well within the grasp of Bayesian optimization. These insights will be useful in developments built upon our work.

The hyperparameter-optimization of the strategy scheduling model, described in Section 4.2.6, has room for much improvement. This step is computationally expensive; each hyperparameter combination that needs to be evaluated has to run through the full gamut of tests, and multiple times for cross-validation to avoid overfitting. But that cost is paid during training time — of which we care less about — and so we did not prioritize its optimization. Though, should this cost become prohibitive in the future, Bayesian statistics can rescue us. In an undeveloped part of this project, we designed a method for hyperparameter optimization, or rather its Bayesian analogue which is model selection, using variational inference. That would not have needed cross validation — instead, overfitting would be avoided by use of the Bayesian Occam's razor (see Murray and Ghahramani [54]). With that developed and implemented, we may not even need to use the HPC service for training the ML model.

The strategy scheduling model can scale with an increase in the number of pre-defined strategies, but only modestly so. We have tested it with a set of a dozen strategies. A notable change, however, is required to push it to scale to a double-digit number of strategies and beyond. The Plakett-Luce probability distribution (see Definition 4.3) has a particularly useful property for this purpose, in that it is still well-defined if, instead of permutations of all the M items, we only consider the permutations of all possible k item subsets, for some fixed k. So in the training data, we can choose to examine only the top-k strategies for any goal. If we keep k small, we retain the excellent performance of our existing system. We have tabled this optimization for future work.

This work serves as an example of the symbolic and stochastic approaches to artificial intelligence complementing each other. Although the research communities pursuing either of these directions are thriving, few pursue synergies across both avenues. We hope for more of such collaborations in the future.

Bibliography

[1] David Hilbert. *Mathematical Problems*. Lecture, International Congress of Mathematicians, Paris. 1900.

[2] Alfred North Whitehead and Bertrand Russell. *Principia Mathematica*. Cambridge University Press, 1910.

[3] Godfrey Harold Hardy. 'Mathematical Proof'. In: *Mind* XXXVIII.149 (Jan. 1929), pp. 1–25.

[4] Alonzo Church. 'A formulation of the simple theory of types'. In: *Journal of Symbolic Logic* 5.2 (1940), pp. 56–68.

[5] A. M. Turing. 'Computing Machinery and Intelligence'. In: *Mind* 59.236 (1950), pp. 433–460.

[6] R. Duncan Luce. *Individual Choice Behavior: A Theoretical Analysis*. Wiley, 1959.

[7] Martin Davis, George Logemann and Donald Loveland. 'A machine program for theorem-proving'. In: *Communications of the ACM* 5.7 (1962), pp. 394–397.

[8] John A. Nelder and R. Mead. 'A Simplex Method for Function Minimization'. In: *The Computer Journal* 7.4 (1965), pp. 308–313.

[9] R. L. Plackett. 'The Analysis of Permutations'. In: *Journal of the Royal Statistical Society: Series C (Applied Statistics)* 24.2 (1975), pp. 193–202.

[10] Imre Lakatos. *Proofs and Refutations: The Logic of Mathematical Discovery*. Cambridge University Press, 1976.

[11] Sheldon B. Akers Jr. 'Binary Decision Diagrams'. In: *IEEE Transactions on Computers* 27.6 (1978), pp. 509–516.

[12] Alan Bundy. *The computer modelling of mathematical reasoning*. Academic Press London, 1983.

[13] Albert Camilleri, Mike Gordon and Tom Melham. *Hardware verification using higher-order logic*. Tech. rep. UCAM-CL-TR-91. University of Cambridge, Computer Laboratory, Sept. 1986.

[14] Leon Sterling and Ehud Y. Shapiro. *The Art of Prolog: Advanced Programming Techniques*. Advanced programming techniques. MIT Press, 1986.

[15] George E. P. Box and Norman Richard Draper. *Empirical model-building and response surfaces*. John Wiley & Sons, 1987.

[16] Carlos M. Jarque and Anil K. Bera. 'A Test for Normality of Observations and Regression Residuals'. In: *International Statistical Review* 55.2 (1987), pp. 163–172.

[17] Lawrence C. Paulson. 'Isabelle: The Next Seven Hundred Theorem Provers'. In: *Proceedings of the 9th International Conference on Automated Deduction*. 1988, pp. 772–773.

[18] V. A. Voevodsky and M. M. Kapranov. '∞-groupoids as a model for a homotopy category'. In: *Uspekhi Mat. Nauk* 45 (1990). *Warning: the main theorem of this paper was shown by Carlos Simpson to be false.*, pp. 183–184.

[19] Ronald L. Graham, Donald E. Knuth and Oren Patashnik. *Concrete Mathematics: A Foundation for Computer Science*. 2nd ed. Addison-Wesley, 1994.

[20] Witold Marciszewski and Roman Murawski. *Mechanization of reasoning in a historical perspective*. Vol. 43. Poznań studies in the philosophy of the sciences and the humanities. Rodopi, 1995.

[21] D. Price. 'Pentium FDIV flaw-lessons learned'. In: *IEEE Micro* 15.2 (1995), pp. 86–88.

[22] Edmund K. Burke and Eric Foxley. *Logic and its applications*. Prentice Hall series in computer science. Prentice Hall, 1996.

[23] John Harrison. *Formalized Mathematics*. Technical Report 36. Turku Centre for Computer Science (TUCS), 1996.

[24] Radford M Neal. *Bayesian Learning for Neural Networks*. Vol. 118. Lecture notes in statistics. New York: Springer, 1996.

[25] Marko Petkovšek. *A=B*. (Forward by Donald E. Knuth). Wellesley, Mass: A K Peters, 1996.

[26] Chris Thornton. 'Parity: The problem that won't go away'. In: *Advances in Artifical Intelligence*. 1996, pp. 362–374.

[27] Robert Veroff. 'Using hints to increase the effectiveness of an automated reasoning program: Case studies'. In: *Journal of Automated Reasoning* 16.3 (1996), pp. 223–239.

[28] Lawrence C. Paulson. 'Proving Properties of Security Protocols by Induction'. In: *10th Computer Security Foundations Workshop (CSFW)*. 1997, pp. 70–83.

[29] Tanel Tammet. 'Gandalf'. In: *Journal of Automated Reasoning* 18.2 (1997), pp. 199–204.

[30] David H. Wolpert and William G. Macready. 'No free lunch theorems for optimization'. In: *IEEE Transactions on Evolutionary Computation* 1.1 (1997), pp. 67–82.

[31] Alan Bundy. 'A Science of Reasoning (Extended Abstract)'. In: *Automated Reasoning with Analytic Tableaux and Related Methods, International Conference, TABLEAUX*. Vol. 1397. Lecture Notes in Computer Science. 1998, pp. 10–17.

[32] Carlos Simpson. *Homotopy types of strict 3-groupoids*. arXiv preprint `math.CT/9810059`. 1998.

[33] Alfred Tarski. 'A Decision Method for Elementary Algebra and Geometry'. In: *Quantifier Elimination and Cylindrical Algebraic Decomposition*. Texts and Monographs in Symbolic Computation. 1998, pp. 24–84.

[34] Michael Dummett. *Elements of Intuitionism*. 2nd ed. Clarendon Press, 2000.

[35] Mike Gordon. 'From LCF to HOL: A Short History'. In: *Proof, Language, and Interaction: Essays in Honour of Robin Milner*. 2000, pp. 169–185.

[36] Gareth O. Roberts and Jeffrey S. Rosenthal. 'Optimal Scaling for Various Metropolis-Hastings Algorithms'. In: *Statistical Science* 16.4 (2001), pp. 351–367.

[37] Michael E Tipping. 'Sparse Bayesian learning and the relevance vector machine'. In: *Journal of machine learning research* 1 (2001), pp. 211–244.

[38] Stephan Schulz. 'E — a Brainiac Theorem Prover'. In: *AI Communications* 15.2,3 (2002), pp. 111–126.

[39] G. Tassey. *The Economic Impacts of Inadequate Infrastructure for Software Testing: Final Report*. Diane Publishing Company, 2002.

[40] Niklas Eén and Niklas Sörensson. 'An Extensible SAT-solver'. In: *Theory and Applications of Satisfiability Testing, 6th International Conference, SAT*. Vol. 2919. Lecture Notes in Computer Science. 2003, pp. 502–518.

[41] Anthony Fox. 'Formal Specification and Verification of ARM6'. In: *Theorem Proving in Higher Order Logics*. 2003, pp. 25–40.

[42] Ian Horrocks, Peter F. Patel-Schneider and Frank van Harmelen. 'From SHIQ and RDF to OWL: the making of a Web Ontology Language'. In: *Journal of Web Semantics* 1.1 (2003), pp. 7–26.

[43] Kevin Leyton-Brown et al. 'A Portfolio Approach to Algorithm Selection'. In: *International Joint Conference on Artificial Intelligence*. 2003, pp. 1542–1543.

[44] Edmund Clarke, Daniel Kroening and Flavio Lerda. 'A Tool for Checking ANSI-C Programs'. In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Vol. 2988. Lecture Notes in Computer Science. 2004, pp. 168–176.

[45] Michael Huth and Mark Ryan. *Logic in Computer Science: Modelling and Reasoning about Systems*. 2nd ed. Cambridge University Press, 2004.

[46] Hoa G. Nguyen et al. 'Segway robotic mobility platform'. In: *Mobile Robots XVII*. Vol. 5609. 2004, pp. 207–220.

[47] J Shawe-Taylor. *Kernel methods for pattern analysis*. Cambridge University Press, 2004.

[48] Josef Urban. 'MPTP — Motivation, Implementation, First Experiments'. In: *Journal of Automated Reasoning* 33.3-4 (2004), pp. 319–339.

[49] Larry Wasserman. *All of Statistics: A Concise Course in Statistical Inference*. Springer, 2004.

[50] Torkel Franzén. *Gödel's Theorem: An Incomplete Guide to Its Use and Abuse*. Wellesley, MA: A K Peters, 2005.

[51] Thomas C. Hales. 'Introduction to the Flyspeck Project'. In: *Mathematics, Algorithms, Proofs* 5021 (2005), pp. 1–11.

[52] John Harrison. 'Floating-Point Verification'. In: *Formal Methods*. 2005, pp. 529–532.

[53] Roman Matuszewski and Piotr Rudnicki. 'Mizar: the first 30 years'. In: *Mechanized Mathematics and Its Applications* 4 (2005).

[54] Iain Murray and Zoubin Ghahramani. *A note on the evidence and Bayesian Occam's razor*. Tech. rep. GCNU-TR 2005-003. Gatsby Computational Neuroscience Unit, University College London, 2005.

[55] Jens Otten. 'Clausal Connection-Based Theorem Proving in Intuitionistic First-Order Logic'. In: *Automated Reasoning with Analytic Tableaux and Related Methods, International Conference*. Vol. 3702. Lecture Notes in Computer Science. 2005, pp. 245–261.

[56] Peter E Rossi, Greg M Allenby and Robert E McCulloch. *Bayesian Statistics and Marketing*. Hoboken, NJ: Wiley, 2005.

[57] Aarti Gupta, Malay K. Ganai and Chao Wang. 'SAT-Based Verification Methods and Applications in Hardware Verification'. In: *Formal Methods for Hardware Verification*. 2006, pp. 108–143.

[58] Ping Li, Trevor Hastie and Kenneth Ward Church. 'Very sparse random projections'. In: *Proceedings of the Twelfth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2006, pp. 287–296.

[59] Ha Quang Minh, Partha Niyogi and Yuan Yao. 'Mercer's Theorem, Feature Maps, and Smoothing'. In: *19th Annual Conference on Learning Theory, COLT, Proceedings*. Vol. 4005. Lecture Notes in Computer Science. 2006, pp. 154–168.

[60] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian processes for machine learning*. Adaptive computation and machine learning. MIT Press, 2006.

[61] Dmitry Tsarkov and Ian Horrocks. 'FaCT++ Description Logic Reasoner: System Description'. In: *Automated Reasoning*. 2006, pp. 292–297.

[62] Zhe Cao et al. 'Learning to Rank: From Pairwise Approach to Listwise Approach'. In: *Machine Learning, Proceedings of the Twenty-Fourth International Conference (ICML)*. Vol. 227. ACM International Conference Proceeding Series. 2007, pp. 129–136.

[63] Frank Hutter, Holger H. Hoos and Thomas Stützle. 'Automatic Algorithm Configuration Based on Local Search'. In: *Proceedings of the Twenty-Second AAAI Conference on Artificial Intelligence*. 2007, pp. 1152–1157.

[64] Frank Hutter et al. 'Boosting Verification by Automatic Tuning of Decision Procedures'. In: *Formal Methods in Computer-Aided Design, 7th International Conference, FMCAD*. 2007, pp. 27–34.

[65] Olga Ohrimenko, Peter J. Stuckey and Michael Codish. 'Propagation = Lazy Clause Generation'. In: *Principles and Practice of Constraint Programming*. 2007, pp. 544–558.

[66] David P. Wipf and Srikantan S. Nagarajan. 'A New View of Automatic Relevance Determination'. In: *Twenty-First Annual Conference on Neural Information Processing*. 2007, pp. 1625–1632.

[67] John Harrison. 'Formal Proof – Theory and Practice'. In: *Notices of the American Mathematical Society* 55 (2008), pp. 1395–1406.

[68] Jens Otten. 'leanCoP 2.0 and ileanCoP 1.2: High Performance Lean Theorem Proving in Classical and Intuitionistic Logic (System Descriptions)'. In: *Automated Reasoning*. 2008, pp. 283–291.

[69] André Platzer and Jan-David Quesel. 'KeYmaera: A Hybrid Theorem Prover for Hybrid Systems (System Description)'. In: *Automated Reasoning*. 2008, pp. 171–178.

[70] Konrad Slind and Michael Norrish. 'A Brief Overview of HOL4'. In: *Theorem Proving in Higher Order Logics*. 2008, pp. 28–32.

[71] Thomas H. Cormen et al. *Introduction to Algorithms*. 3rd ed. MIT Press, 2009.

[72] John Guiver and Edward Lloyd Snelson. 'Bayesian inference for Plackett-Luce ranking models'. In: *Proceedings of the 26th Annual International Conference on Machine Learning, ICML*. Vol. 382. 2009, pp. 377–384.

[73] John Harrison. *Handbook of practical logic and automated reasoning*. Cambridge University Press, 2009.

[74] John Harrison. 'HOL Light: An Overview'. In: *Theorem Proving in Higher Order Logics*. 2009, pp. 60–66.

[75] Randall Hyde. 'The Fallacy of Premature Optimization'. In: *Ubiquity* (Feb. 2009).

[76] Adam Naumowicz and Artur Korniłowicz. 'A Brief Overview of Mizar'. In: *Theorem Proving in Higher Order Logics*. Vol. 5674. Lecture Notes in Computer Science. 2009, pp. 67–72.

[77] Carsten Sinz and Markus Iser. 'Problem-Sensitive Restart Heuristics for the DPLL Procedure'. In: *Theory and Applications of Satisfiability Testing*. Vol. 5584. Lecture Notes in Computer Science. 2009, pp. 356–362.

[78] Weiwei Cheng, Krzysztof Dembczynski and Eyke Hüllermeier. 'Label Ranking Methods based on the Plackett-Luce Model'. In: *International Conference on Machine Learning*. 2010, pp. 215–222.

[79] Brian S. Everitt and Anders Skrondal. *The Cambridge Dictionary of Statistics*. 4th ed. Cambridge University Press, 2010.

[80] Jens Otten. 'Restricting backtracking in connection calculi'. In: *AI Communications* 23.2-3 (2010), pp. 159–182.

[81] Jens Otten and Geoff Sutcliffe. 'Using the TPTP language for representing derivations in tableau and connection calculi'. In: *Proceedings of the Workshop on Practical Aspects of Automated Reasoning, 5th International Joint Conference on Automated Reasoning*. 2010, pp. 90–100.

[82] Bryan Silverthorn and Risto Miikkulainen. 'Latent Class Models for Algorithm Portfolio Methods'. In: *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence*. 2010, pp. 167–172.

[83] Jasmin Christian Blanchette, Lukas Bulwahn and Tobias Nipkow. 'Automatic Proof and Disproof in Isabelle/HOL'. In: *Frontiers of Combining Systems*. 2011, pp. 12–27.

[84] Malka Gorfine, Ruth Heller and Yair Heller. *Comment on "Detecting Novel Associations in Large Data Sets"*. 2011. URL: `https://web.archive.org/web/20170808213131/https://ie.technion.ac.il/~gorfinm/files/science6.pdf` (visited on 02/01/2023).

[85] Kryštof Hoder and Andrei Voronkov. 'Sine Qua Non for Large Theory Reasoning'. In: *CADE-23: 23rd International Conference on Automated Deduction. Proceedings*. 2011, pp. 299–314.

[86] Frank Hutter, Holger H. Hoos and Kevin Leyton-Brown. 'Sequential Model-Based Optimization for General Algorithm Configuration'. In: *Learning and Intelligent Optimization*. 2011, pp. 507–523.

[87] Adam Lally and Paul Fodor. *Natural Language Processing With Prolog in the IBM Watson System*. 31st Mar. 2011. URL: `www.cs.nmsu.edu/ALP/2011/03/natural-language-processing-with-prolog-in-the-ibm-watson-system/` (visited on 25/03/2022).

[88] David N. Reshef et al. 'Detecting Novel Associations in Large Data Sets'. In: *Science* 334.6062 (2011), pp. 1518–1524.

[89] Noah Simon and Robert Tibshirani. *Comment on "Detecting Novel Associations in Large Data Sets"*. 2011. URL: `https://web.archive.org/web/20141009075124/http://statweb.stanford.edu/~tibs/reshef/comment.pdf` (visited on 02/01/2023).

[90] D. Barber. *Bayesian Reasoning and Machine Learning*. Cambridge University Press, 2012.

[91] Pedro M. Domingos. 'A few useful things to know about machine learning'. In: *Communications of the ACM* 55.10 (2012), pp. 78–87.

[92] John L. Hennessy and David A. Patterson. *Computer Architecture - A Quantitative Approach, 5th Edition*. Morgan Kaufmann, 2012.

[93] Daniel Kühlwein et al. 'Overview and Evaluation of Premise Selection Techniques for Large Theory Mathematics'. In: *Automated Reasoning - 6th International Joint Conference, IJCAR*. Vol. 7364. Lecture Notes in Computer Science. 2012, pp. 378–392.

[94] A. Lally et al. 'Question analysis: How Watson reads a clue'. In: *IBM Journal of Research and Development* 56.3.4 (2012), 2:1–2:14.

[95] Kevin P Murphy. *Machine Learning: A Probabilistic Perspective*. MIT press, 2012.

[96] Lawrence C. Paulson and Jasmin Christian Blanchette. 'Three years of experience with Sledgehammer, a Practical Link Between Automatic and Interactive Theorem Provers'. In: *The 8th International Workshop on the Implementation of Logics (IWIL)*. Vol. 2. EPiC Series in Computing. 2012, pp. 1–11.

[97] André Platzer. 'Logics of Dynamical Systems'. In: *2012 27th Annual IEEE Symposium on Logic in Computer Science*. 2012, pp. 13–24.

[98] Joachim Schimpf and Kish Shen. 'ECL^{i}PS^{e} — From LP to CLP'. In: *Theory and Practice of Logic Programming* 12.1-2 (2012), pp. 127–156.

[99] Jasper Snoek, Hugo Larochelle and Ryan P. Adams. 'Practical Bayesian Optimization of Machine Learning Algorithms'. In: *Advances in Neural Information Processing Systems 25, Proceedings*. 2012, pp. 2960–2968.

[100] Jan Wielemaker et al. 'SWI-Prolog'. In: *Theory and Practice of Logic Programming* 12.1-2 (2012), pp. 67–96.

[101] Ankush Desai et al. 'P: Safe Asynchronous Event-Driven Programming'. In: *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI*. 2013, pp. 321–332.

[102] Daniel Kühlwein et al. 'MaSh: Machine Learning for Sledgehammer'. In: *Interactive Theorem Proving - 4th International Conference, ITP*. Vol. 7998. Lecture Notes in Computer Science. 2013, pp. 35–50.

[103] Nikhil Swamy et al. 'Verifying higher-order programs with the Dijkstra monad'. In: *ACM SIGPLAN Notices* 48.6 (2013), pp. 387–398.

[104] Miltiadis Allamanis and Charles Sutton. 'Mining Idioms from Source Code'. In: *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, (FSE-22)*. 2014, pp. 472–483.

[105] Jeremy Avigad and John Harrison. 'Formally Verified Mathematics'. In: *Communications of the ACM* 57.4 (Apr. 2014), pp. 66–75.

[106] Armin Biere. 'Lingeling Essentials, A Tutorial on Design and Implementation Aspects of the SAT Solver Lingeling'. In: *POS-14. Fifth Pragmatics of SAT workshop*. Vol. 27. EPiC Series in Computing. 2014, p. 88.

[107] James P. Bridge, Sean B. Holden and Lawrence C. Paulson. 'Machine Learning for First-Order Theorem Proving'. In: *Journal of Automated Reasoning* 53.2 (2014), pp. 141–172.

[108] Cezary Kaliszyk and Josef Urban. 'Learning-Assisted Automated Reasoning with Flyspeck'. In: *Journal of Automated Reasoning* 53.2 (Aug. 2014), pp. 173–213.

[109] Cezary Kaliszyk, Josef Urban and Jiří Vyskočil. *Machine Learner for Automated Reasoning 0.4 and 0.5*. arXiv preprint `cs.LG/1402.2359`. 2014.

[110] D. Kühlwein and J.C. Blanchette. 'A Survey of Axiom Selection as a Machine Learning Problem'. In: *Infinity, Computability and Metamathematics : Festschrift celebrating the 60th birthdays of Peter Koepke and Philip Welch*. 2014, pp. 1–15.

[111] K. Rustan M. Leino and Michał Moskal. 'Co-induction Simply'. In: *FM 2014: Formal Methods*. 2014, pp. 382–398.

[112] Jasmin Christian Blanchette et al. 'Mining the Archive of Formal Proofs'. In: *Intelligent Computer Mathematics - International Conference, CICM*. Vol. 9150. Lecture Notes in Computer Science. 2015, pp. 3–17.

[113] Cezary Kaliszyk and Josef Urban. 'HOL(y)Hammer: Online ATP Service for HOL Light'. In: *Mathematics in Computer Science* 9.1 (2015), pp. 5–22.

[114] Daniel Kühlwein and Josef Urban. 'MaLeS: A Framework for Automatic Tuning of Automated Theorem Provers'. In: *Journal of Automated Reasoning* 55.2 (2015), pp. 91–116.

[115] Hakjoo Oh, Hongseok Yang and Kwangkeun Yi. 'Learning a strategy for adapting a program analysis via Bayesian optimisation'. In: *ACM SIGPLAN Notices* 50.10 (2015), pp. 572–588.

[116] Lawrence C. Paulson. 'A Mechanised Proof of Gödel's Incompleteness Theorems Using Nominal Isabelle'. In: *Journal of Automated Reasoning* 55.1 (2015), pp. 1–37.

[117] Simon Schäfer and Stephan Schulz. 'Breeding Theorem Proving Heuristics with Genetic Algorithms'. In: *Global Conference on Artificial Intelligence, GCAI*. Vol. 36. EPiC Series in Computing. 2015, pp. 263–274.

[118] Bobak Shahriari et al. 'Taking the human out of the loop: A review of Bayesian optimization'. In: *Proceedings of the IEEE* 104.1 (2015), pp. 148–175.

[119] Jasmin Christian Blanchette et al. 'A Learning-Based Fact Selector for Isabelle/HOL'. In: *Journal of Automated Reasoning* 57.3 (2016), pp. 219–244.

[120] Michael Färber, Cezary Kaliszyk and Josef Urban. 'Monte Carlo Tableau Proof Search'. In: *Proceedings of the 26th International Conference on Automated Deduction, CADE* (Nov. 2016).

[121] Krystof Hoder et al. 'Selecting the Selection'. In: *Automated Reasoning - 8th International Joint Conference, IJCAR*. Vol. 9706. Lecture Notes in Computer Science. 2016, pp. 313–329.

[122] Geoffrey Irving et al. 'DeepMath - Deep Sequence Models for Premise Selection'. In: *Advances in Neural Information Processing Systems 29*. 2016, pp. 2235–2243.

[123] Thomas Hales et al. 'A Formal Proof Of The Kepler Conjecture'. In: *Forum of Mathematics, Pi* 5 (2017).

[124] Jan Jakubův and Josef Urban. 'ENIGMA: Efficient Learning-Based Inference Guiding Machine'. In: *Intelligent Computer Mathematics — 10th International Conference, Proceedings*. Vol. 10383. Lecture Notes in Computer Science. 2017, pp. 292–302.

[125] Sarah Loos et al. 'Deep Network Guided Proof Search'. In: *LPAR-21. 21st International Conference on Logic for Programming, Artificial Intelligence and Reasoning*. Vol. 46. EPiC Series in Computing. 2017, pp. 85–105.

[126] Giles Reger and Martin Suda. 'Measuring progress to predict success: Can a good proof strategy be evolved?' In: 2nd Conference on Artificial Intelligence and Theorem Proving. ABSTRACTS: `https://aitp-conference.org/2017/aitp17-proceedings.pdf`. 2017.

[127] G. Sutcliffe. 'The TPTP Problem Library and Associated Infrastructure. From CNF to TH0, TPTP v6.4.0'. In: *Journal of Automated Reasoning* 59.4 (2017), pp. 483–502.

[128] Mingzhe Wang et al. *Premise Selection for Theorem Proving by Deep Graph Embedding*. arXiv preprint `cs.AI/1709.09994`. 2017.

[129] J. Backes et al. 'Semantic-based Automated Reasoning for AWS Access Policies using SMT'. In: *Formal Methods in Computer Aided Design (FMCAD)*. 2018, pp. 1–9.

[130] Mislav Balunovic, Pavol Bielik and Martin T. Vechev. 'Learning to Solve SMT Formulas'. In: *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems, NeurIPS*. 2018, pp. 10338–10349.

[131] Douglas Bridges and Erik Palmgren. 'Constructive Mathematics'. In: *The Stanford Encyclopedia of Philosophy*. Summer. 2018.

[132] Andrey Chudnov et al. 'Continuous Formal Verification of Amazon s2n'. In: *Computer Aided Verification*. 2018, pp. 430–446.

[133] Byron Cook. 'Formal Reasoning About the Security of Amazon Web Services'. In: *Computer Aided Verification*. 2018, pp. 38–47.

[134] Byron Cook et al. 'Model Checking Boot Code from AWS Data Centers'. In: *Computer Aided Verification*. 2018, pp. 467–486.

[135] Ankush Desai et al. 'Compositional programming and testing of dynamic distributed systems'. In: *Proceedings of the ACM on Programming Languages* 2 (2018), 159:1–159:30.

[136] Dirk Schäfer and Eyke Hüllermeier. 'Dyad ranking using Plackett–Luce models based on joint feature representations'. In: *Machine Learning* 107.5 (2018), pp. 903–941.

[137] Zohreh Shams et al. 'iCon: A Diagrammatic Theorem Prover for Ontologies'. In: *Principles of Knowledge Representation and Reasoning*. 2018, pp. 204–209.

[138] R. S. Sutton and A. G. Barto. *Reinforcement Learning, second edition: An Introduction*. Adaptive Computation and Machine Learning series. MIT Press, 2018.

[139] Takuya Akiba et al. 'Optuna: A Next-generation Hyperparameter Optimization Framework'. In: *International Conference on Knowledge Discovery and Data Mining*. 2019.

[140] Saeed Kayhanian et al. 'Modelling outcomes after paediatric brain injury with admission laboratory values: a machine-learning approach'. In: *Pediatric Research* 86.5 (Nov. 2019), pp. 641–645.

[141] David Reutter and Jamie Vicary. 'High-level methods for homotopy construction in associative n-categories'. In: *34th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS*. 2019, pp. 1–13.

[142] B. Jack Copeland. 'The Church-Turing Thesis'. In: *The Stanford Encyclopedia of Philosophy*. Summer. 2020.

[143] Pablo Duboue. *The Art of Feature Engineering: Essentials for Machine Learning*. Cambridge University Press, 2020.

[144] Eduardo C. Garrido-Merchán and Daniel Hernández-Lobato. 'Dealing with categorical and integer-valued variables in Bayesian Optimization with Gaussian processes'. In: *Neurocomputing* 380 (2020), pp. 20–35.

[145] Stephen R. Johnson, Daniel A. Henderson and Richard J. Boys. *On Bayesian inference for the Extended Plackett-Luce model*. arXiv preprint `stat.AP/2002.05953`. 2020.

[146] Chaitanya Mangla, Sean B. Holden and Lawrence C. Paulson. 'Bayesian Optimisation of Solver Parameters in CBMC'. In: *18th International Workshop on Satisfiability Modulo Theories*. Vol. 2854. CEUR Workshop Proceedings. 2020, pp. 37–47.

[147] Agnieszka Słowik et al. 'Bayesian Optimisation for Heuristic Configuration in Automated Theorem Proving'. In: *5th and 6th Vampire Workshops*. Vol. 71. EPiC Series in Computing. 2020, pp. 45–51.

[148] Grady Booch et al. 'Thinking Fast and Slow in AI'. In: *Thirty-Fifth AAAI Conference on Artificial Intelligence*. 2021, pp. 15042–15046.

[149] Karel Chvalovský and Jan Jakubův. 'Characteristic Subsets of TPTP Benchmarks'. In: 6th Conference on Artificial Intelligence and Theorem Proving. ABSTRACTS: `https://aitp-conference.org/2021/aitp21-proceedings.pdf`. 2021.

[150] Michael Färber, Cezary Kaliszyk and Josef Urban. 'Machine learning guidance for connection tableaux'. In: *Journal of automated reasoning* (2021).

[151] Sean B. Holden. 'Machine Learning for Automated Theorem Proving: Learning to Solve SAT and QSAT'. In: *Foundations and Trends in Machine Learning* 14.6 (2021), pp. 807–989.

[152] Jan Hůla, David Mojžíšek and Mikoláš Janota. 'Graph Neural Networks for Scheduling of SMT Solvers'. In: *33rd IEEE International Conference on Tools with Artificial Intelligence, ICTAI*. 2021, pp. 447–451.

[153] Joan Moschovakis. 'Intuitionistic Logic'. In: *The Stanford Encyclopedia of Philosophy*. Fall. 2021.

[154] Nikhil Pimpalkhare et al. 'MedleySolver: Online SMT Algorithm Selection'. In: *Theory and Applications of Satisfiability Testing*. 2021, pp. 453–470.

[155] Joseph Scott et al. 'MachSMT: A Machine Learning-based Algorithm Selector for SMT Solvers'. In: *Tools and Algorithms for the Construction and Analysis of Systems*. 2021, pp. 303–325.

[156] Walid Mohamed Taha, Abd-Elhamid M. Taha and Johan Thunberg. *Cyber-Physical Systems: A Model-Based Approach*. Springer, 2021.

[157] Martin H. Trauth. *MATLAB® Recipes for Earth Sciences*. Springer, 2021.

[158] Jan Hula et al. 'Targeted Configuration of an SMT Solver'. In: *Intelligent Computer Mathematics*. Vol. 13467. Lecture Notes in Computer Science. 2022, pp. 256–271.

[159] Chaitanya Mangla. *BRASS Source Code*. Version v1.0.0. DOI: 10.5281/zenodo.6028568. Feb. 2022.

[160] Chaitanya Mangla, Sean B. Holden and Lawrence C. Paulson. 'Bayesian Ranking for Strategy Scheduling in Automated Theorem Provers'. In: *Automated Reasoning — 11th International Joint Conference, IJCAR*. Vol. 13385. Lecture Notes in Computer Science. 2022, pp. 559–577.

[161] Ray Monk. *Bertrand Russell*. Encyclopedia Britannica. 29th Jan. 2022. URL: www.britannica.com/biography/Bertrand-Russell (visited on 23/03/2022).

[162] The Coq Development Team. *The Coq Proof Assistant*. Version 8.15. DOI: 10.5281/zenodo.5846982. Jan. 2022.